

AI untuk Programming dan Coding Cepat

Python, Web Programming, dan Android untuk Pemula

Onno W. Purbo
onno@indo.net.id
onno@itts.ac.id
Twitter onnowpurbo

Institut Teknologi Tangerang Selatan
Dinas KOMIFO Tangerang Selatan
OnnoCenter
2026

Lisensi: Karya ini dilisensikan di bawah **Creative Commons Attribution (CC BY) 4.0 International**. Anda bebas menggunakan, membagikan, dan mengadaptasi karya ini untuk tujuan apa pun, termasuk komersial, dengan tetap mencantumkan atribusi yang layak kepada penulis.

Disclaimer: Buku ini disusun untuk tujuan edukasi, dokumentasi, dan refleksi publik berdasarkan sumber-sumber yang dianggap kredibel pada saat penulisan. Penulis telah berupaya menjaga akurasi, namun sangat mungkin masih terdapat kekurangan atau kekeliruan; karena itu, buku ini tidak dimaksudkan sebagai satu-satunya rujukan yang final, dan penulis terbuka terhadap koreksi serta masukan yang membangun

Kata Pengantar

Alhamdulillah, buku **AI untuk Programming dan Coding Cepat: Python, Web Programming, dan Android untuk Pemula** ini dapat disusun sebagai panduan praktis bagi pemula, mahasiswa, dan programmer awal yang ingin cepat memulai coding dengan bantuan AI.

Buku ini menekankan prinsip sederhana: **AI adalah asisten, bukan pengganti programmer**. AI dapat membantu membuat contoh kode, menjelaskan error, merapikan program, membuat dokumentasi, dan mempercepat belajar. Namun, **pemahaman, pengujian, keamanan, dan keputusan akhir tetap menjadi tanggung jawab manusia**.

Isi buku disusun dengan pendekatan **baca sedikit, praktik banyak**. Pembaca diajak menyiapkan ekosistem open-source, belajar Python, membuat otomasi sederhana, membangun API dengan FastAPI, membuat frontend web, mencoba Android, hingga menyusun proyek akhir Todo sederhana.

Buku ini ditulis dengan bantuan AI, tetapi **penulis tetap bertanggung jawab penuh atas isi dan arah pembahasan**. Mohon maaf jika masih ada kekurangan atau kekeliruan. Semoga buku ini bermanfaat dan membantu pembaca menjadi programmer yang **mandiri, kritis, produktif, dan tidak sekadar copy-paste kode AI**.

Jakarta, Mei 2026

Penulis

Daftar Isi

Kata Pengantar.....	3
Daftar Isi.....	4
Cara Menggunakan Buku.....	10
Langkah Menggunakan Buku.....	10
Pola Belajar Buku Ini.....	10
Aturan Utama.....	11
BAB 1. Mindset Coding Cepat dengan AI.....	12
1.1 AI sebagai Asisten, Bukan Pengganti Programmer.....	12
1.2 Dari Vibe Coding ke Coding yang Bertanggung Jawab.....	13
1.3 Pola Kerja Cepat.....	14
1.4 Contoh Prompt Dasar.....	16
1.5 Hands-on: Program Python Pertama dengan Bantuan AI.....	17
Langkah 1 — Buat Folder Proyek.....	17
Langkah 2 — Minta AI Membuat Kode.....	17
Langkah 3 — Jalankan Kode.....	19
Langkah 4 — Uji Input Salah.....	19
Langkah 5 — Perbaiki Kode dengan Error Handling.....	20
Langkah 6 — Buat Test Manual.....	21
Langkah 7 — Dokumentasikan Program.....	22
Langkah 8 — Simpan dengan Git.....	22
Checklist Akhir Bab 1.....	23
Ringkasan Bab.....	23
BAB 2. Menyiapkan Ekosistem Coding Open-Source.....	24
2.1 Sistem Operasi dan Terminal.....	24
2.2 Editor Code.....	25
2.3 Python, Git, dan Project Folder.....	26
Menenal Git.....	27
2.4 Manajemen Python dengan uv.....	28
2.5 AI Lokal dengan Ollama.....	30
2.6 AI Coding Assistant Open-Source.....	31
2.7 Docker untuk Lingkungan Konsisten.....	33
2.8 Hands-on: Setup Ekosistem Coding.....	35
Langkah 1 — Install Paket Dasar.....	35
Langkah 2 — Install uv.....	35
Langkah 3 — Buat Program Python.....	35
Langkah 4 — Simpan dengan Git.....	36
Langkah 5 — Install dan Coba Ollama.....	36
Langkah 6 — Install Aider.....	36
Langkah 7 — Uji Docker.....	37
Checklist Akhir Bab 2.....	37
Ringkasan Bab.....	38
BAB 3. Python Dasar dengan Bantuan AI.....	39

3.1 Mengetahui Python.....	39
Menjalankan File .py.....	39
Cara Membaca Pesan Error.....	40
3.2 Dasar Sintaks Python.....	41
Variabel.....	41
Tipe Data Dasar.....	41
Kondisi: if, elif, else.....	42
Perulangan: for dan while.....	42
Fungsi: def, Parameter, dan return.....	43
3.3 Struktur Data.....	44
List.....	44
Dict.....	44
Tuple.....	44
Set.....	45
Latihan Struktur Data.....	45
3.4 Error Handling.....	46
Membaca Traceback.....	47
3.5 AI untuk Belajar Python.....	48
3.6 Hands-on.....	49
Hands-on 1 — Program Kalkulator Sederhana.....	49
Hands-on 2 — Program Konversi Suhu.....	50
Hands-on 3 — Program Manajemen Catatan Berbasis Terminal.....	51
Hands-on 4 — Membaca dan Menulis File Teks.....	52
Hands-on 5 — Membaca File CSV Sederhana.....	53
Mini Proyek Bab 3 — Ringkasan Nilai Mahasiswa.....	55
Checklist Akhir Bab 3.....	56
Ringkasan Bab.....	57
BAB 4. Python untuk Otomasi dan Data Sederhana.....	58
4.1 Python untuk Pekerjaan Nyata.....	58
4.2 Mengolah Data Sederhana.....	59
4.3 Mengambil Data dari API.....	60
4.4 AI untuk Membantu Script.....	61
4.5 Hands-on.....	62
Hands-on 1 — Rename File Otomatis.....	62
Hands-on 2 — Membuat Laporan Ringkas dari CSV.....	63
Hands-on 3 — Versi Ringkas dengan pandas.....	66
Hands-on 4 — Mengambil Data dari API Publik.....	67
Hands-on 5 — Script Backup Folder Sederhana.....	68
Hands-on 6 — Meminta AI Memperbaiki Script yang Error.....	69
Mini Proyek Bab 4 — Laporan Otomatis Pengeluaran.....	70
Checklist Akhir Bab 4.....	72
Ringkasan Bab.....	72
BAB 5. Web Programming Cepat dengan Python dan FastAPI.....	73
5.1 Konsep Dasar Web.....	73

5.2 Mengapa FastAPI.....	74
5.3 Struktur Proyek FastAPI.....	75
5.4 Routing Dasar.....	76
5.5 Database Ringan dengan SQLite.....	77
5.6 Testing Manual API.....	80
Test GET Home.....	80
Test Tambah Todo.....	80
Test Lihat Todo.....	80
Test Update Todo.....	80
Test Hapus Todo.....	80
5.7 Hands-on.....	81
Hands-on 1 — API Hello World.....	81
Hands-on 2 — API Todo List Tanpa Database.....	82
Hands-on 3 — API Todo + SQLite.....	83
Hands-on 4 — Prompt AI untuk Membuat Endpoint Baru.....	84
Hands-on 5 — Debug Error API.....	85
Mini Proyek Bab 5 — Todo API Sederhana.....	86
Checklist Akhir Bab 5.....	87
Ringkasan Bab.....	87
BAB 6. Web Frontend Sederhana untuk Pemula.....	88
6.1 HTML, CSS, dan JavaScript Secukupnya.....	88
6.2 Frontend Bertemu Backend.....	89
6.3 AI untuk Membuat Tampilan Awal.....	91
6.4 Hands-on.....	92
Hands-on 1 — Siapkan Backend FastAPI.....	92
Hands-on 2 — Membuat Halaman Todo.....	94
Hands-on 3 — Jalankan Frontend dari CLI.....	97
Hands-on 4 — Test API dari CLI.....	97
Hands-on 5 — Memperbaiki Tampilan dengan CSS.....	98
Hands-on 6 — Debug Error fetch().....	99
6.5 Alternatif Ringan: HTMX.....	100
Mini Proyek Bab 6 — Todo Web Client.....	101
Ringkasan Bab.....	102
BAB 7. Android Programming dengan AI.....	103
7.1 Konsep Dasar Android.....	103
7.2 Kotlin Dasar untuk Pemula.....	104
Variabel.....	104
Fungsi.....	104
Class Sederhana.....	104
List.....	105
Lambda Sederhana.....	105
7.3 Jetpack Compose.....	106
7.4 Komponen UI Dasar.....	107
7.5 Android App Sederhana.....	108

7.6 AI untuk Android.....	109
7.7 Hands-on.....	111
Hands-on 1 — Membuat Project Android.....	111
Hands-on 2 — Membuat Tampilan Sederhana.....	112
Hands-on 3 — Menambahkan Input dan Tombol.....	113
Hands-on 4 — Menampilkan List Todo.....	115
Hands-on 5 — Meminta AI Menjelaskan Error Build.....	117
Hands-on 6 — Meminta AI Merapikan UI.....	118
Mini Proyek Bab 7 — Todo Android Lokal.....	119
Ringkasan Bab.....	120
BAB 8. Proyek Akhir: Aplikasi Todo AI-Ready.....	121
8.1 Tujuan Proyek.....	121
8.2 Arsitektur Sederhana.....	121
8.3 Fitur Minimal.....	122
8.4 Peran AI dalam Proyek.....	123
8.5 Hands-on Bertahap.....	124
Langkah 1 — Membuat Repo Git.....	124
Langkah 2 — Membuat Backend.....	125
Langkah 3 — Menguji Endpoint Backend.....	128
Langkah 4 — Membuat Frontend Web.....	129
Langkah 5 — Android Client Sederhana.....	132
Langkah 6 — Menulis Dokumentasi Proyek.....	134
Langkah 7 — Membuat Screenshot Hasil.....	135
Langkah 8 — Commit Final.....	135
8.6 Checklist Proyek.....	136
Prompt AI untuk Audit Proyek.....	137
Ringkasan Bab.....	138
BAB 9. Strategi Belajar, Debugging, Testing, dan Anti-Halusinasi Kode.....	139
9.1 Workflow Harian Programmer AI.....	139
9.2 Prompt Coding yang Baik.....	140
9.3 Debugging dengan AI.....	141
9.4 Testing Kode AI.....	142
Contoh Unit Test Sederhana.....	142
Jenis Test Minimal.....	143
9.5 Anti-Halusinasi Kode.....	144
9.6 Kesalahan Umum Pemula.....	146
9.7 Roadmap Setelah Buku Ini.....	147
Tahap 1 — Perkuat Dasar.....	147
Tahap 2 — Web dan Database.....	147
Tahap 3 — Lingkungan Produksi.....	147
Tahap 4 — Android Lanjut.....	147
Tahap 5 — AI Lokal dan RAG.....	148
Tahap 6 — Keamanan dan Kualitas.....	148
Hands-on Penutup — Audit Proyek Todo.....	148

Ringkasan Bab.....	150
Lampiran A. Template Prompt AI untuk Membuat Kode.....	151
A.1 Template Prompt Dasar.....	151
A.2 Template Prompt Lengkap.....	152
A.3 Contoh Prompt Membuat Kode Baru.....	153
A.4 Contoh Prompt Menambahkan Fitur.....	153
A.5 Contoh Prompt Belajar Konsep.....	154
A.6 Template Prompt Review Kode.....	154
A.7 Template Prompt Anti-Halusinasi.....	155
A.8 Template Prompt untuk Debugging.....	155
A.9 Template Prompt Refactoring.....	155
A.10 Template Prompt Dokumentasi.....	156
A.11 Checklist Prompt yang Baik.....	156
Golden Rule Lampiran A.....	156
Lampiran B. Template Prompt Debugging.....	157
B.1 Template Prompt Debugging Dasar.....	157
B.2 Template Prompt Debugging yang Lebih Lengkap.....	158
B.3 Template Debugging Python.....	159
B.4 Template Debugging FastAPI.....	160
B.5 Template Debugging Frontend fetch().....	160
B.6 Template Debugging Android / Kotlin.....	161
B.7 Template Debugging Logic Error.....	161
B.8 Template Debugging Input Kosong / Salah.....	162
B.9 Checklist Sebelum Bertanya ke AI.....	162
B.10 Pola Membaca Error.....	163
B.11 Contoh Prompt Siap Pakai.....	164
Golden Rule Lampiran B.....	164
Lampiran C. Template Prompt Refactoring.....	165
C.1 Template Prompt Refactoring Dasar.....	165
C.2 Template Prompt Refactoring Lengkap.....	165
C.3 Template Refactoring Python Dasar.....	166
C.4 Template Refactoring FastAPI.....	166
C.5 Template Refactoring Frontend.....	167
C.6 Template Refactoring Android / Kotlin.....	168
C.7 Contoh Refactoring Sederhana.....	168
C.8 Checklist Refactoring.....	170
C.9 Prompt untuk Mengecek Hasil Refactoring.....	170
C.10 Prompt Refactoring Aman.....	171
C.11 Kesalahan Umum Saat Refactoring.....	171
C.12 Pola Kerja Refactoring dengan Git.....	172
Golden Rule Lampiran C.....	172
Lampiran D. Template Prompt Dokumentasi.....	173
D.1 Template Prompt Dokumentasi Dasar.....	173
D.2 Template Prompt Dokumentasi Lengkap.....	173

D.3 Template README.md untuk Proyek Python.....	175
D.4 Template README.md untuk Proyek FastAPI.....	176
D.5 Template README.md untuk Proyek Frontend.....	177
D.6 Template README.md untuk Proyek Android.....	178
D.7 Template README.md untuk Proyek Akhir Todo AI-Ready.....	179
D.8 Contoh Kerangka README.md.....	180
D.11 Prompt untuk Membuat Dokumentasi dari Kode.....	182
D.12 Kesalahan Umum dalam Dokumentasi.....	183
Golden Rule Lampiran D.....	183
Lampiran E. Checklist Setup Laptop Coding.....	184
E.1 Checklist Utama.....	184
E.2 Cek Sistem Operasi.....	185
E.3 Cek Terminal.....	186
E.4 Cek Python.....	187
E.5 Cek Git.....	188
E.6 Cek uv.....	189
E.7 Cek Editor Kode.....	190
E.8 Cek Ollama.....	191
E.9 Cek Docker.....	192
E.10 Cek Tool Android.....	193
E.11 Checklist Final Setup.....	194
E.12 Masalah Umum dan Solusi.....	195
E.13 Prompt AI untuk Mengecek Setup.....	196
Golden Rule Lampiran E.....	196
Lampiran F. Checklist Review Kode dari AI.....	197
F.1 Checklist Utama Review Kode AI.....	197
F.2 Cara Memakai Checklist.....	198
F.3 Pertanyaan Review yang Lebih Detail.....	198
F.4 Checklist Review untuk Kode Python.....	199
F.5 Checklist Review untuk API FastAPI.....	199
F.6 Checklist Review untuk Frontend.....	200
F.7 Checklist Review untuk Android / Kotlin.....	200
F.8 Checklist Risiko Keamanan Dasar.....	201
F.9 Prompt AI untuk Review Kode.....	201
F.10 Prompt AI untuk Mengecek Library.....	202
F.11 Prompt AI untuk Menyederhanakan Kode.....	202
F.12 Perintah Git Setelah Kode Lolos Review.....	202
Golden Rule Lampiran F.....	203
Glossary.....	204
Referensi.....	210
Tentang Penulis.....	213

Cara Menggunakan Buku

Buku ini sebaiknya dipakai dengan cara **langsung praktik**, bukan hanya dibaca. Prinsip utamanya:

Baca sedikit, praktik banyak.

Langkah Menggunakan Buku

1. **Baca tujuan bab terlebih dahulu**

Pahami apa yang ingin dicapai pada setiap bab.

2. **Ikuti contoh CLI dan kode secara langsung**

Ketik ulang perintah dan kode di laptop, jangan hanya dibaca.

3. **Gunakan AI sebagai asisten belajar**

Pakai AI untuk:

- menjelaskan kode,
- menjelaskan error,
- memberi contoh tambahan,
- membantu memperbaiki kode.

4. **Jangan langsung percaya kode dari AI**

Setiap kode harus:

- dibaca,
- dijalankan,
- diuji,
- dipahami.

5. **Coba input salah**

Jangan hanya mencoba contoh yang benar. Uji juga input kosong, salah, atau tidak lengkap.

6. **Simpan hasil latihan dengan Git**

Setelah kode berjalan, simpan:

```
git add .
git commit -m "selesai latihan bab ini"
```

Pola Belajar Buku Ini

Prompt → Kode → Jalankan → Error → Perbaiki → Pahami → Simpan di Git

Aturan Utama

Jangan pernah menjalankan kode yang tidak dipahami.

Jika belum paham, gunakan prompt:

```
Jelaskan kode ini dengan bahasa sederhana untuk pemula.  
Tunjukkan bagian penting dan kemungkinan error.
```

Dengan cara ini, pembaca tidak hanya cepat membuat kode, tetapi juga belajar menjadi programmer yang **mandiri, kritis, dan bertanggung jawab**.

BAB 1. Mindset Coding Cepat dengan AI

1.1 AI sebagai Asisten, Bukan Pengganti Programmer

Cara paling sehat memakai AI untuk coding adalah menganggapnya sebagai **teman kerja yang cepat**, tetapi belum tentu benar. Ia bisa menulis kode dalam hitungan detik, menjelaskan konsep, memberi alternatif solusi, dan membantu membaca error. Namun ia tidak benar-benar memahami konteks hidup pengguna, kebutuhan organisasi, risiko keamanan, atau konsekuensi dari keputusan teknis yang salah.

Dalam praktik coding modern, AI dapat berperan sebagai *pair programmer*. Artinya, pembaca tidak bekerja sendirian. Pembaca bisa bertanya, meminta contoh, meminta penjelasan, dan meminta perbaikan. Aider, misalnya, mendeskripsikan dirinya sebagai *AI pair programming* di terminal dan dapat digunakan bersama repo Git lokal. (aider.chat)

Namun posisi AI harus jelas: **AI membantu, manusia memimpin**. AI boleh membuat draf kode, tetapi manusia harus membaca. AI boleh memberi solusi error, tetapi manusia harus menguji. AI boleh menyarankan arsitektur, tetapi manusia harus menilai apakah cocok, aman, dan bisa dirawat.

AI sangat berguna untuk membantu:

- membuat **contoh kode awal**,
- menjelaskan **pesan error**,
- menulis **komentar dan dokumentasi**,
- membuat **unit test sederhana**,
- memberi **alternatif pendekatan**,
- merapikan kode agar lebih mudah dibaca,
- membantu memahami kode lama.

Referensi *The AI-Powered Developer* juga menekankan bahwa AI coding dapat membantu dalam siklus testing, debugging, menjelaskan kode, dan menjelaskan konsep. Tetapi buku tersebut juga menegaskan pentingnya memahami tool yang digunakan, bukan sekadar menerima hasilnya.

Yang **tidak boleh diserahkan penuh** kepada AI adalah:

- **desain sistem**,
- **keamanan aplikasi**,
- **logika bisnis**,
- **validasi hasil**,
- **keputusan akhir**,
- **akses ke data sensitif**,
- **kode produksi tanpa review**.

Kesalahan besar pemula adalah merasa bahwa kode yang terlihat rapi pasti benar. Dalam coding, kode yang bisa jalan belum tentu **benar**, belum tentu **aman**, dan belum tentu **layak dipakai jangka panjang**. Kode harus diuji dengan input normal, input salah, input kosong, dan kondisi ekstrem.

Kalimat kunci:

AI boleh menulis kode pertama, tetapi programmer wajib menjadi pemeriksa terakhir.

1.2 Dari Vibe Coding ke Coding yang Bertanggung Jawab

Istilah *vibe coding* sering dipakai untuk menggambarkan cara membuat software dengan memberi instruksi natural kepada AI, lalu membiarkan AI menghasilkan kode. Cara ini sangat menarik karena terasa cepat: pengguna menjelaskan ide, AI membuat program, lalu program dicoba. Untuk prototipe, eksperimen, demo, atau belajar cepat, pola ini sangat berguna.

Tetapi ada bahaya besar: **kode cepat sering terlihat meyakinkan sebelum benar-benar diuji**. Pemula mudah terjebak karena AI bisa membuat kode yang tampak profesional, lengkap dengan komentar dan struktur folder. Padahal mungkin ada bug, library tidak cocok, validasi input lemah, atau logika salah.

Referensi *Beyond Vibe Coding* menekankan bahwa AI dapat mempercepat prototipe, tetapi tetap membutuhkan review, refinement, testing, verifikasi, dan ownership dari developer. Di dalam indeksnya, buku tersebut juga menyoroti risiko seperti *package hallucination*, vulnerability, kode yang sulit dirawat, dan pentingnya testing pada sistem yang dibuat dengan bantuan AI.

Bagi pemula, *vibe coding* boleh dipakai sebagai pintu masuk. Tetapi setelah program pertama jalan, proses harus naik kelas menjadi **coding yang bertanggung jawab**:

- baca kode,
- pahami fungsi setiap bagian,
- jalankan program,
- uji dengan beberapa input,
- perbaiki error,
- rapikan struktur,
- tulis dokumentasi,
- simpan perubahan dengan Git.

Git penting karena coding adalah proses eksperimen. Git adalah sistem *version control* yang mencatat perubahan file dari waktu ke waktu sehingga versi lama dapat dipanggil kembali jika terjadi kesalahan. (git-scm.com)

OWASP juga mengingatkan bahwa praktik secure coding sebaiknya masuk ke dalam siklus pengembangan, bukan ditempel di akhir. Panduan OWASP bersifat teknologi-agnostik dan berisi checklist umum seperti validasi input, autentikasi, manajemen session, akses kontrol, kriptografi, error handling, logging, dan perlindungan data. (owasp.org)

Perubahan mindset yang penting:

Mindset Salah	Mindset Benar
AI pasti benar	AI harus diverifikasi
Kode jalan berarti selesai	Kode harus diuji dan dipahami
Copy-paste lebih cepat	Paham kode lebih aman
Error itu gangguan	Error adalah bahan belajar
Dokumentasi nanti saja	Dokumentasi dibuat sejak awal
Security urusan akhir	Security harus dipikir sejak awal

Dengan cara ini, pembaca tidak menjadi **operator prompt pasif**, tetapi menjadi programmer yang belajar berpikir. Inilah inti buku ini: **coding cepat, tetapi tetap sadar risiko**.

1.3 Pola Kerja Cepat

Pola kerja yang dipakai dalam buku ini sederhana:

Ide → Prompt → Kode → Jalankan → Error → Debug → Test → Dokumentasi → Commit

Pola ini harus diulang terus sampai menjadi kebiasaan.

Ide adalah kebutuhan awal. Contohnya: “Saya ingin membuat kalkulator sederhana,” “Saya ingin membaca file CSV,” atau “Saya ingin membuat aplikasi Todo.”

Prompt adalah instruksi kepada AI. Prompt yang baik harus jelas: bahasa pemrograman apa, target pembaca siapa, batasan apa, input-output seperti apa, dan cara penjelasan yang diinginkan.

Kode adalah hasil awal, bukan hasil akhir. Kode dari AI harus dianggap sebagai **draf pertama**.

Jalankan berarti kode harus benar-benar dicoba di komputer. Jangan hanya dibaca.

Error adalah hal normal. Dalam coding, error bukan tanda gagal. Error adalah pesan dari mesin yang memberi tahu bagian mana yang perlu diperiksa.

Debug adalah proses mencari penyebab error. AI bisa membantu menjelaskan error, tetapi pembaca tetap harus memahami akar masalahnya.

Test memastikan kode bekerja pada beberapa kondisi. Jangan hanya menguji satu input.

Dokumentasi menjelaskan cara menjalankan, fungsi program, contoh penggunaan, dan catatan error umum.

Commit menyimpan perubahan ke Git. Ini penting agar pembaca bisa kembali ke versi sebelumnya jika eksperimen berikutnya gagal.

Dokumentasi resmi Python menyediakan tutorial, referensi library, dan panduan penggunaan Python yang dapat dijadikan sumber utama saat hasil AI perlu dicek ulang. ([Python documentation](#))

Contoh pola kerja mini:

Langkah	Contoh
Ide	Membuat kalkulator sederhana
Prompt	Minta AI membuat kode Python tanpa library tambahan
Kode	AI menghasilkan file <code>kalkulator.py</code>
Jalankan	<code>python3 kalkulator.py</code>
Error	Ada salah ketik atau input tidak valid
Debug	Minta AI jelaskan error
Test	Coba tambah, kurang, kali, bagi, dan pembagian nol
Dokumentasi	Tulis cara menjalankan
Commit	<code>git add . && git commit -m "buat kalkulator awal"</code>

Golden rule:

Kode yang belum dijalankan belum boleh dipercaya. Kode yang belum diuji belum boleh dipakai.

1.4 Contoh Prompt Dasar

Prompt dasar untuk bab ini:

```
Saya pemula Python. Buat program kalkulator sederhana.  
Jelaskan setiap baris kode.  
Tambahkan contoh input dan output.  
Jangan gunakan library tambahan.
```

Prompt ini baik karena jelas. Ada empat elemen penting:

- **identitas pembaca:** pemula Python,
- **tujuan:** membuat kalkulator sederhana,
- **cara penjelasan:** jelaskan setiap baris,
- **batasan:** jangan gunakan library tambahan.

Prompt yang terlalu pendek biasanya menghasilkan jawaban yang terlalu umum. Prompt yang terlalu panjang bisa membuat AI kehilangan fokus. Untuk pemula, prompt terbaik adalah **pendek, jelas, dan bertahap**.

Contoh prompt yang lebih kuat:

```
Saya pemula Python.  
  
Buat program kalkulator sederhana dengan fitur:  
1. tambah  
2. kurang  
3. kali  
4. bagi  
  
Syarat:  
- jangan gunakan library tambahan  
- gunakan input dari keyboard  
- jika pembagian dengan nol, tampilkan pesan error yang mudah dipahami  
- jelaskan kode per bagian  
- berikan contoh input dan output
```

Prompt ini lebih baik karena ada **fitur, batasan, validasi error, dan format penjelasan**.

Setelah AI memberi kode, jangan langsung percaya. Gunakan prompt lanjutan:

```
Periksa kode ini.  
Apakah ada bug?  
Apakah ada kondisi input yang bisa membuat program error?  
Berikan perbaikan minimal saja.  
Jangan ubah seluruh struktur kode.
```


Prompt lanjutan ini penting karena AI sering terlalu agresif mengubah kode. Untuk pemula, perubahan kecil lebih mudah dipahami daripada perubahan besar.

1.5 Hands-on: Program Python Pertama dengan Bantuan AI

Tujuan hands-on ini sederhana: pembaca membuat program kalkulator, menjalankannya, membaca error, lalu memperbaikinya. Fokusnya bukan membuat program canggih, tetapi membangun kebiasaan kerja yang benar.

Langkah 1 — Buat Folder Proyek

Di terminal Linux:

```
mkdir latihan-ai-coding
cd latihan-ai-coding
```

Buat file Python:

```
nano kalkulator.py
```

Langkah 2 — Minta AI Membuat Kode

Gunakan prompt:

```
Saya pemula Python. Buat program kalkulator
sederhana.
Fitur: tambah, kurang, kali, bagi.
Gunakan input dari keyboard.
Jangan gunakan library tambahan.
Jika pembagian dengan nol, tampilkan pesan error.
Jelaskan kode per bagian.
```

Misalnya AI memberi kode seperti ini:

```
print("Kalkulator Sederhana")
print("1. Tambah")
print("2. Kurang")
print("3. Kali")
print("4. Bagi")

pilihan = input("Pilih operasi (1/2/3/4): ")

angka1 = float(input("Masukkan angka pertama: "))
angka2 = float(input("Masukkan angka kedua: "))

if pilihan == "1":
    hasil = angka1 + angka2
    print("Hasil:", hasil)
elif pilihan == "2":
    hasil = angka1 - angka2
    print("Hasil:", hasil)
elif pilihan == "3":
    hasil = angka1 * angka2
    print("Hasil:", hasil)
elif pilihan == "4":
    if angka2 == 0:
        print("Error: pembagian dengan nol tidak boleh.")
    else:
        hasil = angka1 / angka2
        print("Hasil:", hasil)
else:
    print("Pilihan tidak valid.")
```

Langkah 3 — Jalankan Kode

Simpan file, lalu jalankan:

```
python3 kalkulator.py
```

Coba input normal:

```
Pilih operasi (1/2/3/4): 1
Masukkan angka pertama: 10
Masukkan angka kedua: 5
Hasil: 15.0
```

Coba input pembagian nol:

```
Pilih operasi (1/2/3/4): 4
Masukkan angka pertama: 10
Masukkan angka kedua: 0
Error: pembagian dengan nol tidak boleh.
```

Sampai sini pembaca sudah membuat program pertama. Tetapi proses belum selesai. Program masih perlu diuji dengan input yang salah.

Langkah 4 — Uji Input Salah

Coba masukkan huruf pada angka:

Masukkan angka pertama: abc

Biasanya Python akan menampilkan error seperti:

```
ValueError: could not convert string to float: 'abc'
```

Ini momen belajar yang penting. Jangan panik. Copy error tersebut, lalu tanya ke AI:

```
Saya mendapat error ini:
```

```
ValueError: could not convert string to float: 'abc'
```

```
Jelaskan artinya dengan bahasa sederhana.
Berikan perbaikan minimal agar program tidak langsung
berhenti.
```

AI kemungkinan akan menyarankan **try-except**.

Langkah 5 — Perbaiki Kode dengan Error Handling

Versi yang lebih aman:

```
print("Kalkulator Sederhana")
print("1. Tambah")
print("2. Kurang")
print("3. Kali")
print("4. Bagi")

pilihan = input("Pilih operasi (1/2/3/4): ")

try:
    angka1 = float(input("Masukkan angka pertama: "))
    angka2 = float(input("Masukkan angka kedua: "))

    if pilihan == "1":
        hasil = angka1 + angka2
        print("Hasil:", hasil)
    elif pilihan == "2":
        hasil = angka1 - angka2
        print("Hasil:", hasil)
    elif pilihan == "3":
        hasil = angka1 * angka2
        print("Hasil:", hasil)
    elif pilihan == "4":
        if angka2 == 0:
            print("Error: pembagian dengan nol tidak boleh.")
        else:
            hasil = angka1 / angka2
            print("Hasil:", hasil)
    else:
        print("Pilihan tidak valid.")

except ValueError:
    print("Error: input angka harus berupa angka, bukan huruf.")
```

Penjelasan singkat:

- **try** dipakai untuk mencoba menjalankan kode yang mungkin error.
- **except ValueError** menangkap error ketika input tidak bisa diubah menjadi angka.
- Program tidak langsung berhenti, tetapi memberi pesan yang lebih ramah.

Langkah 6 — Buat Test Manual

Gunakan tabel sederhana:

No	Skenario	Input	Hasil yang Diharapkan	Status
1	Tambah	10 dan 5	15	☐
2	Kurang	10 dan 5	5	☐
3	Kali	10 dan 5	50	☐
4	Bagi	10 dan 5	2	☐
5	Bagi nol	10 dan 0	Pesan error	☐
6	Input huruf	abc	Pesan error	☐
7	Pilihan salah	9	Pilihan tidak valid	☐

Di sinilah pembaca belajar bahwa **testing bukan teori akademik**, tetapi kebiasaan harian. Referensi *Coding with AI* juga menempatkan testing, debugging, dokumentasi, dan maintenance sebagai bagian penting dari penggunaan AI untuk coding, bukan hanya membuat kode cepat.

Langkah 7 — Dokumentasikan Program

Buat file:

```
nano README.md
```

Isi sederhana:

```
# Kalkulator Sederhana

Program Python untuk latihan AI coding.

## Fitur

- Tambah
- Kurang
- Kali
- Bagi
- Validasi pembagian dengan nol
- Validasi input huruf

## Cara Menjalankan

python3 kalkulator.py
```

Catatan

Program ini dibuat untuk belajar mindset coding cepat dengan AI. Kode tetap harus dibaca, dijalankan, diuji, dan diperbaiki oleh manusia.

Langkah 8 — Simpan dengan Git

```
git init
git add kalkulator.py README.md
git commit -m "buat kalkulator sederhana dengan validasi input"
```

Perintah ini membiasakan pembaca bekerja seperti programmer sungguhan. Setiap perubahan penting disimpan. Jika nanti kode rusak karena eksperimen, versi lama masih bisa dilacak.

Git sendiri adalah sistem *version control* bebas dan terbuka yang dirancang untuk menangani proyek kecil sampai sangat besar dengan cepat dan efisien. (git-scm.com)

Checklist Akhir Bab 1

Sebelum lanjut ke Bab 2, pembaca sebaiknya sudah bisa menjawab:

Pertanyaan	Ya/Tidak
Apakah saya paham bahwa AI hanya asisten?	☐
Apakah saya bisa membuat prompt sederhana untuk coding?	☐
Apakah saya bisa menjalankan file Python?	☐
Apakah saya bisa membaca error dasar?	☐
Apakah saya bisa meminta AI menjelaskan error?	☐
Apakah saya bisa memperbaiki kode sederhana?	☐
Apakah saya sudah mencoba input salah?	☐
Apakah saya sudah membuat README?	☐
Apakah saya sudah menyimpan kode dengan Git?	☐

Ringkasan Bab

AI membuat coding terasa lebih cepat, tetapi kecepatan tanpa pemahaman adalah risiko. Untuk pemula, AI harus dipakai sebagai **guru sabar**, **teman diskusi**, dan **pembuat draf awal**, bukan sebagai pengganti belajar. Pembaca perlu membiasakan pola kerja:

ide → **prompt** → **kode** → **jalankan** → **error** → **debug** → **test** → **dokumentasi** → **commit**.

Bab ini memberi fondasi penting: **kode AI tidak boleh dipercaya sebelum dijalankan, diuji, dipahami, dan disimpan dengan rapi**. Dengan mindset ini, pembaca siap masuk ke Bab 2 untuk menyiapkan ekosistem coding open-source yang lebih lengkap.

Golden rule Bab 1:

AI mempercepat coding, tetapi pemahaman, pengujian, keamanan, dan tanggung jawab tetap milik manusia.

BAB 2. Menyiapkan Ekosistem Coding Open-Source

2.1 Sistem Operasi dan Terminal

Linux/Ubuntu cocok untuk belajar coding karena dekat dengan dunia server, mudah dipakai untuk Python, Git, Docker, database, API, dan eksperimen AI lokal. Banyak dokumentasi resmi pengembangan Python di Ubuntu juga langsung menggunakan terminal sebagai jalur utama instalasi dan kerja. ([Dokumentasi Ubuntu](#))

Terminal penting karena programmer harus terbiasa memberi perintah langsung ke sistem. Untuk pemula, terminal bukan sesuatu yang menakutkan. Terminal adalah **ruang kerja utama** untuk menjalankan program, mengelola folder, menginstall package, membaca error, dan menjalankan tool AI.

Perintah dasar yang wajib dikuasai:

Perintah	Fungsi
<code>pwd</code>	melihat posisi folder saat ini
<code>ls</code>	melihat isi folder
<code>cd</code>	pindah folder
<code>mkdir</code>	membuat folder
<code>nano</code>	mengedit file teks dari terminal
<code>python3</code>	menjalankan Python
<code>pip</code>	menginstall package Python

Contoh latihan CLI:

```
pwd
ls
mkdir latihan-coding
cd latihan-coding
nano catatan.txt
```


Isi file `catatan.txt`:

```
Hari ini saya mulai belajar coding cepat dengan AI.
```

Lihat kembali isi folder:

```
ls
```

Catatan penting: pemula tidak perlu langsung menghafal semua perintah Linux. Cukup kuasai perintah yang dipakai setiap hari.

2.2 Editor Code

Editor code adalah tempat membaca, menulis, dan merapikan program. Untuk ekosistem terbuka, gunakan editor seperti **VSCodium**, **Geany**, **Kate**, **Neovim**, atau editor lain yang nyaman. Yang penting bukan nama editornya, tetapi kemampuan pembaca untuk:

- membuka folder proyek,
- membaca file kode,
- membuat file baru,
- mencari teks,
- melihat terminal,
- memahami struktur proyek,
- menyimpan perubahan.

Extension atau fitur dasar yang berguna:

- **Python** untuk membaca dan menjalankan kode Python,
- **Markdown** untuk menulis dokumentasi,
- **Git** untuk melihat perubahan kode,
- **HTTP/REST client** untuk menguji API sederhana.

Untuk buku 100 halaman, jangan terlalu banyak membahas editor. Cukup ajarkan pembaca membuat folder proyek, membuka file, menjalankan kode dari terminal, dan membaca error.

Editor hanya alat bantu. Terminal tetap pusat praktik.

Contoh membuat file Python dari CLI:

```
mkdir bab2-latihan
cd bab2-latihan
nano hello.py
```

Isi:

```
print("Halo, saya siap belajar coding cepat dengan AI.")
```

Jalankan:

```
python3 hello.py
```

2.3 Python, Git, dan Project Folder

Python adalah bahasa utama buku ini karena mudah dibaca, banyak dipakai untuk AI, otomasi, API, data, dan scripting. Untuk memulai, install komponen dasar:

```
sudo apt update
sudo apt install python3 python3-pip python3-venv git curl
ca-certificates -y
```

Cek versi:

```
python3 --version
pip --version
git --version
```

Python menyediakan **venv** untuk membuat *virtual environment*, yaitu lingkungan Python terpisah agar package proyek tidak bercampur dengan sistem utama. Dokumentasi resmi Python menjelaskan bahwa **venv** membuat lingkungan ringan dan terisolasi dengan package sendiri. ([Python documentation](#))

Buat folder proyek:

```
mkdir proyek-python-pertama
cd proyek-python-pertama
```

Buat *virtual environment*:

```
python3 -m venv .venv
source .venv/bin/activate
```

Cek Python yang aktif:

```
which python
python --version
```

Buat program pertama:

```
nano main.py
```

Isi:

```
nama = input("Nama Anda: ")
print(f"Halo {nama}, selamat belajar coding cepat dengan
AI!")
```

Jalankan:

```
python main.py
```

Mengenal Git

Git dipakai untuk menyimpan riwayat perubahan kode. Ini penting karena coding adalah proses coba, error, perbaikan, dan eksperimen. Perintah `git init` membuat repository Git kosong di folder proyek. ([Git SCM](#))

Mulai Git:

```
git init
git status
```

Tambahkan file ke *staging area*:

```
git add main.py
```

Perintah `git add` menambahkan file baru atau file yang berubah ke area persiapan commit. ([Git SCM](#))

Commit pertama:

```
git commit -m "program Python pertama"
```

Perintah `git commit` membuat commit baru dari isi *staging area* dengan pesan perubahan. ([Git SCM](#))

Golden rule Git:

Setiap fitur kecil yang sudah jalan sebaiknya langsung di-commit.

2.4 Manajemen Python dengan uv

Untuk proyek sederhana, `venv` dan `pip` sudah cukup. Tetapi untuk proyek yang makin serius, dependency harus lebih rapi. Di sinilah `uv` berguna. `uv` adalah *Python package and project manager* yang sangat cepat dan ditulis dengan Rust. ([docs.astral.sh](https://docs.astral.sh/uv/))

Install `uv`:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Tutup dan buka ulang terminal, atau jalankan:

```
source ~/.bashrc
```

Cek instalasi:

```
uv --version
```

Buat proyek Python modern:

```
mkdir proyek-uv
cd proyek-uv
uv init
```

Tambahkan package:

```
uv add requests
```

Buat file:

```
nano main.py
```

Isi:

```
import requests

response = requests.get("https://httpbin.org/get")
print(response.status_code)
print(response.json()["url"])
```

Jalankan:

```
uv run python main.py
```

Mengapa `uv` menarik untuk buku ini?

- **lebih rapi** untuk dependency,
- cepat untuk install package,
- cocok untuk proyek Python modern,
- memudahkan pemula memahami struktur proyek,

- membantu transisi dari latihan kecil ke aplikasi nyata.

Dalam outline buku, [uv](#) memang disarankan sebagai bagian dari ekosistem Python yang lebih rapi, bukan sebagai teori tambahan yang panjang.

2.5 AI Lokal dengan Ollama

AI lokal penting karena pembaca dapat belajar tanpa selalu bergantung pada layanan cloud. Dengan AI lokal, eksperimen menjadi lebih **mandiri**, lebih **privat**, dan lebih cocok untuk belajar coding berbasis laptop. Dokumentasi Ollama menyatakan bahwa Ollama dapat berjalan di Linux, dan API-nya memungkinkan aplikasi berinteraksi dengan model secara programatik. (docs.ollama.com)

Install Ollama di Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Jalankan service:

```
sudo systemctl start ollama
sudo systemctl status ollama
```

Coba model kecil untuk coding atau percakapan teknis:

```
ollama run qwen2.5-coder:7b
```

Atau model kecil lain jika mesin terbatas:

```
ollama run deepseek-coder:6.7b
```

Cek daftar model lokal:

```
ollama list
```

Contoh bertanya ke model lokal:

```
Buatkan program Python hello world.
Jelaskan setiap baris kode untuk pemula.
```

Ollama berjalan lokal, dan dokumentasi FAQ menjelaskan bahwa prompt dan data tidak dikirim ke pihak mereka saat model dijalankan secara lokal. (docs.ollama.com)

Catatan realistis: model lokal butuh RAM/VRAM. Jika laptop lemah, gunakan model kecil. Jangan memaksakan model besar hanya karena terlihat lebih canggih.

2.6 AI Coding Assistant Open-Source

Ada dua pola utama memakai AI untuk coding:

- melalui **terminal**,
- melalui **editor kode**.

Untuk terminal, Aider menarik karena dirancang sebagai *AI pair programming in your terminal* dan dapat bekerja dengan proyek lokal. Dokumentasinya juga menekankan penggunaan Aider untuk mengedit kode di repo Git lokal. ([GitHub](#))

Install Aider dengan **uv**:

```
uv tool install aider-chat
```

Masuk ke folder proyek:

```
cd proyek-python-pertama
```

Jalankan Aider:

```
aider
```

Contoh perintah di dalam Aider:

```
Baca file main.py. Tambahkan validasi agar nama kosong tidak
diterima.
Jelaskan perubahan yang kamu lakukan.
```

Untuk editor kode, Continue dapat dipakai bersama model lokal melalui Ollama. Dokumentasi Continue menyediakan panduan penggunaan Ollama untuk *local AI development* dan *offline coding assistance*. ([docs.continue.dev](#))

Namun ada prinsip penting: **jangan masukkan seluruh proyek besar ke konteks AI tanpa seleksi**. AI perlu konteks, tetapi terlalu banyak konteks bisa membuat jawaban melebar, lambat, atau salah fokus. Referensi *AI-Assisted Coding* juga menunjukkan bahwa Continue dapat memakai konteks seperti kode, perubahan Git, terminal, dan URL, tetapi konteks harus dipilih dengan sadar agar jawaban tetap relevan.

Praktik yang lebih aman:

- kirim hanya file yang relevan,
- jelaskan tujuan perubahan,
- minta perubahan minimal,
- minta AI menjelaskan alasannya,
- jalankan test setelah perubahan,
- commit jika sudah benar.

Contoh prompt yang baik:

```
Saya punya file main.py.  
Tolong tambahkan validasi input kosong.  
Jangan ubah struktur utama.  
Jelaskan perubahan singkat.  
Setelah itu berikan contoh test manual.
```


2.7 Docker untuk Lingkungan Konsisten

Docker berguna untuk menjalankan aplikasi dalam *container*, sehingga lingkungan aplikasi lebih konsisten dan tidak mengotori sistem utama. Untuk belajar web, API, database, dan AI lokal, container sangat membantu karena service dapat dijalankan, dihentikan, dan dihapus dengan lebih rapi. Dokumentasi Docker menyediakan panduan resmi instalasi Docker Engine untuk Ubuntu, termasuk dukungan Ubuntu 24.04 LTS dan 26.04 LTS. ([Docker Documentation](#))

Install Docker Engine mengikuti pola resmi:

```
sudo apt update
sudo apt install ca-certificates curl -y
```

Buat folder keyring:

```
sudo install -m 0755 -d /etc/apt/keyrings
```

Tambahkan repository Docker sesuai dokumentasi resmi terbaru, lalu install:

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin -y
```

Tambahkan user ke grup Docker:

```
sudo usermod -aG docker $USER
```

Logout dan login kembali, lalu cek:

```
docker --version
docker compose version
docker run hello-world
```

Docker Compose berguna untuk menjalankan beberapa service sekaligus, misalnya backend API, database, dan tool pendukung. Dokumentasi Docker Compose menjelaskan beberapa skenario instalasi, termasuk plugin untuk Linux. ([Docker Documentation](#))

Contoh file `compose.yaml` sederhana:

```
services:
  app:
    image: python:3.12-slim
    working_dir: /app
    volumes:
      - ./app
    command: python main.py
```

Buat `main.py`:

```
print("Halo dari container Python.")
```

Jalankan:

```
docker compose up
```

Catatan kritis: Docker bukan kewajiban di awal. Untuk Bab 2, cukup install dan uji. Penggunaan lebih serius akan muncul saat pembaca mulai membuat API, database, dan proyek akhir.

2.8 Hands-on: Setup Ekosistem Coding

Hands-on ini merangkum seluruh Bab 2. Targetnya sederhana: laptop siap untuk latihan Python, Git, AI lokal, dan project kecil.

Langkah 1 — Install Paket Dasar

```
sudo apt update
sudo apt install python3 python3-pip python3-venv git
curl ca-certificates nano -y
```

Cek:

```
python3 --version
pip --version
git --version
```

Langkah 2 — Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
uv --version
```

Buat proyek:

```
mkdir latihan-bab2
cd latihan-bab2
uv init
```

Langkah 3 — Buat Program Python

```
nano main.py
```

Isi:

```
print("Ekosistem coding open-source siap.")

nama = input("Nama: ")
print(f"Halo {nama}, sekarang kita siap coding cepat
dengan AI.")
```

Jalankan:

```
uv run python main.py
```

Langkah 4 — Simpan dengan Git

```
git init
git status
git add .
git commit -m "setup proyek Python pertama"
```

Jika Git meminta identitas lokal, isi:

```
git config --global user.name "Nama Anda"
git config --global user.email "email@contoh.local"
```

Ulangi commit:

```
git commit -m "setup proyek Python pertama"
```

Langkah 5 — Install dan Coba Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
sudo systemctl start ollama
ollama --version
```

Jalankan model:

```
ollama run qwen2.5-coder:7b
```

Prompt uji:

```
Buatkan fungsi Python untuk mengecek apakah sebuah angka
genap.
Jelaskan kode untuk pemula.
```

Langkah 6 — Install Aider

```
uv tool install aider-chat
```

Masuk ke proyek:

```
cd latihan-bab2
aider
```

Prompt di Aider:

```
Baca main.py. Tambahkan pengecekan agar nama kosong
tidak diterima.
Jelaskan perubahan singkat.
```

Setelah Aider mengubah kode, jalankan:

```
uv run python main.py
```

Jika benar, simpan:

```
git status
git add .
git commit -m "tambah validasi nama kosong"
```

Langkah 7 — Uji Docker

```
docker --version
docker compose version
docker run hello-world
```

Jika belum bisa menjalankan Docker tanpa **sudo**, logout dan login kembali setelah menjalankan:

```
sudo usermod -aG docker $USER
```

Checklist Akhir Bab 2

Komponen	Status
Terminal bisa digunakan	☐
Python terinstall	☐
pip tersedia	☐
venv bisa dibuat	☐
Git terinstall	☐
git init , git add , git commit dipahami	☐
uv terinstall	☐
Proyek Python bisa dijalankan dengan uv run	☐
Ollama terinstall	☐
Model lokal bisa dijalankan	☐
Aider bisa dijalankan di repo Git lokal	☐
Docker bisa diuji	☐
Commit pertama sudah dibuat	☐

Ringkasan Bab

Ekosistem coding yang baik tidak harus mahal dan tidak harus rumit. Dengan **Linux/Ubuntu, terminal, Python, Git, uv, Ollama, Aider, Continue, dan Docker**, pembaca sudah memiliki fondasi yang cukup kuat untuk belajar coding cepat secara mandiri.

Bab ini sengaja tidak memperluas terlalu banyak teori. Targetnya praktis: **laptop siap, terminal siap, Python siap, Git siap, AI lokal siap, dan proyek pertama sudah di-commit**. Ini sejalan dengan strategi buku: satu tahap, satu hasil nyata, lalu naik ke tahap berikutnya.

Golden rule Bab 2:

Ekosistem yang rapi membuat coding lebih cepat, debugging lebih mudah, dan belajar AI lebih aman.

BAB 3. Python Dasar dengan Bantuan AI

3.1 Mengenal Python

Python cocok untuk pemula karena bentuk kodenya ringkas dan mudah dibaca. Dalam banyak kasus, pembaca dapat langsung memahami maksud program hanya dengan membaca baris demi baris. Ini penting untuk buku yang targetnya **cepat bisa coding**, karena pembaca tidak perlu terlalu lama tenggelam dalam sintaks yang rumit.

Namun mudah bukan berarti boleh sembarangan. Kode Python tetap harus:

- **dibaca,**
- **dijalankan,**
- **diuji,**
- **diperbaiki,**
- **dipahami,**
- lalu **disimpan dengan Git.**

Dokumentasi resmi Python menjelaskan bahwa tutorial Python dirancang untuk mengenalkan konsep dan fitur dasar bahasa Python secara praktis, dan contoh-contohnya dapat dicoba sendiri. Ini cocok untuk pendekatan buku ini yang menekankan *hands-on*. ([Python documentation](#))

Menjalankan File .py

Buat folder latihan:

```
mkdir bab3-python-dasar
cd bab3-python-dasar
```

Buat file Python:

```
nano hello.py
```

Isi:

```
print("Halo, saya belajar Python dengan bantuan AI.")
```

Jalankan:

```
python3 hello.py
```

Output:

```
Halo, saya belajar Python dengan bantuan AI.
```

Inilah pola paling dasar: **tulis file** → **jalankan** → **lihat hasil**.

Cara Membaca Pesan Error

Error bukan musuh. Error adalah pesan yang memberi tahu bagian mana yang salah. Dokumentasi resmi Python menjelaskan bahwa pesan error sering menampilkan *stack traceback*, yaitu jejak baris program yang menyebabkan masalah. Bagian terakhir biasanya menunjukkan tipe error dan penyebabnya. ([Python documentation](#))

Contoh file error:

```
nano error.py
```

Isi:

```
nama = "Budi"  
print(namaa)
```

Jalankan:

```
python3 error.py
```

Kemungkinan error:

```
NameError: name 'namaa' is not defined
```

Artinya Python tidak mengenal variabel **namaa**. Yang benar adalah **nama**.

Gunakan prompt AI:

```
Saya mendapat error Python berikut:
```

```
NameError: name 'namaa' is not defined
```

```
Jelaskan artinya dengan bahasa pemula.  
Tunjukkan kemungkinan baris yang salah.  
Berikan perbaikan minimal.
```

Catatan penting: jangan minta AI langsung menulis ulang semua program. Untuk pemula, minta **perbaikan minimal** agar proses belajar tetap terasa.

3.2 Dasar Sintaks Python

Bagian ini mengenalkan fondasi Python yang akan terus dipakai di bab berikutnya: variabel, tipe data, kondisi, perulangan, dan fungsi.

Variabel

Variabel adalah nama untuk menyimpan nilai.

```
nama = "Aisyah"
umur = 20

print(nama)
print(umur)
```

Jalankan:

```
nano variabel.py
python3 variabel.py
```

Variabel membuat program lebih fleksibel. Daripada menulis nilai langsung berkali-kali, kita menyimpan nilai dalam nama yang mudah dipahami.

Tipe Data Dasar

Tipe data dasar yang perlu dikuasai pemula:

Tipe Data	Contoh	Kegunaan
<code>str</code>	<code>"Budi"</code>	teks
<code>int</code>	<code>25</code>	bilangan bulat
<code>float</code>	<code>3.14</code>	bilangan desimal
<code>bool</code>	<code>True / False</code>	logika benar/salah

Contoh:

```
nama = "Budi"
umur = 21
tinggi = 170.5
aktif = True

print(type(nama))
print(type(umur))
print(type(tinggi))
print(type(aktif))
```

Jalankan:

```
nano tipe_data.py
python3 tipe_data.py
```

Kondisi: if, elif, else

Kondisi membuat program bisa mengambil keputusan.

```
nilai = 75

if nilai >= 80:
    print("Sangat baik")
elif nilai >= 60:
    print("Lulus")
else:
    print("Perlu belajar lagi")
```

Jalankan:

```
nano kondisi.py
python3 kondisi.py
```

Kondisi akan sering dipakai untuk validasi input, login sederhana, filter data, dan logika aplikasi.

Perulangan: for dan while

Perulangan dipakai untuk menjalankan perintah berkali-kali.

Contoh **for**:

```
for angka in range(1, 6):
    print("Angka:", angka)
```

Contoh **while**:

```
hitung = 1

while hitung <= 5:
    print("Hitung:", hitung)
    hitung = hitung + 1
```

Jalankan:

```
nano perulangan.py
python3 perulangan.py
```

Catatan kritis: hati-hati dengan `while`. Jika kondisi tidak pernah berubah, program bisa berjalan tanpa berhenti.

Fungsi: def, Parameter, dan return

Fungsi adalah blok kode yang diberi nama agar bisa dipakai ulang.

```
def sapa(nama):  
    return f"Halo {nama}, selamat belajar Python!"  
  
pesan = sapa("Dina")  
print(pesan)
```

Jalankan:

```
nano fungsi.py  
python3 fungsi.py
```

Fungsi penting karena membuat kode lebih rapi. Referensi *AI-Assisted Coding* menunjukkan bahwa AI dapat membantu pembelajar menyusun struktur fungsi, parameter, dan alur data sebelum menulis kode lengkap. Ini lebih sehat daripada langsung meminta AI menulis seluruh program.

Prompt belajar fungsi:

```
Saya pemula Python.  
Tolong jelaskan struktur fungsi berikut:  
- nama fungsi  
- parameter  
- isi fungsi  
- return value  
Jangan buat kode baru dulu.
```

3.3 Struktur Data

Struktur data dipakai untuk menyimpan banyak nilai. Untuk pemula, empat struktur dasar yang perlu dikuasai adalah **list**, **dict**, **tuple**, dan **set**.

List

list menyimpan banyak data berurutan.

```
buah = ["apel", "pisang", "mangga"]

print(buah)
print(buah[0])
print(buah[1])
```

Tambah data:

```
buah.append("jeruk")
print(buah)
```

Dict

dict menyimpan data dalam pasangan **key-value**.

```
mahasiswa = {
    "nama": "Rina",
    "umur": 20,
    "jurusan": "Informatika"
}

print(mahasiswa["nama"])
print(mahasiswa["jurusan"])
```

dict sangat penting untuk memahami JSON, API, dan data web.

Tuple

tuple mirip **list**, tetapi biasanya dipakai untuk data yang tidak ingin diubah.

```
koordinat = (-6.2, 106.8)

print(koordinat[0])
print(koordinat[1])
```

Set

set menyimpan data unik.

```
angka = {1, 2, 2, 3, 3, 3}

print(angka)
```

Output hanya akan berisi nilai unik:

```
{1, 2, 3}
```

Latihan Struktur Data

Buat file:

```
nano struktur_data.py
```

Isi:

```
mahasiswa = [
    {"nama": "Ayu", "nilai": 85},
    {"nama": "Bima", "nilai": 70},
    {"nama": "Citra", "nilai": 95},
]

for data in mahasiswa:
    print(data["nama"], "mendapat nilai", data["nilai"])
```

Jalankan:

```
python3 struktur_data.py
```

Struktur seperti ini akan sering muncul saat membaca file CSV, JSON, API, dan database.

3.4 Error Handling

Program yang baik bukan program yang tidak pernah error. Program yang baik adalah program yang **bisa menangani error dengan jelas**.

Python memakai `try` dan `except` untuk menangani kondisi yang berpotensi error. Dokumentasi resmi Python menjelaskan bahwa setelah error terjadi, bagian akhir pesan error menjelaskan tipe exception dan detail penyebabnya; Python juga menyediakan daftar *built-in exceptions* yang dapat dipakai untuk memahami jenis error. ([Python documentation](#))

Contoh tanpa error handling:

```
angka = int(input("Masukkan angka: "))
print("Angka dikali dua:", angka * 2)
```

Jika pengguna memasukkan huruf, program akan berhenti.

Versi lebih aman:

```
try:
    angka = int(input("Masukkan angka: "))
    print("Angka dikali dua:", angka * 2)
except ValueError:
    print("Error: input harus berupa angka.")
```

Jalankan:

```
nano error_handling.py
python3 error_handling.py
```

Membaca Traceback

Contoh error:

```
ValueError: invalid literal for int() with base 10:  
'abc'
```

Cara membacanya:

- **ValueError** = nilai yang diberikan tidak cocok.
- **int()** mencoba mengubah teks menjadi angka.
- **'abc'** tidak bisa diubah menjadi angka.

Prompt untuk AI:

```
Saya mendapat traceback Python berikut:
```

```
[paste traceback]
```

```
Tolong jelaskan:
```

1. tipe error,
2. penyebab utama,
3. baris yang perlu diperiksa,
4. perbaikan minimal,
5. contoh test setelah diperbaiki.

Referensi *Coding with AI* menunjukkan contoh nyata bahwa program yang “secara teknis berjalan” belum tentu sesuai kebutuhan, sehingga developer tetap harus membaca output, memahami error, dan memperbaiki perilaku program.

3.5 AI untuk Belajar Python

AI sangat berguna sebagai tutor Python, asalkan prompt-nya jelas. Jangan hanya bertanya, “kode ini salah kenapa?” Lebih baik jelaskan konteks, tujuan, error, dan batasan.

Prompt dasar:

```
Jelaskan kode Python berikut baris demi baris.  
Tandai bagian yang sulit untuk pemula.  
Berikan latihan kecil setelah penjelasan.
```

```
[kode ditempel di sini]
```

Prompt untuk belajar, bukan copy-paste:

```
Saya ingin belajar, bukan hanya mendapatkan jawaban.  
Tolong bantu saya memahami cara berpikirnya.  
Jangan langsung berikan solusi final.  
Berikan petunjuk bertahap.
```

Prompt untuk memperbaiki kode:

```
Perbaiki kode berikut secara minimal.  
Jangan ubah struktur besar.  
Jelaskan bagian yang diubah.  
Tambahkan satu contoh input dan output.
```

Prompt untuk latihan:

```
Buatkan 5 latihan Python dasar tentang list dan dict.  
Urutkan dari mudah ke sedang.  
Jangan berikan jawaban dulu.  
Berikan petunjuk singkat untuk setiap soal.
```

Ini sejalan dengan strategi buku: pembaca harus **cepat menulis kode, cepat melihat hasil, cepat paham error, terbiasa testing, dan tidak menjadi tukang copy-paste kode AI**.

3.6 Hands-on

Bagian ini adalah inti Bab 3. Semua latihan memakai CLI agar pembaca terbiasa bekerja seperti programmer.

Hands-on 1 — Program Kalkulator Sederhana

Buat file:

```
nano kalkulator.py
```

Isi:

```
print("Kalkulator Sederhana")
print("1. Tambah")
print("2. Kurang")
print("3. Kali")
print("4. Bagi")

pilihan = input("Pilih operasi (1/2/3/4): ")

try:
    angka1 = float(input("Masukkan angka pertama: "))
    angka2 = float(input("Masukkan angka kedua: "))

    if pilihan == "1":
        print("Hasil:", angka1 + angka2)
    elif pilihan == "2":
        print("Hasil:", angka1 - angka2)
    elif pilihan == "3":
        print("Hasil:", angka1 * angka2)
    elif pilihan == "4":
        if angka2 == 0:
            print("Error: pembagian dengan nol tidak boleh.")
        else:
            print("Hasil:", angka1 / angka2)
    else:
        print("Pilihan tidak valid.")

except ValueError:
    print("Error: input harus berupa angka.")
```

Jalankan:

```
python3 kalkulator.py
```

Test manual:

Skenario	Input	Harapan
Tambah	10 dan 5	15
Kurang	10 dan 5	5
Kali	10 dan 5	50
Bagi	10 dan 5	2
Bagi nol	10 dan 0	pesan error
Input huruf	abc	pesan error

Prompt AI untuk review:

```
Review kode kalkulator ini.  
Apakah ada bug?  
Apakah validasi input sudah cukup untuk pemula?  
Berikan perbaikan minimal saja.
```

Hands-on 2 — Program Konversi Suhu

Buat file:

```
nano suhu.py
```

Isi:

```
try:  
    celcius = float(input("Masukkan suhu dalam Celcius:  
"))  
  
    fahrenheit = (celcius * 9/5) + 32  
    kelvin = celcius + 273.15  
  
    print("Fahrenheit:", fahrenheit)  
    print("Kelvin:", kelvin)  
  
except ValueError:  
    print("Error: suhu harus berupa angka.")
```

Jalankan:

```
python3 suhu.py
```

Prompt AI:

Jelaskan rumus konversi suhu dalam kode ini.
Lalu tambahkan validasi sederhana tanpa membuat kode terlalu rumit.

Catatan kritis: jangan terlalu cepat menambahkan menu, GUI, atau fitur besar.
Untuk pemula, satu program kecil yang dipahami lebih baik daripada program besar yang hanya disalin.

Hands-on 3 — Program Manajemen Catatan Berbasis Terminal

Buat file:

```
nano catatan.py
```

Isi:

```
catatan = []

while True:
    print("\nMenu Catatan")
    print("1. Tambah catatan")
    print("2. Lihat catatan")
    print("3. Keluar")

    pilihan = input("Pilih menu: ")

    if pilihan == "1":
        teks = input("Tulis catatan: ")
        if teks.strip() == "":
            print("Catatan tidak boleh kosong.")
        else:
            catatan.append(teks)
            print("Catatan disimpan.")

    elif pilihan == "2":
        if len(catatan) == 0:
            print("Belum ada catatan.")
        else:
            for nomor, isi in enumerate(catatan,
start=1):
                print(f"{nomor}. {isi}")

    elif pilihan == "3":
        print("Selesai.")
        break

    else:
```

```
print("Pilihan tidak valid.")
```

Jalankan:

```
python3 catatan.py
```

Konsep yang dipakai:

- `list`,
- `while`,
- `if`,
- `append`,
- `enumerate`,
- validasi input kosong.

Prompt AI:

```
Jelaskan program catatan ini untuk pemula.  
Fokus pada while, list, append, dan break.  
Berikan 3 latihan pengembangan kecil.
```

Hands-on 4 — Membaca dan Menulis File Teks

Versi catatan sebelumnya hilang saat program ditutup. Sekarang kita simpan ke file.

Buat file:

```
nano file_teks.py
```

Isi:

```
nama_file = "catatan.txt"  
  
teks = input("Tulis catatan baru: ")  
  
with open(nama_file, "a", encoding="utf-8") as file:  
    file.write(teks + "\n")  
  
print("Catatan tersimpan.")  
  
print("\nIsi catatan:")  
with open(nama_file, "r", encoding="utf-8") as file:  
    isi = file.read()  
    print(isi)
```

Jalankan:

```
python3 file_teks.py
```

Cek isi file dari CLI:

```
cat catatan.txt
```

Penjelasan singkat:

- `"a"` berarti *append*, menambah isi di akhir file.
- `"r"` berarti *read*, membaca file.
- `with open(...)` membantu membuka dan menutup file dengan aman.

Hands-on 5 — Membaca File CSV Sederhana

CSV adalah format sederhana untuk data tabel. Python memiliki modul bawaan `csv` untuk membaca dan menulis data tabular. Dokumentasi resmi Python menjelaskan bahwa modul `csv` memungkinkan programmer membaca dan menulis data CSV tanpa harus mengurus detail format CSV secara manual. ([Python documentation](#))

Buat file CSV:

```
nano nilai.csv
```

Isi:

```
nama, nilai
Ayu, 85
Bima, 70
Citra, 95
Doni, 60
```

Buat program:

```
nano baca_csv.py
```

Isi:

```
import csv

with open("nilai.csv", "r", encoding="utf-8") as file:
    pembaca = csv.DictReader(file)

    for baris in pembaca:
        nama = baris["nama"]
        nilai = int(baris["nilai"])

        if nilai >= 75:
            status = "Lulus"
        else:
            status = "Perlu belajar lagi"

        print(nama, nilai, status)
```

Jalankan:

```
python3 baca_csv.py
```

Output:

```
Ayu 85 Lulus
Bima 70 Perlu belajar lagi
Citra 95 Lulus
Doni 60 Perlu belajar lagi
```

Prompt AI:

```
Jelaskan kode baca_csv.py baris demi baris.
Mengapa csv.DictReader cocok untuk pemula?
Tambahkan latihan kecil untuk menghitung rata-rata
nilai.
```

Mini Proyek Bab 3 — Ringkasan Nilai Mahasiswa

Gabungkan konsep `csv`, `list`, `dict`, kondisi, fungsi, dan file output.

Buat file:

```
nano ringkasan_nilai.py
```

Isi:

```
import csv

def tentukan_status(nilai):
    if nilai >= 75:
        return "Lulus"
    return "Perlu belajar lagi"

total = 0
jumlah = 0

with open("nilai.csv", "r", encoding="utf-8") as file_input:
    pembaca = csv.DictReader(file_input)

    with open("ringkasan.txt", "w", encoding="utf-8") as file_output:
        for baris in pembaca:
            nama = baris["nama"]
            nilai = int(baris["nilai"])
            status = tentukan_status(nilai)

            total = total + nilai
            jumlah = jumlah + 1

            file_output.write(f"{nama}: {nilai} - {status}\n")

        rata_rata = total / jumlah
        file_output.write(f"\nRata-rata nilai: {rata_rata}\n")

print("Ringkasan selesai dibuat: ringkasan.txt")
```

Jalankan:

```
python3 ringkasan_nilai.py
cat ringkasan.txt
```

Prompt AI untuk review:

```
Review kode ringkasan_nilai.py.
Apakah ada risiko error jika file CSV kosong?
Berikan perbaikan minimal.
Jelaskan dengan bahasa pemula.
```

Pertanyaan ini penting karena kode di atas masih punya risiko: jika CSV kosong, **jumlah** bernilai 0 dan pembagian **total / jumlah** akan error. Inilah cara belajar yang benar: **program jalan dulu, lalu diuji dengan kondisi tidak ideal.**

Checklist Akhir Bab 3

Kemampuan	Status
Bisa membuat file .py dari CLI	☐
Bisa menjalankan program Python	☐
Bisa memakai variabel	☐
Bisa memakai str , int , float , bool	☐
Bisa memakai if , elif , else	☐
Bisa memakai for dan while	☐
Bisa membuat fungsi dengan def	☐
Bisa memakai list dan dict	☐
Bisa membaca error sederhana	☐
Bisa memakai try-except	☐
Bisa membaca dan menulis file teks	☐
Bisa membaca CSV sederhana	☐
Bisa meminta AI menjelaskan kode tanpa copy-paste buta	☐

Ringkasan Bab

Bab ini membangun dasar Python yang langsung bisa dipakai: variabel, tipe data, kondisi, perulangan, fungsi, struktur data, error handling, file teks, dan CSV. Semua latihan dibuat pendek agar sesuai dengan target buku 100 halaman dan tetap menjaga komposisi **70% praktik, 30% teori**.

Yang paling penting bukan sekadar “bisa membuat kode”, tetapi **bisa membaca, menjalankan, menguji, memperbaiki, dan memahami kode**. AI membantu mempercepat proses belajar, tetapi hasil AI tetap harus diperiksa dengan dokumentasi resmi, dijalankan di terminal, dan diuji dengan input berbeda.

Golden rule Bab 3:

Belajar Python dengan AI bukan berarti melewati dasar; justru AI harus dipakai untuk mempercepat pemahaman dasar.

BAB 4. Python untuk Otomasi dan Data Sederhana

4.1 Python untuk Pekerjaan Nyata

Python kuat untuk otomasi karena sintaksnya ringkas, mudah dibaca, dan memiliki banyak library untuk file, teks, data, API, dan sistem. Situs resmi Python menjelaskan Python sebagai bahasa yang membuat programmer dapat bekerja cepat dan mengintegrasikan sistem secara efektif. ([Python.org](https://python.org))

Dalam pekerjaan nyata, banyak tugas yang sebenarnya berulang:

- mengganti nama banyak file,
- membaca banyak data dari CSV,
- membuat laporan ringkas,
- mengubah file teks menjadi format lain,
- mengambil data dari API,
- menyimpan hasil ke file,
- membuat backup sederhana.

Tugas seperti ini sangat cocok untuk Python. AI dapat membantu membuat script awal, tetapi pembaca tetap harus memeriksa hasilnya.

Prinsip penting:

Otomasi yang baik bukan hanya cepat, tetapi juga aman, bisa diuji, dan tidak merusak data asli.

Sebelum membuat script otomasi, biasakan:

- buat folder latihan,
- pakai data contoh,
- jangan langsung jalankan pada data penting,
- tampilkan rencana perubahan dulu,
- baru jalankan perubahan sebenarnya.

4.2 Mengolah Data Sederhana

Data sederhana biasanya muncul dalam format **CSV**, **JSON**, **list**, dan **dict**.

CSV cocok untuk data tabel, misalnya daftar mahasiswa, nilai, transaksi, inventaris, atau log sederhana. Python memiliki modul bawaan **csv** untuk membaca dan menulis file CSV. Modul ini membantu programmer menangani data CSV tanpa harus memecah teks secara manual.

JSON cocok untuk data dari API. Format JSON mirip struktur **dict** dan **list** di Python, sehingga mudah dipahami oleh pemula.

Contoh data JSON:

```
{
  "nama": "Ayu",
  "nilai": 85,
  "lulus": true
}
```

Di Python, bentuknya mirip **dict**:

```
data = {
    "nama": "Ayu",
    "nilai": 85,
    "lulus": True
}
```

Untuk data yang lebih besar, **pandas** bisa membantu membaca CSV menjadi *DataFrame*. Dokumentasi resmi **pandas.read_csv()** menjelaskan bahwa fungsi ini membaca file CSV menjadi *DataFrame* dan juga mendukung pembacaan bertahap untuk file besar. (pandas.pydata.org)

Namun untuk pemula, jangan langsung semua hal memakai **pandas**. Gunakan urutan sehat:

1. pahami CSV dengan modul bawaan **csv**,
2. pahami **list** dan **dict**,
3. baru gunakan **pandas** untuk ringkasan data.

4.3 Mengambil Data dari API

API adalah cara aplikasi berkomunikasi dengan aplikasi lain. Dalam konteks Python, API sering diakses melalui HTTP dan hasilnya biasanya berbentuk JSON.

Library `requests` memudahkan Python mengirim HTTP request. Dokumentasi resminya menyebut Requests sebagai library HTTP yang sederhana dan elegan untuk Python, dan mendukung Python 3.10+. (requests.readthedocs.io)

Install `requests`:

```
mkdir bab4-otomasi
cd bab4-otomasi
python3 -m venv .venv
source .venv/bin/activate
pip install requests
```

Contoh mengambil data dari API publik:

```
nano ambil_api.py
```

Isi:

```
import requests

url = "https://httpbin.org/get"

response = requests.get(url, timeout=10)

print("Status:", response.status_code)
print(response.json())
```

Jalankan:

```
python ambil_api.py
```

Parameter `timeout=10` penting agar program tidak menggantung terlalu lama jika koneksi bermasalah.

Catatan kritis: API publik bisa berubah, lambat, atau tidak bisa diakses. Karena itu, selalu siapkan error handling.

4.4 AI untuk Membantu Script

AI sangat berguna untuk membuat script otomatisasi awal. Tetapi prompt harus spesifik agar hasilnya tidak terlalu kompleks.

Prompt contoh:

```
Buatkan script Python untuk membaca file CSV,  
menghitung total per kategori,  
lalu menyimpan hasilnya ke file summary.csv.  
Gunakan kode sederhana untuk pemula.  
Tambahkan komentar singkat.
```

Prompt yang lebih aman:

```
Saya pemula Python.
```

```
Buat script Python untuk membaca file transaksi.csv.  
Kolomnya: tanggal, kategori, jumlah.
```

```
Tugas:
```

1. hitung total jumlah per kategori,
2. simpan hasil ke summary.csv,
3. gunakan modul csv bawaan Python,
4. jangan gunakan library tambahan,
5. tambahkan komentar singkat,
6. berikan contoh isi transaksi.csv dan output summary.csv.

Prompt untuk review:

```
Periksa script ini.  
Apakah ada resiko file tidak ditemukan?  
Apakah ada risiko nilai jumlah bukan angka?  
Berikan perbaikan minimal.  
Jangan ubah seluruh struktur kode.
```

AI boleh membantu, tetapi pembaca harus tetap bertanya:

- Apakah file input benar?
- Apakah kolom CSV sesuai?
- Apakah angka bisa dikonversi?
- Apakah output bisa dicek manual?
- Apakah script aman untuk data asli?

Referensi buku AI coding yang diunggah menekankan bahwa debugging, testing, dokumentasi, dan perbaikan kode adalah bagian penting dari penggunaan AI untuk pengembangan software, bukan sekadar menghasilkan kode cepat.

4.5 Hands-on

Bagian ini adalah inti bab. Semua contoh memakai CLI dan data lokal.

Hands-on 1 — Rename File Otomatis

Tujuan: mengganti nama banyak file agar rapi.

Buat folder latihan:

```
mkdir rename-latihan
cd rename-latihan
touch foto1.jpg foto2.jpg foto3.jpg
ls
```

Buat script:

```
nano rename_file.py
```

Isi:

```
from pathlib import Path

folder = Path(".")
nomor = 1

for file in folder.glob("*.jpg"):
    nama_baru = f"foto_liburan_{nomor}.jpg"
    print(f"Rename: {file.name} -> {nama_baru}")
    file.rename(nama_baru)
    nomor += 1
```

Jalankan:

```
python3 rename_file.py
ls
```

Penting: script ini langsung mengganti nama file. Untuk data penting, buat mode simulasi dulu.

Versi simulasi:

```
from pathlib import Path

folder = Path(".")
nomor = 1

for file in folder.glob("*.jpg"):
    nama_baru = f"foto_liburan_{nomor}.jpg"
    print(f"SIMULASI: {file.name} akan menjadi {nama_baru}")
    nomor += 1
```

Golden rule otomasi file:

Jangan jalankan script massal pada data asli sebelum diuji di folder contoh.

Hands-on 2 — Membuat Laporan Ringkas dari CSV

Buat file CSV:

```
cd ..
mkdir laporan-csv
cd laporan-csv
nano transaksi.csv
```

Isi:

```
tanggal,kategori,jumlah
2026-01-01,makan,25000
2026-01-01,transport,15000
2026-01-02,makan,30000
2026-01-02,buku,75000
2026-01-03,transport,20000
```

Buat script:

```
nano laporan_csv.py
```

Isi:

```
import csv

total_per_kategori = {}

with open("transaksi.csv", "r", encoding="utf-8") as file:
    pembaca = csv.DictReader(file)

    for baris in pembaca:
        kategori = baris["kategori"]
        jumlah = int(baris["jumlah"])

        if kategori not in total_per_kategori:
            total_per_kategori[kategori] = 0

        total_per_kategori[kategori] += jumlah

with open("summary.csv", "w", encoding="utf-8",
newline="") as file:
    kolom = ["kategori", "total"]
    penulis = csv.DictWriter(file, fieldnames=kolom)

    penulis.writeheader()

    for kategori, total in total_per_kategori.items():
        penulis.writerow({
            "kategori": kategori,
            "total": total
        })

print("Laporan selesai dibuat: summary.csv")
```

Jalankan:

```
python3 laporan_csv.py
cat summary.csv
```

Output:

```
kategori,total
makan,55000
transport,35000
buku,75000
```


Konsep yang dipakai:

- `csv.DictReader`,
- `dict`,
- perulangan,
- akumulasi total,
- `csv.DictWriter`.

Prompt AI untuk belajar:

```
Jelaskan script laporan_csv.py baris demi baris.  
Fokus pada csv.DictReader, dict, dan proses menghitung  
total per kategori.  
Berikan 2 latihan pengembangan sederhana.
```

Hands-on 3 — Versi Ringkas dengan pandas

Untuk data tabel yang lebih besar, **pandas** sering lebih praktis. Dokumentasi **pandas** menjelaskan bahwa **groupby()** digunakan untuk operasi *split-apply-combine*, yaitu memecah data ke kelompok, melakukan operasi, lalu menggabungkan hasil. (pandas.pydata.org)

Install:

```
python3 -m venv .venv
source .venv/bin/activate
pip install pandas
```

Buat script:

```
nano laporan_pandas.py
```

Isi:

```
import pandas as pd

data = pd.read_csv("transaksi.csv")
summary =
data.groupby("kategori")["jumlah"].sum().reset_index()

summary.to_csv("summary_pandas.csv", index=False)

print("Laporan pandas selesai dibuat:
summary_pandas.csv")
print(summary)
```

Jalankan:

```
python laporan_pandas.py
```

Kapan memakai **csv** bawaan?

- data kecil,
- ingin belajar dasar,
- tidak ingin install library tambahan.

Kapan memakai **pandas**?

- data tabel makin besar,
- perlu filter, agregasi, sorting,
- ingin membuat laporan lebih cepat.

Catatan: untuk pemula, **pandas** jangan dijadikan jalan pintas sebelum paham struktur **list** dan **dict**.

Hands-on 4 — Mengambil Data dari API Publik

Buat folder:

```
cd ..
mkdir api-latihan
cd api-latihan
python3 -m venv .venv
source .venv/bin/activate
pip install requests
```

Buat script:

```
nano api_sederhana.py
```

Isi:

```
import json
import requests
url = "https://httpbin.org/get"

try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    data = response.json()

    with open("hasil_api.json", "w", encoding="utf-8") as file:
        json.dump(data, file, indent=2)
    print("Data API berhasil disimpan ke hasil_api.json")

except requests.exceptions.RequestException as error:
    print("Gagal mengambil data dari API.")
    print("Detail error:", error)
```

Jalankan:

```
python api_sederhana.py
cat hasil_api.json
```

Dokumentasi Requests menjelaskan bahwa objek response dapat diproses sebagai teks atau JSON; penggunaan `response.json()` umum dipakai saat API mengembalikan data JSON. (requests.readthedocs.io)

Prompt AI:

```
Jelaskan script api_sederhana.py.
Apa fungsi timeout?
Apa fungsi response.raise_for_status()?
Mengapa error handling penting saat mengambil data dari
API?
```

Hands-on 5 — Script Backup Folder Sederhana

Tujuan: membuat backup folder kerja ke folder lain.

Buat folder contoh:

```
cd ..
mkdir data-kerja
echo "catatan penting" > data-kerja/catatan.txt
mkdir backup-latihan
```

Buat script:

```
nano backup_folder.py
```

Isi:

```
from pathlib import Path
import shutil
from datetime import datetime

sumber = Path("data-kerja")
folder_backup = Path("backup-latihan")
waktu = datetime.now().strftime("%Y%m%d-%H%M%S")
tujuan = folder_backup / f"backup-{waktu}"

if not sumber.exists():
    print("Folder sumber tidak ditemukan.")
else:
    shutil.copytree(sumber, tujuan)
    print(f"Backup selesai: {tujuan}")
```

Jalankan:

```
python3 backup_folder.py
find backup-latihan -type f
```

Konsep yang dipakai:

- `Path`,
- `shutil.copytree`,
- `datetime`,
- pengecekan folder,
- nama backup berbasis waktu.

Prompt AI untuk audit sederhana:

```
Review script backup_folder.py.
Apa risiko jika folder tujuan sudah ada?
Apa risiko jika folder sumber sangat besar?
Berikan perbaikan minimal untuk pemula.
```

Hands-on 6 — Meminta AI Memperbaiki Script yang Error

Buat script yang sengaja salah:

```
nano script_error.py
```

Isi:

```
import csv

with open("data_tidak_ada.csv", "r") as file:
    pembaca = csv.DictReader(file)
    for baris in pembaca:
        print(baris["nama"])
```

Jalankan:

```
python3 script_error.py
```

Akan muncul error karena file tidak ada. Gunakan prompt:

Saya mendapat error Python saat menjalankan script ini:

[paste error]

Ini kode saya:

[paste kode]

Tolong:

1. jelaskan penyebab error,
2. tunjukkan baris bermasalah,
3. berikan perbaikan minimal,
4. jangan ubah seluruh kode,
5. tambahkan pesan error yang mudah dipahami pemula.

Perbaikan minimal:

```
import csv
from pathlib import Path

nama_file = Path("data_tidak_ada.csv")

if not nama_file.exists():
    print(f"File tidak ditemukan: {nama_file}")
else:
    with open(nama_file, "r", encoding="utf-8") as file:
        pembaca = csv.DictReader(file)
        for baris in pembaca:
            print(baris["nama"])
```

Pelajaran penting: AI bukan hanya untuk membuat kode baru. AI sangat berguna untuk **membaca error, menjelaskan penyebab, dan memberi perbaikan kecil.**

Mini Proyek Bab 4 — Laporan Otomatis Pengeluaran

Gabungkan CSV, `dict`, file output, dan error handling.

Buat file:

```
mkdir proyek-laporan
cd proyek-laporan
nano pengeluaran.csv
```

Isi:

```
tanggal,kategori,jumlah
2026-01-01,makan,25000
2026-01-01,transport,15000
2026-01-02,makan,30000
2026-01-02,buku,75000
2026-01-03,transport,20000
```

Buat script:

```
nano laporan_pengeluaran.py
```

Isi:

```
import csv
from pathlib import Path

file_input = Path("pengeluaran.csv")
file_output = Path("laporan_pengeluaran.txt")

if not file_input.exists():
    print("File pengeluaran.csv tidak ditemukan.")
else:
    total_per_kategori = {}
    total_semua = 0

    with open(file_input, "r", encoding="utf-8") as file:
        pembaca = csv.DictReader(file)

        for baris in pembaca:
            try:
                kategori = baris["kategori"]
                jumlah = int(baris["jumlah"])

                if kategori not in total_per_kategori:
                    total_per_kategori[kategori] = 0

                total_per_kategori[kategori] += jumlah
```

```

        total_semua += jumlah

    except ValueError:
        print("Ada nilai jumlah yang bukan angka.
Baris dilewati.")

    with open(file_output, "w", encoding="utf-8") as file:
        file.write("Laporan Pengeluaran\n")
        file.write("=====\n\n")

        for kategori, total in total_per_kategori.items():
            file.write(f"{kategori}: {total}\n")

        file.write(f"\nTotal semua: {total_semua}\n")

    print(f"Laporan selesai dibuat: {file_output}")

```

Jalankan:

```

python3 laporan_pengeluaran.py
cat laporan_pengeluaran.txt

```

Commit hasil:

```

git init
git add .
git commit -m "buat laporan otomatis pengeluaran"

```

Checklist Akhir Bab 4

Kemampuan	Status
Bisa membuat script otomasi sederhana	☐
Bisa membaca dan menulis file teks	☐
Bisa mengganti nama file massal dengan aman	☐
Bisa membaca CSV dengan <code>csv.DictReader</code>	☐
Bisa membuat laporan ringkas dari CSV	☐
Bisa memahami JSON dasar	☐
Bisa mengambil data API dengan <code>requests</code>	☐
Bisa menyimpan hasil API ke file JSON	☐
Bisa membuat backup folder sederhana	☐
Bisa meminta AI menjelaskan dan memperbaiki error	☐
Bisa menyimpan hasil kerja dengan Git	☐

Ringkasan Bab

Bab ini menunjukkan bahwa Python bukan hanya bahasa latihan, tetapi **alat kerja nyata**. Dengan Python, pembaca dapat mengotomasi file, membaca CSV, membuat laporan, mengambil data API, menyimpan JSON, dan membuat backup sederhana. AI membantu mempercepat pembuatan script, tetapi hasilnya tetap harus diuji pada data contoh sebelum digunakan pada data asli.

Fokus bab ini adalah **praktik langsung dari CLI**. Pembaca belajar bahwa otomasi yang baik harus aman, bertahap, dan mudah dicek. Jangan mengejar script canggih jika script sederhana sudah menyelesaikan masalah.

Golden rule Bab 4:

Otomasi terbaik adalah otomasi yang kecil, jelas, aman diuji, dan benar-benar menghemat waktu.

BAB 5. Web Programming Cepat dengan Python dan FastAPI

5.1 Konsep Dasar Web

Sebelum coding, pembaca perlu memahami istilah dasar.

Website adalah halaman web yang dibuka pengguna melalui browser. Isinya bisa berupa teks, gambar, formulir, tabel, atau halaman informasi.

Web app adalah aplikasi yang berjalan melalui web. Contohnya aplikasi Todo, dashboard, sistem input data, atau aplikasi administrasi sederhana.

API adalah pintu komunikasi antar aplikasi. API memungkinkan frontend, Android app, atau sistem lain meminta data ke backend.

Request adalah permintaan dari client ke server. Misalnya: “ambil daftar Todo” atau “simpan Todo baru”.

Response adalah jawaban dari server ke client. Biasanya berbentuk JSON.

JSON adalah format data ringan yang mudah dibaca manusia dan mudah diproses program. Contoh:

```
{
  "id": 1,
  "title": "Belajar FastAPI",
  "done": false
}
```

Backend adalah bagian aplikasi yang berjalan di server. Tugasnya mengatur logika, API, database, validasi, dan keamanan.

Frontend adalah bagian yang dilihat pengguna. Di buku ini, frontend akan dibahas lebih lanjut di Bab 6.

Pola sederhananya:

```
Browser / Android App
    ↓ request
FastAPI Backend
    ↓ query
SQLite Database
    ↑ data
    ↑ response JSON
```

Inti bab ini: pembaca membuat backend kecil, bukan sekadar memahami definisi.

5.2 Mengapa FastAPI

FastAPI cocok untuk belajar web programming cepat karena:

- berbasis Python,
- struktur kode ringkas,
- cocok untuk API modern,
- mendukung validasi data,
- otomatis membuat dokumentasi API,
- mudah diuji dari browser dan CLI.

Dokumentasi resmi FastAPI menunjukkan langkah dasar: import `FastAPI`, membuat instance aplikasi, membuat *path operation*, mendefinisikan fungsi, lalu mengembalikan konten. (fastapi.tiangolo.com)

FastAPI juga membuat dokumentasi API interaktif berbasis OpenAPI. Ini membantu pemula karena endpoint bisa diuji langsung tanpa membuat frontend dulu. Dokumentasi FastAPI sendiri menyebut tutorialnya disusun bertahap dan dapat dipakai sebagai referensi ketika membutuhkan fitur tertentu. (fastapi.tiangolo.com)

Dalam buku ini, FastAPI dipakai untuk tujuan praktis:

- membuat API Hello World,
- membuat API Todo,
- menyimpan data ke SQLite,
- menguji endpoint,
- membaca response JSON,
- meminta AI membantu membuat endpoint baru,
- debug error API.

Catatan kritis: FastAPI mempercepat pembuatan API, tetapi tidak otomatis membuat aplikasi aman. Validasi input, error handling, autentikasi, dan pembatasan akses tetap harus dipelajari bertahap.

5.3 Struktur Proyek FastAPI

Buat folder kerja dari CLI:

```
mkdir bab5-fastapi
cd bab5-fastapi
python3 -m venv .venv
source .venv/bin/activate
```

Install FastAPI dan server ASGI:

```
pip install fastapi uvicorn
```

Struktur proyek awal:

```
bab5-fastapi/
├── .venv/
└── main.py
```

Buat file:

```
nano main.py
```

Dalam FastAPI, beberapa istilah dasar yang perlu dipahami:

Istilah	Arti
<code>main.py</code>	file utama aplikasi
<code>route</code>	alamat endpoint API
model data	struktur data input/output
folder proyek	tempat file aplikasi disusun
virtual environment	lingkungan Python terpisah untuk proyek

Untuk buku 100 halaman, struktur jangan dibuat terlalu kompleks. Gunakan satu file dulu agar pemula paham alur kerja. Pemisahan folder bisa dilakukan di proyek lanjutan.

5.4 Routing Dasar

Routing menentukan alamat API dan operasi yang dijalankan. Empat operasi dasar yang dipakai dalam bab ini:

Method	Fungsi Praktis
GET	membaca data
POST	menambah data
PUT	mengubah data
DELETE	menghapus data

Buat API pertama:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Halo, FastAPI siap dipakai"}
```

Jalankan:

```
uvicorn main:app --reload
```

Buka dari CLI lain:

```
curl http://127.0.0.1:8000/
```

Output:

```
{"message": "Halo, FastAPI siap dipakai"}
```

Penjelasan singkat:

- `from fastapi import FastAPI` mengambil class utama FastAPI.
- `app = FastAPI()` membuat aplikasi.
- `@app.get("/")` membuat route untuk method GET.
- `return {"message": ...}` mengembalikan JSON.

Golden rule awal API:

Jika endpoint belum bisa dites dengan `curl`, jangan buru-buru membuat frontend.

5.5 Database Ringan dengan SQLite

SQLite cocok untuk latihan karena ringan, tidak perlu server database terpisah, dan cukup untuk proyek kecil. Situs resmi SQLite menjelaskan SQLite sebagai library C kecil, cepat, mandiri, andal, dan penuh fitur untuk SQL database engine. (sqlite.org)

SQLite juga bersifat *serverless* dan *zero-configuration*, sehingga cocok untuk pemula yang ingin fokus memahami alur API dan data. (sqlite.org)

Python sudah memiliki modul bawaan `sqlite3`. Dokumentasi Python menjelaskan bahwa modul `sqlite3` menyediakan antarmuka DB-API 2.0 untuk database SQLite, termasuk tutorial dan referensi penggunaan. ([Python documentation](https://docs.python.org/3/library/sqlite3.html))

Kita akan membuat database Todo sederhana:

```
Todo:
- id
- title
- done
```

Buat file baru:

```
nano main.py
```

Isi versi SQLite:

```
import sqlite3
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
DB_NAME = "todo.db"

class TodoInput(BaseModel):
    title: str

def get_connection():
    return sqlite3.connect(DB_NAME)

def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS todos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        title TEXT NOT NULL,
        done INTEGER NOT NULL DEFAULT 0
    )
    """

    conn.commit()
    conn.close()

init_db()

@app.get("/")
def home():
    return {"message": "Todo API siap dipakai"}

@app.get("/todos")
def list_todos():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT id, title, done FROM todos")
    rows = cursor.fetchall()

    conn.close()

    todos = []
    for row in rows:
        todos.append({
            "id": row[0],
            "title": row[1],
            "done": bool(row[2])
        })

    return todos

@app.post("/todos")
def create_todo(todo: TodoInput):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO todos (title, done) VALUES (?, ?)",
        (todo.title, 0)
    )

```

```

    conn.commit()
    todo_id = cursor.lastrowid
    conn.close()

    return {
        "id": todo_id,
        "title": todo.title,
        "done": False
    }

@app.put("/todos/{todo_id}")
def update_todo(todo_id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "UPDATE todos SET done = 1 WHERE id = ?",
        (todo_id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Todo ditandai selesai", "id":
    todo_id}

@app.delete("/todos/{todo_id}")
def delete_todo(todo_id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "DELETE FROM todos WHERE id = ?",
        (todo_id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Todo dihapus", "id": todo_id}

```

Jalankan:

```
uvicorn main:app --reload
```

Script ini masih sederhana. Untuk produksi, perlu validasi lebih kuat, pengecekan data tidak ditemukan, autentikasi, logging, dan struktur proyek yang lebih rapi. Tetapi untuk pemula, versi ini cukup untuk memahami alur API dan database.

5.6 Testing Manual API

FastAPI membuat dokumentasi interaktif otomatis. Setelah server jalan, endpoint bisa diuji melalui:

```
http://127.0.0.1:8000/docs
```

atau dokumentasi alternatif:

```
http://127.0.0.1:8000/redoc
```

Namun dalam buku ini, pembaca juga harus bisa menguji dari CLI menggunakan `curl`.

Test GET Home

```
curl http://127.0.0.1:8000/
```

Test Tambah Todo

```
curl -X POST http://127.0.0.1:8000/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "Belajar FastAPI"}'
```

Test Lihat Todo

```
curl http://127.0.0.1:8000/todos
```

Test Update Todo

```
curl -X PUT http://127.0.0.1:8000/todos/1
```

Test Hapus Todo

```
curl -X DELETE http://127.0.0.1:8000/todos/1
```

Catatan penting: testing manual bukan formalitas. Setiap endpoint harus diuji dengan input benar, input kosong, ID tidak ada, dan data yang tidak sesuai.

5.7 Hands-on

Bagian ini adalah inti Bab 5. Semua latihan memakai CLI.

Hands-on 1 — API Hello World

Buat proyek:

```
mkdir fastapi-hello
cd fastapi-hello
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn
```

Buat file:

```
nano main.py
```

Isi:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Halo dari FastAPI"}
```

Jalankan:

```
uvicorn main:app --reload
```

Test:

```
curl http://127.0.0.1:8000/
```

Jika keluar JSON, API pertama berhasil.

Hands-on 2 — API Todo List Tanpa Database

Untuk memahami konsep endpoint, mulai dari data di memori.

```
mkdir fastapi-todo-memory
cd fastapi-todo-memory
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn
nano main.py
```

Isi:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

todos = []

class TodoInput(BaseModel):
    title: str

@app.get("/todos")
def list_todos():
    return todos

@app.post("/todos")
def create_todo(todo: TodoInput):
    data = {
        "id": len(todos) + 1,
        "title": todo.title,
        "done": False
    }
    todos.append(data)
    return data
```

Jalankan:

```
uvicorn main:app --reload
```

Test tambah data:

```
curl -X POST http://127.0.0.1:8000/todos \
  -H "Content-Type: application/json" \
  -d '{"title": "Belajar API"}'
```

Test lihat data:

```
curl http://127.0.0.1:8000/todos
```

Kritik penting: data di memori akan hilang saat server restart. Karena itu kita perlu SQLite.

Hands-on 3 — API Todo + SQLite

Buat proyek:

```
mkdir fastapi-todo-sqlite
cd fastapi-todo-sqlite
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn
nano main.py
```

Gunakan kode Todo + SQLite dari bagian 5.5.

Jalankan:

```
uvicorn main:app --reload
```

Test tambah data:

```
curl -X POST http://127.0.0.1:8000/todos \
-H "Content-Type: application/json" \
-d '{"title": "Simpan Todo ke SQLite"}'
```

Lihat data:

```
curl http://127.0.0.1:8000/todos
```

Cek file database:

```
ls
```

Seharusnya muncul:

```
main.py
todo.db
```

Hands-on 4 — Prompt AI untuk Membuat Endpoint Baru

Contoh kebutuhan: menambahkan endpoint untuk mencari Todo berdasarkan ID.

Gunakan prompt:

```
Saya punya aplikasi FastAPI Todo dengan SQLite.  
Tolong tambahkan endpoint GET /todos/{todo_id}  
untuk mengambil satu Todo berdasarkan ID.
```

Syarat:

- gunakan sqlite3 bawaan Python,
- jangan ubah struktur besar,
- jika ID tidak ditemukan, kembalikan pesan error sederhana,
- jelaskan perubahan singkat untuk pemula.

Contoh kode yang bisa ditambahkan:

```
@app.get("/todos/{todo_id}")  
def get_todo(todo_id: int):  
    conn = get_connection()  
    cursor = conn.cursor()  
  
    cursor.execute(  
        "SELECT id, title, done FROM todos WHERE id =  
?",  
        (todo_id,) )  
  
    row = cursor.fetchone()  
    conn.close()  
  
    if row is None:  
        return {"error": "Todo tidak ditemukan"}  
  
    return {  
        "id": row[0],  
        "title": row[1],  
        "done": bool(row[2])  
    }
```

Test:

```
curl http://127.0.0.1:8000/todos/1
```

Test ID tidak ada:

```
curl http://127.0.0.1:8000/todos/999
```

Catatan: untuk aplikasi serius, error sebaiknya memakai HTTP status code yang tepat. Tetapi untuk pemula, pesan sederhana cukup dulu agar alur mudah dipahami.

Hands-on 5 — Debug Error API

Buat error sengaja:

```
@app.get("/error")
def error_demo():
    return {"hasil": 10 / 0}
```

Jalankan:

```
curl http://127.0.0.1:8000/error
```

Server akan menampilkan error **ZeroDivisionError**.

Gunakan prompt:

```
Saya mendapat error FastAPI:
```

```
[paste traceback]
```

```
Tolong jelaskan:
```

1. jenis error,
2. baris yang salah,
3. penyebab,
4. perbaikan minimal,
5. cara test ulang dengan curl.

Perbaiki sederhana:

```
@app.get("/error")
def error_demo():
    pembagi = 0

    if pembagi == 0:
        return {"error": "Pembagi tidak boleh nol"}

    return {"hasil": 10 / pembagi}
```

Pelajaran penting: **API harus siap menghadapi input dan kondisi buruk**, bukan hanya kondisi ideal.

Mini Proyek Bab 5 — Todo API Sederhana

Target akhir Bab 5:

FastAPI + SQLite + endpoint CRUD sederhana + test curl

Checklist endpoint:

Endpoint	Method	Fungsi	Status
/	GET	cek API hidup	☐
/todos	GET	lihat semua Todo	☐
/todos	POST	tambah Todo	☐
/todos/{id}	GET	lihat satu Todo	☐
/todos/{id}	PUT	tandai selesai	☐
/todos/{id}	DELETE	hapus Todo	☐

Commit hasil kerja:

```
git init
git add .
git commit -m "buat Todo API dengan FastAPI dan SQLite"
```

Checklist Akhir Bab 5

Kemampuan	Status
Bisa menjelaskan website, web app, API, request, response	☐
Bisa membuat FastAPI Hello World	☐
Bisa menjalankan server dengan <code>uvicorn</code>	☐
Bisa menguji API dengan <code>curl</code>	☐
Bisa membuat endpoint <code>GET</code>	☐
Bisa membuat endpoint <code>POST</code>	☐
Bisa membuat endpoint <code>PUT</code>	☐
Bisa membuat endpoint <code>DELETE</code>	☐
Bisa menyimpan Todo ke SQLite	☐
Bisa membaca response JSON	☐
Bisa meminta AI membuat endpoint baru	☐
Bisa meminta AI menjelaskan error API	☐
Bisa commit proyek API ke Git	☐

Ringkasan Bab

Bab ini menunjukkan bahwa web programming tidak harus dimulai dari sistem besar. Dengan Python dan FastAPI, pembaca bisa membuat **backend API sederhana** dalam waktu singkat. FastAPI membantu mempercepat pembuatan endpoint, validasi input dasar, dan dokumentasi interaktif. SQLite membantu menyimpan data tanpa setup database yang rumit.

Yang paling penting adalah pola kerja: **buat endpoint kecil, jalankan server, test dengan `curl`, baca JSON, debug error, lalu commit**. AI boleh membantu membuat endpoint baru, tetapi pembaca tetap harus membaca kode, menjalankan test, dan memahami alurnya.

Golden rule Bab 5:

API yang baik bukan yang terlihat canggih, tetapi yang bisa diuji, dipahami, dan menyimpan data dengan benar.

BAB 6. Web Frontend Sederhana untuk Pemula

6.1 HTML, CSS, dan JavaScript Secukupnya

Frontend web minimal membutuhkan tiga komponen.

HTML dipakai untuk struktur halaman: judul, form, tombol, input, tabel, dan daftar.

CSS dipakai untuk tampilan: warna, jarak, ukuran, layout, dan kerapian.

JavaScript dipakai untuk interaksi: membaca input, menekan tombol, memanggil API, dan menampilkan data.

MDN Web Docs layak dijadikan rujukan utama karena menyediakan dokumentasi teknologi web terbuka, termasuk HTML, CSS, JavaScript, API browser, dan materi belajar web. ([MDN Web Docs](#))

Untuk bab ini, pembaca cukup memahami pola dasar berikut:

```
HTML = struktur
CSS = tampilan
JavaScript = interaksi
FastAPI = backend
SQLite = penyimpanan data
```

Contoh HTML sangat sederhana:

```
<h1>Aplikasi Todo</h1>
<input id="todoInput" placeholder="Tulis todo baru">
<button>Tambah</button>
```

Contoh CSS sangat sederhana:

```
body {
  font-family: sans-serif;
  margin: 40px;
}
```

Contoh JavaScript sangat sederhana:

```
console.log("Frontend siap");
```

Catatan penting: jangan mulai dari framework besar. Untuk pemula, **HTML, CSS, dan JavaScript murni** lebih baik agar hubungan frontend–backend terlihat jelas.

6.2 Frontend Bertemu Backend

Frontend bertemu backend melalui **HTTP request**. Pada Bab 5, backend FastAPI sudah menyediakan endpoint seperti:

```
GET      /todos
POST     /todos
PUT      /todos/{id}
DELETE  /todos/{id}
```

Frontend akan memanggil endpoint tersebut menggunakan `fetch()`. MDN menjelaskan bahwa *Fetch API* menyediakan antarmuka JavaScript untuk membuat HTTP request dan memproses response dari server. ([MDN Web Docs](#))

Pola sederhananya:

```
User isi form
      ↓
JavaScript baca input
      ↓
fetch() kirim request ke FastAPI
      ↓
FastAPI proses data
      ↓
FastAPI kirim response JSON
      ↓
JavaScript tampilkan data ke halaman
```

Contoh `fetch()` untuk mengambil data:

```
const response = await fetch("http://127.0.0.1:8000/todos");
const data = await response.json();
console.log(data);
```

MDN juga menjelaskan bahwa `window.fetch()` mengembalikan *Promise* yang akan terpenuhi ketika response tersedia. ([MDN Web Docs](#))

Masalah yang sering muncul saat frontend dan backend berbeda alamat adalah **CORS**. FastAPI menjelaskan bahwa *Cross-Origin Resource Sharing* terjadi ketika frontend di browser berkomunikasi dengan backend pada *origin* berbeda. ([FastAPI](#))

Untuk latihan lokal, tambahkan CORS di backend FastAPI:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Catatan kritis: `allow_origins=["*"]` boleh untuk latihan lokal. Untuk produksi, asal akses harus dibatasi.

6.3 AI untuk Membuat Tampilan Awal

AI dapat membantu membuat halaman awal lebih cepat, tetapi prompt harus membatasi kompleksitas. Jangan minta UI besar. Minta halaman kecil, mudah dibaca, dan tanpa library tambahan.

Prompt contoh:

```
Buatkan halaman HTML sederhana untuk aplikasi Todo.  
Ada form input, tombol tambah, dan tabel daftar todo.  
Gunakan HTML, CSS, dan JavaScript murni.  
Kode harus mudah dibaca pemula.
```

Prompt yang lebih kuat:

```
Saya pemula web frontend.  
  
Buatkan satu file index.html untuk aplikasi Todo sederhana.  
  
Syarat:  
- gunakan HTML, CSS, dan JavaScript murni  
- ada input todo  
- ada tombol tambah  
- ada tabel daftar todo  
- ambil data dari API FastAPI di http://127.0.0.1:8000/todos  
- gunakan fetch()  
- tambahkan komentar singkat  
- jangan gunakan framework
```

Prompt review:

```
Review kode index.html berikut.  
Apakah fetch() sudah benar?  
Apakah ada kemungkinan error jika API mati?  
Berikan perbaikan minimal.  
Jangan ubah seluruh struktur kode.
```

Referensi buku yang diunggah juga menekankan pola penting: AI membantu mempercepat pembuatan kode, tetapi developer tetap harus melakukan debugging, testing, dokumentasi, dan validasi hasil.

6.4 Hands-on

Bagian ini adalah inti Bab 6. Semua contoh memakai CLI.

Hands-on 1 — Siapkan Backend FastAPI

Gunakan backend Todo dari Bab 5. Jika belum ada, buat ulang secara ringkas.

```
mkdir bab6-frontend
cd bab6-frontend
mkdir backend frontend
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn
nano main.py
```

Isi `main.py`:

```
import sqlite3
from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
DB_NAME = "todo.db"

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class TodoInput(BaseModel):
    title: str

def get_connection():
    return sqlite3.connect(DB_NAME)

def init_db():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
CREATE TABLE IF NOT EXISTS todos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,

```

```

        done INTEGER NOT NULL DEFAULT 0
    )
    """)
    conn.commit()
    conn.close()

init_db()

@app.get("/")
def home():
    return {"message": "Todo API siap"}

@app.get("/todos")
def list_todos():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, title, done FROM todos")
    rows = cursor.fetchall()
    conn.close()

    return [
        {"id": row[0], "title": row[1], "done":
bool(row[2])}
        for row in rows
    ]

@app.post("/todos")
def create_todo(todo: TodoInput):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO todos (title, done) VALUES (?, ?)",
        (todo.title, 0)
    )
    conn.commit()
    todo_id = cursor.lastrowid
    conn.close()

    return {"id": todo_id, "title": todo.title, "done":
False}

```

Jalankan backend:

```
uvicorn main:app --reload
```

Test dari terminal lain:

```

curl http://127.0.0.1:8000/
curl http://127.0.0.1:8000/todos

```

Hands-on 2 — Membuat Halaman Todo

Masuk ke folder frontend:

```
cd ../frontend
nano index.html
```

Isi:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Aplikasi Todo</title>
  <style>
    body {
      font-family: sans-serif;
      margin: 40px;
      max-width: 700px;
    }

    input {
      padding: 8px;
      width: 70%;
    }

    button {
      padding: 8px 12px;
      cursor: pointer;
    }

    table {
      border-collapse: collapse;
      width: 100%;
      margin-top: 20px;
    }

    th, td {
      border: 1px solid #ccc;
      padding: 8px;
      text-align: left;
    }

    .error {
      color: darkred;
      margin-top: 10px;
    }
  </style>
</head>
```

```

<body>
  <h1>Aplikasi Todo</h1>

  <input id="todoInput" placeholder="Tulis todo baru">
  <button onclick="tambahTodo()">Tambah</button>

  <p id="pesan" class="error"></p>

  <table>
    <thead>
      <tr>
        <th>ID</th>
        <th>Todo</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody id="todoTable"></tbody>
  </table>

  <script>
    const API_URL = "http://127.0.0.1:8000/todos";

    async function ambilTodos() {
      try {
        const response = await fetch(API_URL);
        const data = await response.json();

        const table =
document.getElementById("todoTable");
        table.innerHTML = "";

        data.forEach(todo => {
          const row = document.createElement("tr");

          row.innerHTML = `
            <td>${todo.id}</td>
            <td>${todo.title}</td>
            <td>${todo.done ? "Selesai" : "Belum"}</td>
          `;

          table.appendChild(row);
        });
      } catch (error) {
        document.getElementById("pesan").textContent =
          "Gagal mengambil data dari API.";
        console.log(error);
      }
    }
  </script>

```

```

    async function tambahTodo() {
      const input =
document.getElementById("todoInput");
      const title = input.value.trim();

      if (title === "") {
        document.getElementById("pesan").textContent =
          "Todo tidak boleh kosong.";
        return;
      }

      try {
        await fetch(API_URL, {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({ title: title })
        });

        input.value = "";
        document.getElementById("pesan").textContent =
"";

        ambilTodos();
      } catch (error) {
        document.getElementById("pesan").textContent =
          "Gagal menambah todo.";
        console.log(error);
      }
    }

    ambilTodos();
  </script>
</body>
</html>

```


Hands-on 3 — Jalankan Frontend dari CLI

Jangan buka file langsung dengan klik ganda. Jalankan server lokal sederhana:

```
python3 -m http.server 8080
```

Buka di browser:

```
http://127.0.0.1:8080
```

Coba:

- isi Todo baru,
- klik Tambah,
- lihat tabel berubah,
- refresh halaman,
- pastikan data tetap ada karena tersimpan di SQLite.

Hands-on 4 — Test API dari CLI

Sebelum menyalahkan frontend, cek backend dengan `curl`.

Tambah Todo:

```
curl -X POST http://127.0.0.1:8000/todos \
  -H "Content-Type: application/json" \
  -d '{"title":"Belajar frontend"}'
```

Lihat Todo:

```
curl http://127.0.0.1:8000/todos
```

Jika `curl` berhasil tetapi frontend gagal, masalah kemungkinan ada di JavaScript, CORS, atau URL API.

Hands-on 5 — Memperbaiki Tampilan dengan CSS

Tambahkan sedikit perapian tampilan:

```
h1 {  
  border-bottom: 2px solid #222;  
  padding-bottom: 10px;  
}  
  
tr:nth-child(even) {  
  background: #f2f2f2;  
}
```

Catatan: jangan terlalu lama mempercantik tampilan. Pada bab ini, tujuan utama adalah **menghubungkan frontend dan backend**.

Hands-on 6 — Debug Error `fetch()`

Error yang sering muncul:

Gejala	Kemungkinan Penyebab	Solusi
<code>Failed to fetch</code>	Backend belum jalan	Jalankan <code>uvicorn main:app --reload</code>
Data tidak muncul	URL API salah	Cek <code>API_URL</code>
CORS error	Backend beda origin dan belum izinkan CORS	Tambahkan <code>CORSMiddleware</code>
JSON error	Response bukan JSON	Cek endpoint dengan <code>curl</code>
Tombol tidak bereaksi	Nama fungsi salah	Cek <code>onclick</code> dan console browser

Prompt AI untuk debugging:

```
Saya mendapat error fetch() di browser.
```

```
Ini pesan error:  
[paste error]
```

```
Ini kode JavaScript saya:  
[paste kode]
```

```
Backend FastAPI berjalan di http://127.0.0.1:8000
```

Tolong:

1. jelaskan penyebab kemungkinan,
2. berikan urutan pengecekan,
3. berikan perbaikan minimal,
4. jangan ubah seluruh kode.

Urutan debug yang sehat:

```
Cek backend hidup → cek endpoint dengan curl → cek CORS  
→ cek API_URL → cek console browser
```

6.5 Alternatif Ringan: HTMX

Untuk pemula, JavaScript kadang terasa terlalu cepat menjadi rumit. Salah satu alternatif ringan adalah **HTMX**. Dokumentasi HTMX menjelaskan bahwa HTMX memungkinkan penggunaan fitur browser modern langsung dari HTML, bukan banyak menulis JavaScript. (htmx.org)

HTMX dapat mengakses AJAX, transisi CSS, WebSocket, dan Server-Sent Events memakai atribut HTML. Situs resminya juga menyebut HTMX kecil, *dependency-free*, dan dirancang untuk membuat antarmuka modern dengan kekuatan hypertext. (htmx.org)

Contoh sederhana:

```
<button hx-get="http://127.0.0.1:8000/todos"
  hx-target="#hasil">
  Ambil Todo
</button>

<pre id="hasil"></pre>
```

Namun untuk buku ini, HTMX cukup dikenalkan sebagai alternatif. Latihan utama tetap memakai **HTML**, **CSS**, **JavaScript murni**, dan **fetch()**, karena pembaca perlu memahami alur dasar web modern.

Kapan HTMX cocok?

- ingin mengurangi JavaScript,
- membuat UI interaktif sederhana,
- backend lebih dominan,
- pemula ingin cepat melihat hasil.

Kapan JavaScript murni tetap penting?

- ingin memahami **fetch()**,
- ingin membangun frontend yang lebih fleksibel,
- ingin siap masuk ke Android/web client berikutnya,
- ingin paham cara membaca response JSON.

Mini Proyek Bab 6 — Todo Web Client

Target akhir Bab 6:

```
Frontend HTML + CSS + JavaScript
      ↓ fetch()
FastAPI Todo API
      ↓
SQLite
```

Checklist:

Komponen	Status
Backend FastAPI berjalan	☐
Endpoint <code>/todos</code> bisa dites dengan <code>curl</code>	☐
Frontend berjalan di <code>python3 -m http.server</code>	☐
Form input Todo tersedia	☐
Tombol Tambah bekerja	☐
Data muncul di tabel	☐
Error kosong ditangani	☐
Error fetch bisa dibaca	☐
Kode disimpan dengan Git	☐

Commit hasil kerja:

```
cd ..
git init
git add .
git commit -m "buat frontend Todo terhubung ke FastAPI"
```

Ringkasan Bab

Bab ini menunjukkan hubungan nyata antara **frontend dan backend**. HTML menyusun halaman, CSS merapikan tampilan, JavaScript memanggil API dengan `fetch()`, FastAPI memproses request, SQLite menyimpan data, lalu response JSON ditampilkan kembali ke halaman.

Yang paling penting adalah pola kerja: **buat halaman kecil, jalankan backend, test API dengan `curl`, jalankan frontend, debug `fetch()`, lalu commit**. Jangan mulai dari framework besar sebelum memahami alur dasar ini.

Golden rule Bab 6:

**Frontend yang baik bukan yang paling mewah, tetapi yang jelas alurnya:
input pengguna → request API → response JSON → tampilan berubah.**

BAB 7. Android Programming dengan AI

7.1 Konsep Dasar Android

Android app adalah aplikasi yang berjalan di perangkat Android. Aplikasi dapat berupa catatan, Todo, kalkulator, dashboard, pemutar media, aplikasi IoT, atau client untuk API backend.

Beberapa istilah dasar yang perlu dipahami:

Istilah	Arti Praktis
Activity	layar/entry point utama aplikasi
UI	tampilan yang dilihat pengguna
State	data yang berubah dan mempengaruhi tampilan
Build	proses mengubah kode menjadi aplikasi yang bisa dijalankan
Emulator	perangkat Android virtual untuk testing
APK	file paket aplikasi Android

Dalam pengembangan modern, UI Android banyak dibuat dengan **Jetpack Compose**. Dokumentasi resmi Android menjelaskan Jetpack Compose sebagai *toolkit* modern untuk membangun UI Android dan menyederhanakan pengembangan aplikasi yang dapat menyesuaikan berbagai ukuran layar. ([Android Developers](#))

Untuk pemula, konsep yang paling penting adalah **state**. Jika data berubah, tampilan harus ikut berubah. Dokumentasi Compose menjelaskan bahwa Compose membantu developer lebih eksplisit dalam menyimpan dan memakai *state* pada aplikasi Android. ([Android Developers](#))

Contoh sederhana:

```
User mengetik Todo
      ↓
State berubah
      ↓
UI menampilkan Todo baru
```

Catatan kritis: jangan mulai dengan aplikasi besar. Mulai dari satu layar, satu input, satu tombol, dan satu daftar.

7.2 Kotlin Dasar untuk Pemula

Kotlin dipakai luas dalam ekosistem Android modern. Dokumentasi Kotlin menjelaskan bahwa Jetpack Compose adalah toolkit modern yang direkomendasikan untuk membangun UI native Android dalam Kotlin. ([Kotlin](#))

Untuk bab ini, pembaca cukup mengenal lima hal:

- variabel,
- fungsi,
- class sederhana,
- list,
- lambda sederhana.

Variabel

```
val nama = "Ayu"
var umur = 20

println(nama)
println(umur)
```

val berarti nilai tetap.

var berarti nilai bisa berubah.

Fungsi

```
fun sapa(nama: String): String {
    return "Halo $nama"
}
```

Fungsi membantu memecah kode agar lebih rapi.

Class Sederhana

```
data class Todo(
    val title: String,
    val done: Boolean = false
)
```

data class cocok untuk menyimpan data sederhana seperti Todo.

List

```
val todos = listOf(  
    Todo("Belajar Kotlin"),  
    Todo("Belajar Compose")  
)
```

Untuk data yang bisa berubah, gunakan `mutableListOf`.

```
val todos = mutableListOf<Todo>()  
todos.add(Todo("Belajar Android"))
```

Lambda Sederhana

Lambda adalah fungsi pendek yang sering dipakai untuk event seperti klik tombol.

```
val aksi = {  
    println("Tombol diklik")  
}
```

Dalam Compose, lambda sangat sering muncul, misalnya pada `Button(onClick = { ... })`. Dokumentasi Compose untuk Kotlin juga menjelaskan bahwa Compose banyak memakai fungsi tingkat tinggi dan lambda, termasuk pada parameter `onClick`. ([Android Developers](#))

7.3 Jetpack Compose

Jetpack Compose adalah cara modern membangun UI Android dengan kode Kotlin. Dokumentasi Android menyebut Compose sebagai toolkit modern yang menyederhanakan dan mempercepat pengembangan UI dengan lebih sedikit kode dan API Kotlin yang intuitif. ([Android Developers](#))

Perbedaan cara pikir yang penting:

Cara lama: menyusun tampilan terpisah dari logika
Compose: tampilan ditulis sebagai fungsi Kotlin

Dalam Compose, UI dibuat dengan fungsi yang disebut *composable*. Contoh:

```
@Composable
fun Halo() {
    Text("Halo Android")
}
```

Compose bersifat *declarative*. Artinya, programmer mendeskripsikan tampilan berdasarkan data. Jika data berubah, Compose memperbarui tampilan. Dokumentasi Compose Basics menjelaskan bahwa Compose menggabungkan model pemrograman reaktif dengan ringkasnya Kotlin, dan ketika data berubah, UI diperbarui otomatis. ([Android Developers](#))

Untuk pemula, cukup ingat:

- `@Composable` = fungsi UI,
- `Text()` = menampilkan teks,
- `Button()` = tombol,
- `TextField()` = input,
- `Column()` = susun vertikal,
- `Row()` = susun horizontal,
- `LazyColumn()` = daftar panjang,
- `remember` dan `mutableStateOf` = menyimpan *state* sederhana.

7.4 Komponen UI Dasar

Komponen yang cukup untuk aplikasi Todo sederhana:

Komponen	Fungsi
<code>Text</code>	menampilkan teks
<code>Button</code>	tombol aksi
<code>TextField</code>	input teks
<code>Column</code>	menyusun komponen vertikal
<code>Row</code>	menyusun komponen horizontal
<code>LazyColumn</code>	menampilkan daftar

Contoh sederhana:

```
@Composable
fun LayarSederhana() {
    Column {
        Text("Aplikasi Todo")
        Button(onClick = {
            println("Tombol ditekan")
        }) {
            Text("Tambah")
        }
    }
}
```

Contoh *state* sederhana:

```
var teks by remember { mutableStateOf("") }
```

Artinya nilai `teks` disimpan sebagai state. Jika `teks` berubah, UI yang memakai `teks` akan diperbarui.

Catatan penting: untuk Bab 7, cukup gunakan satu file `MainActivity`. Struktur yang terlalu kompleks akan membingungkan pemula.

7.5 Android App Sederhana

Aplikasi yang cocok untuk pemula adalah **catatan** atau **Todo lokal**. Fitur minimal:

- input data,
- tombol tambah,
- tampilkan list,
- simpan sementara di memori aplikasi.

Versi awal tidak perlu database. Targetnya adalah memahami hubungan:

Input → State → List → UI berubah

Untuk aplikasi Todo lokal, data cukup disimpan dalam `mutableStateListOf`.

Contoh pola data:

```
data class Todo(  
    val title: String,  
    val done: Boolean = false  
)
```

Pola UI:

```
TextField untuk input  
Button untuk tambah  
LazyColumn untuk daftar
```

Catatan kritis: jangan langsung menambahkan login, database, notifikasi, cloud, atau sinkronisasi. Untuk pemula, terlalu banyak fitur akan membuat fokus hilang.

7.6 AI untuk Android

AI sangat membantu saat belajar Android karena error *build* sering panjang dan membingungkan. Tetapi prompt harus jelas dan bertahap.

Prompt dasar:

```
Saya pemula Android.  
Buatkan contoh aplikasi Todo sederhana dengan Kotlin dan  
Jetpack Compose.  
Gunakan satu file MainActivity dulu.  
Jelaskan bagian UI, state, dan event button.
```

Prompt yang lebih baik:

```
Saya pemula Android dan Kotlin.  
  
Buatkan aplikasi Todo lokal dengan Jetpack Compose.  
  
Syarat:  
- gunakan satu file MainActivity.kt  
- ada TextField untuk input  
- ada Button untuk tambah Todo  
- tampilkan daftar Todo dengan LazyColumn  
- gunakan mutableStateListOf  
- jangan gunakan database dulu  
- jelaskan bagian state dan onClick  
- kode harus mudah dibaca pemula
```

Prompt untuk error *build*:

Saya mendapat error build Android berikut:

[paste error]

Ini potongan kode saya:

[paste kode]

Tolong:

1. jelaskan arti error,
2. tunjukkan kemungkinan baris penyebab,
3. berikan perbaikan minimal,
4. jangan ubah seluruh struktur aplikasi,
5. jelaskan dengan bahasa pemula.

Prompt untuk merapikan UI:

Rapikan UI Compose berikut agar lebih mudah dibaca.
Gunakan Column, Row, padding sederhana, dan LazyColumn.
Jangan menambahkan fitur baru.
Jelaskan perubahan singkat.

Referensi buku AI coding yang diunggah menekankan bahwa AI berguna untuk menjelaskan error, membuat contoh kode, membuat dokumentasi, dan membantu debugging, tetapi developer tetap harus memahami dan menguji kode.

7.7 Hands-on

Bagian ini adalah inti Bab 7. Karena pembuatan proyek Android biasanya dilakukan dengan tool grafis, bagian CLI difokuskan pada **build, cek file, dan menghasilkan APK** dari terminal.

Hands-on 1 — Membuat Project Android

Buat project Android dengan template **Empty Activity** dan pilih:

```
Language: Kotlin
UI: Jetpack Compose
Minimum SDK: sesuai kebutuhan latihan
```

Nama project:

```
TodoComposePemula
```

Setelah project dibuat, buka terminal di folder project. Cek struktur:

```
ls
```

Biasanya ada file seperti:

```
app/
gradle/
settings.gradle.kts
build.gradle.kts
gradlew
```

Cek apakah build tool bisa dijalankan:

```
./gradlew tasks
```

Jika perintah ini menampilkan daftar task, project siap di build.

Hands-on 2 — Membuat Tampilan Sederhana

Buka file:

```
nano
app/src/main/java/com/example/todocomposepemula/MainActi
vity.kt
```

Struktur paket bisa berbeda. Sesuaikan dengan folder project masing-masing.

Isi contoh sederhana:

```
package com.example.todocomposepemula

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AplikasiTodo()
        }
    }
}

@Composable
fun AplikasiTodo() {
    Text("Halo, ini aplikasi Todo pertama.")
}
```

Build dari CLI:

```
./gradlew assembleDebug
```

Jika berhasil, APK debug biasanya berada di:

```
app/build/outputs/apk/debug/
```

Cek:

```
ls app/build/outputs/apk/debug/
```


Hands-on 3 — Menambahkan Input dan Tombol

Ubah `MainActivity.kt` menjadi versi berikut:

```
package com.example.todocomposepemula

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AplikasiTodo()
        }
    }
}

@Composable
fun AplikasiTodo() {
    var teks by remember { mutableStateOf("") }

    Column(
        modifier = Modifier.padding(16.dp)
    ) {
        Text("Aplikasi Todo")

        OutlinedTextField(
            value = teks,
            onChange = { teks = it },
            label = { Text("Todo baru") },
            modifier = Modifier.fillMaxWidth()
        )
    }
}
```

```

        Button(
            onClick = {
                println("Todo: $teks")
            }
        ) {
            Text("Tambah")
        }
    }
}

```

Build:

```
./gradlew assembleDebug
```

Pelajaran penting:

- `teks` adalah *state*,
- `OutlinedTextField` membaca dan mengubah *state*,
- `Button` menjalankan aksi saat diklik,
- `println()` hanya untuk cek awal.

Hands-on 4 — Menampilkan List Todo

Ubah file menjadi aplikasi Todo lokal sederhana:

```
package com.example.todocomposepemula

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateListOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

data class Todo(
    val title: String
)

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AplikasiTodo()
        }
    }
}

@Composable
fun AplikasiTodo() {
    var teks by remember { mutableStateOf("") }
    val todos = remember { mutableStateListOf<Todo>() }

    Column(
        modifier = Modifier.padding(16.dp)
    ) {
        Text("Aplikasi Todo")
    }
}
```

```

OutlinedTextField(
    value = teks,
    onValueChange = { teks = it },
    label = { Text("Todo baru") },
    modifier = Modifier.fillMaxWidth()
)

Button(
    onClick = {
        if (teks.isNotBlank()) {
            todos.add(Todo(teks))
            teks = ""
        }
    }
) {
    Text("Tambah")
}

LazyColumn {
    items(todos) { todo ->
        Text(
            text = "- ${todo.title}",
            modifier = Modifier.padding(top =
8.dp)
        )
    }
}
}

```

Build:

```
./gradlew assembleDebug
```

Jika muncul error import, minta AI menjelaskan error, bukan langsung mengganti semua kode.

Hands-on 5 — Meminta AI Menjelaskan Error Build

Contoh error yang sering muncul:

```
Unresolved reference: LazyColumn
```

Prompt:

```
Saya mendapat error Android:
```

```
Unresolved reference: LazyColumn
```

```
Ini potongan import dan kode saya:
```

```
[paste kode]
```

```
Tolong jelaskan:
```

1. arti error,
2. import apa yang mungkin kurang,
3. perbaikan minimal,
4. jangan ubah seluruh file.

Kemungkinan solusi: import belum lengkap atau dependency Compose belum sesuai. Untuk pemula, mulai dari cek import:

```
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
```

Jika error masih muncul, cek konfigurasi Compose di project.

Urutan debug yang sehat:

```
Baca error terakhir → cek nama komponen → cek import → cek
dependency → build ulang
```

Hands-on 6 — Meminta AI Merapikan UI

Prompt:

Rapikan UI Compose berikut agar lebih enak dibaca pemula.

Syarat:

- jangan tambah fitur baru
- gunakan padding sederhana
- beri jarak antar komponen
- gunakan nama fungsi yang jelas
- jelaskan perubahan singkat

Contoh perbaikan kecil:

```
Column(  
    modifier = Modifier  
        .padding(16.dp)  
) {  
    Text("Aplikasi Todo")  
    // komponen lain  
}
```

Untuk bab ini, cukup rapi dan bisa dibaca. Jangan membuat UI terlalu kompleks.

Mini Proyek Bab 7 — Todo Android Lokal

Target akhir Bab 7:

Android Kotlin + Jetpack Compose

↓
TextField

↓
Button

↓
mutableStateListOf

↓
LazyColumn

Checklist:

Komponen	Status
Project Android berhasil dibuat	☐
MainActivity.kt bisa dibuka	☐
Aplikasi bisa menampilkan Text	☐
TextField bisa menerima input	☐
Button bisa menjalankan aksi	☐
Todo kosong tidak ditambahkan	☐
Todo tampil di LazyColumn	☐
Build debug berhasil	☐
APK debug ditemukan	☐
Error build bisa dibaca dan dijelaskan AI	☐

Commit hasil kerja:

```
git init
git add .
git commit -m "buat aplikasi Todo Android lokal dengan
Compose"
```

Ringkasan Bab

Bab ini mengenalkan Android programming secara cepat dan praktis. Pembaca belajar konsep dasar Android, Kotlin dasar, Jetpack Compose, *state*, komponen UI, dan pembuatan aplikasi Todo lokal. AI dipakai sebagai tutor dan asisten debugging, bukan pengganti pemahaman.

Yang paling penting adalah pola kerja: **buat project kecil, edit `MainActivity`, build dari CLI, baca error, minta AI menjelaskan, perbaiki minimal, lalu commit**. Dengan pola ini, pembaca tidak hanya menyalin kode Android, tetapi mulai memahami hubungan antara UI, *state*, dan aksi pengguna.

Golden rule Bab 7:

Aplikasi Android pertama tidak perlu besar; yang penting input masuk, state berubah, UI tampil, build berhasil, dan kode dipahami.

BAB 8. Proyek Akhir: Aplikasi Todo AI-Ready

8.1 Tujuan Proyek

Proyek akhir ini membuat aplikasi **Todo AI-Ready**. Disebut *AI-Ready* bukan karena aplikasinya harus langsung memakai model AI, tetapi karena cara kerjanya sudah siap dibantu AI:

- kode disusun rapi,
- API jelas,
- data tersimpan,
- frontend bisa memanggil backend,
- Android client sederhana bisa membaca data,
- README tersedia,
- error utama sudah diuji,
- semua perubahan disimpan di Git.

Target minimal:

Frontend Web + Backend FastAPI + SQLite + Android Client sederhana

FastAPI cocok dipakai karena mendukung pembuatan API Python modern berbasis *type hints* dan menyediakan dokumentasi interaktif otomatis untuk mencoba endpoint. ([FastAPI](#))

Output akhir bab: pembaca punya proyek kecil yang bisa ditunjukkan sebagai portofolio awal.

8.2 Arsitektur Sederhana

Arsitektur proyek:

```
Browser / Android App
    ↓
FastAPI Backend
    ↓
SQLite Database
```

Penjelasan singkat:

- **Browser** menjalankan frontend HTML, CSS, dan JavaScript.
- **Android App** menjadi client sederhana yang membaca data dari API.
- **FastAPI Backend** menerima request dan mengirim response JSON.
- **SQLite Database** menyimpan data Todo secara lokal.

SQLite cocok untuk proyek pemula karena Python sudah menyediakan modul `sqlite3` yang memberi antarmuka SQL sesuai DB-API 2.0. ([Python documentation](#))

Untuk frontend, `fetch()` dipakai untuk mengirim request ke backend dan membaca response dari server. MDN menjelaskan *Fetch API* sebagai antarmuka JavaScript untuk membuat HTTP request dan memproses response. ([MDN Web Docs](#))

8.3 Fitur Minimal

Fitur aplikasi tidak perlu banyak. Yang penting **lengkap dan berjalan**.

Fitur	Fungsi
Tambah Todo	memasukkan pekerjaan baru
Lihat daftar Todo	menampilkan semua data
Update status	menandai Todo selesai
Hapus Todo	menghapus data
Simpan ke SQLite	data tidak hilang saat server mati
Akses dari web	frontend browser bisa dipakai
Akses dari Android sederhana	Android client bisa membaca Todo

Catatan kritis: proyek pemula sering gagal karena terlalu ambisius. Jangan menambahkan login, dashboard, sinkronisasi, notifikasi, atau AI agent sebelum fitur minimal stabil.

8.4 Peran AI dalam Proyek

AI dipakai sebagai **asisten proyek**, bukan pengganti programmer. Perannya:

- membantu membuat *skeleton* kode,
- membantu membuat endpoint,
- membantu membuat UI,
- membantu membaca error,
- membantu membuat README,
- membantu membuat test sederhana,
- membantu memberi alternatif perbaikan.

Namun AI tidak boleh menggantikan:

- pemahaman alur aplikasi,
- validasi data,
- keputusan desain,
- pengecekan keamanan,
- testing akhir,
- tanggung jawab kode.

Prompt umum proyek:

```
Saya membuat aplikasi Todo dengan FastAPI, SQLite, frontend
HTML/JavaScript,
dan Android client sederhana.
```

```
Tolong bantu secara bertahap.
Jangan buat terlalu kompleks.
Setiap perubahan harus dijelaskan.
Kode harus mudah dipahami pemula.
```

Prompt review:

```
Review proyek Todo ini.
Cari bug dasar, endpoint yang belum diuji, dan bagian
dokumentasi yang kurang.
Berikan daftar perbaikan prioritas.
Jangan menulis ulang semua kode.
```

Golden rule:

AI boleh mempercepat proyek, tetapi testing dan keputusan akhir tetap milik manusia.

8.5 Hands-on Bertahap

Bagian ini adalah inti Bab 8. Semua dimulai dari CLI.

Langkah 1 — Membuat Repo Git

Buat folder proyek:

```
mkdir todo-ai-ready
cd todo-ai-ready
git init
mkdir backend frontend android-docs screenshots
```

Struktur awal:

```
todo-ai-ready/
├── backend/
├── frontend/
├── android-docs/
├── screenshots/
└── README.md
```

Buat README awal:

```
nano README.md
```

Isi:

```
# Todo AI-Ready

Proyek akhir buku AI untuk Programming dan Coding Cepat.

## Komponen

- Backend: FastAPI
- Database: SQLite
- Frontend: HTML, CSS, JavaScript
- Android client: Kotlin + Jetpack Compose sederhana

## Status

Masih dalam pengembangan.
```

Commit awal:

```
git add .
git commit -m "inisialisasi proyek todo ai-ready"
```

Langkah 2 — Membuat Backend

Masuk ke folder backend:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install fastapi uvicorn
nano main.py
```

Isi `main.py`:

```
import sqlite3
from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
DB_NAME = "todo.db"

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class TodoInput(BaseModel):
    title: str

def get_connection():
    return sqlite3.connect(DB_NAME)

def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
CREATE TABLE IF NOT EXISTS todos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    done INTEGER NOT NULL DEFAULT 0
)
""")
```

```

        conn.commit()
        conn.close()

init_db()

@app.get("/")
def home():
    return {"message": "Todo AI-Ready API berjalan"}

@app.get("/todos")
def list_todos():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT id, title, done FROM todos")
    rows = cursor.fetchall()

    conn.close()

    return [
        {"id": row[0], "title": row[1], "done":
bool(row[2])}
        for row in rows
    ]

@app.post("/todos")
def create_todo(todo: TodoInput):
    if todo.title.strip() == "":
        return {"error": "Todo tidak boleh kosong"}

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO todos (title, done) VALUES (?, ?)",
        (todo.title, 0)
    )

    conn.commit()
    todo_id = cursor.lastrowid
    conn.close()

    return {"id": todo_id, "title": todo.title, "done":
False}

```

```

@app.put("/todos/{todo_id}")
def update_todo(todo_id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "UPDATE todos SET done = 1 WHERE id = ?",
        (todo_id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Todo ditandai selesai", "id":
todo_id}

@app.delete("/todos/{todo_id}")
def delete_todo(todo_id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "DELETE FROM todos WHERE id = ?",
        (todo_id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Todo dihapus", "id": todo_id}

```

Jalankan:

```
uvicorn main:app --reload
```

Test dari terminal lain:

```

curl http://127.0.0.1:8000/
curl http://127.0.0.1:8000/todos

```

Langkah 3 — Menguji Endpoint Backend

Tambah Todo:

```
curl -X POST http://127.0.0.1:8000/todos \
  -H "Content-Type: application/json" \
  -d '{"title":"Belajar proyek akhir"}'
```

Lihat Todo:

```
curl http://127.0.0.1:8000/todos
```

Update status:

```
curl -X PUT http://127.0.0.1:8000/todos/1
```

Hapus Todo:

```
curl -X DELETE http://127.0.0.1:8000/todos/1
```

Cek file database:

```
ls
```

Harus muncul:

```
main.py
todo.db
```

Commit backend:

```
cd ..
git add .
git commit -m "buat backend FastAPI Todo dengan SQLite"
```


Langkah 4 — Membuat Frontend Web

Masuk folder frontend:

```
cd frontend
nano index.html
```

Isi:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Todo AI-Ready</title>
  <style>
    body {
      font-family: sans-serif;
      margin: 40px;
      max-width: 720px;
    }

    input {
      padding: 8px;
      width: 70%;
    }

    button {
      padding: 8px 12px;
      cursor: pointer;
    }

    table {
      border-collapse: collapse;
      width: 100%;
      margin-top: 20px;
    }

    th, td {
      border: 1px solid #ccc;
      padding: 8px;
    }

    .error {
      color: darkred;
      margin-top: 10px;
    }
  </style>
</head>
<body>
  <h1>Todo AI-Ready</h1>

  <input id="todoInput" placeholder="Tulis todo baru">
  <button onclick="tambahTodo()">Tambah</button>
```

```

<p id="pesan" class="error"></p>

<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Todo</th>
      <th>Status</th>
      <th>Aksi</th>
    </tr>
  </thead>
  <tbody id="todoTable"></tbody>
</table>

<script>
  const API_URL = "http://127.0.0.1:8000/todos";

  async function ambilTodos() {
    try {
      const response = await fetch(API_URL);
      const data = await response.json();

      const table = document.getElementById("todoTable");
      table.innerHTML = "";

      data.forEach(todo => {
        const row = document.createElement("tr");

        row.innerHTML = `
          <td>${todo.id}</td>
          <td>${todo.title}</td>
          <td>${todo.done ? "Selesai" : "Belum"}</td>
          <td>
            <button
onclick="selesaikanTodo(${todo.id})">Selesai</button>
            <button
onclick="hapusTodo(${todo.id})">Hapus</button>
          </td>
        `;

        table.appendChild(row);
      });
    } catch (error) {
      document.getElementById("pesan").textContent =
        "Gagal mengambil data dari API.";
      console.log(error);
    }
  }

  async function tambahTodo() {
    const input = document.getElementById("todoInput");
    const title = input.value.trim();
  }

```

```

        if (title === "") {
            document.getElementById("pesan").textContent =
                "Todo tidak boleh kosong.";
            return;
        }

        await fetch(API_URL, {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({ title: title })
        });

        input.value = "";
        document.getElementById("pesan").textContent = "";
        ambilTodos();
    }

    async function selesaikanTodo(id) {
        await fetch(`${API_URL}/${id}`, {
            method: "PUT"
        });

        ambilTodos();
    }

    async function hapusTodo(id) {
        await fetch(`${API_URL}/${id}`, {
            method: "DELETE"
        });

        ambilTodos();
    }

    ambilTodos();
</script>
</body>
</html>

```

Jalankan frontend:

```
python3 -m http.server 8080
```

Buka:

```
http://127.0.0.1:8080
```

Commit frontend:

```

cd ..
git add .
git commit -m "buat frontend Todo terhubung ke backend"

```

Langkah 5 — Android Client Sederhana

Untuk Bab 8, Android client cukup **membaca data Todo dari API**. Tidak perlu semua fitur CRUD dulu, karena halaman bab dibatasi 83–92 dan targetnya portofolio awal.

Konsep Compose yang dipakai:

- `Text` untuk judul,
- `Button` untuk ambil data,
- `LazyColumn` untuk daftar,
- `state` untuk menyimpan teks hasil API.

Dokumentasi Compose menjelaskan bahwa `state` menentukan apa yang ditampilkan UI, dan Compose memperbarui UI ketika `state` berubah. ([Android Developers](#))

Contoh logika minimal di `MainActivity.kt`:

```
var hasil by remember { mutableStateOf("Belum ada data")
}

Column {
    Text("Todo Android Client")

    Button(onClick = {
        hasil = "Ambil data dari API..."
    }) {
        Text("Ambil Todo")
    }

    Text(hasil)
}
```

Untuk build dari CLI, gunakan Gradle Wrapper bawaan project:

```
./gradlew assembleDebug
```

Gradle merekomendasikan penggunaan *Wrapper* karena project dapat menjalankan versi Gradle yang sudah dideklarasikan tanpa harus mengandalkan instalasi sistem. (docs.gradle.org)

Simpan catatan Android:

```
nano android-docs/ANDROID_CLIENT.md
```

Isi:

```
# Android Client Sederhana
```

Target awal:

- menampilkan layar Todo
- tombol ambil data
- menampilkan hasil dari API

Build:

```
./gradlew assembleDebug
```

Catatan:

Pastikan backend berjalan di alamat yang dapat diakses oleh emulator/perangkat.

Commit:

```
git add .  
git commit -m "tambah dokumentasi Android client sederhana"
```

Langkah 6 — Menulis Dokumentasi Proyek

Update **README.md**:

```
nano README.md
```

Isi ringkas:

```
# Todo AI-Ready

Aplikasi Todo sederhana sebagai proyek akhir buku AI
untuk Programming dan Coding Cepat.

## Komponen
- Backend: FastAPI
- Database: SQLite
- Frontend: HTML, CSS, JavaScript
- Android client: Kotlin + Jetpack Compose sederhana

## Cara Menjalankan Backend
cd backend
source .venv/bin/activate
uvicorn main:app --reload

## Cara Test API
curl http://127.0.0.1:8000/todos

## Cara Menjalankan Frontend
cd frontend
python3 -m http.server 8080

## Buka:
http://127.0.0.1:8080
```

Endpoint

Method	Endpoint	Fungsi
GET	/todos	Lihat daftar Todo
POST	/todos	Tambah Todo
PUT	/todos/{id}	Tandai selesai
DELETE	/todos/{id}	Hapus Todo

Commit dokumentasi:

```
git add README.md
git commit -m "lengkapi dokumentasi proyek"
```

Langkah 7 — Membuat Screenshot Hasil

Buat folder screenshot sudah ada:

```
ls screenshots
```

Simpan screenshot manual ke folder **screenshots/**, misalnya:

```
screenshots/frontend-todo.png
screenshots/api-docs.png
screenshots/android-client.png
```

Tambahkan ke Git:

```
git add screenshots
git commit -m "tambah screenshot hasil proyek"
```

Langkah 8 — Commit Final

Cek status:

```
git status
```

Jika masih ada perubahan:

```
git add .
git commit -m "finalisasi proyek Todo AI-Ready"
```

Lihat riwayat:

```
git log --oneline
```

8.6 Checklist Proyek

Komponen	Status
Backend jalan	☐
Database tersimpan	☐
Endpoint <code>GET /todos</code> berjalan	☐
Endpoint <code>POST /todos</code> berjalan	☐
Endpoint <code>PUT /todos/{id}</code> berjalan	☐
Endpoint <code>DELETE /todos/{id}</code> berjalan	☐
Frontend bisa tambah data	☐
Frontend bisa menampilkan data	☐
Frontend bisa update status	☐
Frontend bisa hapus data	☐
Android bisa tampil data sederhana	☐
README tersedia	☐
Screenshot hasil tersedia	☐
Error utama sudah diperbaiki	☐
Kode sudah di-commit	☐

Prompt AI untuk Audit Proyek

Gunakan setelah proyek berjalan:

Saya membuat proyek Todo AI-Ready.

Komponen:

- FastAPI backend
- SQLite database
- frontend HTML/JavaScript
- Android client sederhana

Tolong audit proyek ini:

1. endpoint apa saja yang harus diuji,
2. error umum apa yang mungkin muncul,
3. bagian README apa yang perlu dilengkapi,
4. perbaikan prioritas untuk pemula,
5. jangan menyarankan fitur besar dulu.

Prompt untuk debugging:

Saya mendapat error berikut saat menjalankan proyek Todo:

[paste error]

Konteks:

- backend FastAPI
- database SQLite
- frontend memakai fetch()
- dijalankan lokal

Tolong jelaskan penyebab paling mungkin,
berikan urutan pengecekan,
dan perbaikan minimal.

Ringkasan Bab

Bab ini menyatukan seluruh perjalanan buku menjadi satu proyek kecil tetapi lengkap. Pembaca membuat backend FastAPI, menyimpan data ke SQLite, membuat frontend web, menyiapkan Android client sederhana, menulis README, membuat screenshot, dan menyimpan semua perubahan dengan Git.

Hasil akhirnya bukan hanya aplikasi Todo, tetapi **pola kerja programmer**: membuat fitur kecil, menguji dari CLI, membaca JSON, memperbaiki error, mendokumentasikan hasil, dan melakukan commit. AI dipakai sebagai asisten, tetapi pembaca tetap menjadi pemilik keputusan.

Golden rule Bab 8:

Portofolio pertama tidak harus besar; yang penting selesai, berjalan, terdokumentasi, diuji, dan bisa dijelaskan.

BAB 9. Strategi Belajar, Debugging, Testing, dan Anti-Halusinasi Kode

9.1 Workflow Harian Programmer AI

Workflow harian yang dipakai dalam buku ini:

```
Rancang → Generate → Jalankan → Test → Pahami → Refactor → Commit
```

Penjelasan singkat:

- **Rancang:** tulis dulu tujuan program, input, output, dan batasannya.
- **Generate:** minta AI membuat draf kode.
- **Jalankan:** coba kode dari CLI.
- **Test:** uji input normal, salah, kosong, dan ekstrem.
- **Pahami:** minta AI menjelaskan bagian yang belum dipahami.
- **Refactor:** rapikan kode tanpa mengubah fungsi utama.
- **Commit:** simpan perubahan dengan Git.

Perintah dasar harian:

```
git status
python3 main.py
pytest
git add .
git commit -m "perbaiki validasi input"
```

Git penting karena `git commit` membuat commit baru dari isi *staging area* dengan pesan perubahan. Dengan kata lain, commit adalah titik simpan resmi dalam riwayat proyek. ([Git SCM](#))

Golden rule workflow:

Jangan generate kode baru sebelum kode lama dijalankan, diuji, dan dipahami.

9.2 Prompt Coding yang Baik

Prompt coding yang baik harus jelas, pendek, dan punya batasan. Jangan hanya menulis: “buatkan aplikasi”. Itu terlalu kabur. AI akan menebak terlalu banyak.

Struktur prompt yang disarankan:

Tujuan:
Konteks:
Bahasa:
Batasan:
Input:
Output:
Style kode:
Cara penjelasan:

Contoh prompt:

Tujuan:
Buat program Python untuk menghitung total pengeluaran dari file CSV.

Konteks:
Saya pemula Python dan sedang belajar file CSV.

Bahasa:
Python 3.

Batasan:
Gunakan modul csv bawaan Python. Jangan gunakan library tambahan.

Input:
File pengeluaran.csv dengan kolom tanggal,kategori,jumlah.

Output:
File summary.csv berisi kategori,total.

Style kode:
Sederhana, mudah dibaca, komentar singkat.

Cara penjelasan:
Jelaskan kode per bagian untuk pemula.

Prompt untuk memperbaiki kode:

```
Perbaiki kode berikut secara minimal.  
Jangan ubah struktur besar.  
Tunjukkan baris yang diubah.  
Jelaskan alasan perubahan.  
Tambahkan satu contoh test manual.
```

Prompt seperti ini membuat AI bekerja sebagai **asisten teknis**, bukan sebagai mesin yang mengambil alih seluruh keputusan.

9.3 Debugging dengan AI

Debugging adalah proses mencari penyebab error. AI sangat membantu untuk membaca error panjang, tetapi pembaca tetap harus paham urutan pengecekannya.

Prompt debugging:

```
Saya mendapat error berikut.  
  
[paste error]  
  
Jelaskan penyebabnya dalam bahasa sederhana.  
Berikan 3 kemungkinan solusi.  
Jangan langsung ubah semua kode.  
Tunjukkan bagian kode yang perlu diperiksa.
```

Contoh error Python:

```
ValueError: invalid literal for int() with base 10: 'abc'
```

Prompt ke AI:

```
Saya mendapat error Python:  
  
ValueError: invalid literal for int() with base 10: 'abc'  
  
Jelaskan:  
1. apa arti error ini,  
2. mengapa terjadi,  
3. baris seperti apa yang perlu diperiksa,  
4. perbaiki minimal,  
5. test apa yang harus saya lakukan setelah diperbaiki.
```

Urutan debugging yang sehat:

```
Baca error terakhir → cari baris penyebab → pahami tipe error  
→ perbaiki kecil → jalankan ulang → test
```

Jangan langsung meminta AI menulis ulang seluruh program. Itu membuat pembaca kehilangan kesempatan belajar. **Debugging yang baik memperbaiki akar masalah, bukan mengganti semua kode.**

9.4 Testing Kode AI

Kode AI harus diuji. Kode yang tampak rapi belum tentu benar. Testing adalah cara memastikan program bekerja sesuai harapan.

Untuk Python, pembaca bisa mulai dari `unittest` bawaan atau `pytest`. Dokumentasi Python menjelaskan `unittest` sebagai *unit testing framework* bawaan Python, sedangkan dokumentasi `pytest` menyebut framework ini memudahkan penulisan test kecil yang mudah dibaca dan dapat berkembang untuk testing aplikasi yang lebih kompleks. ([Python documentation](#))

Contoh Unit Test Sederhana

Buat file:

```
mkdir bab9-testing
cd bab9-testing
nano kalkulator.py
```

Isi:

```
def tambah(a, b):
    return a + b

def bagi(a, b):
    if b == 0:
        raise ValueError("Pembagi tidak boleh nol")
    return a / b
```

Buat test:

```
nano test_kalkulator.py
```

Isi:

```
from kalkulator import tambah, bagi

def test_tambah():
    assert tambah(2, 3) == 5

def test_bagi():
    assert bagi(10, 2) == 5

def test_bagi_nol():
```

```

try:
    bagi(10, 0)
    assert False
except ValueError:
    assert True

```

Install dan jalankan **pytest**:

```

python3 -m venv .venv
source .venv/bin/activate
pip install pytest
pytest

```

Dokumentasi **pytest** menyediakan langkah *getting started*, termasuk membuat test pertama, menjalankan banyak test, dan menguji exception. (docs.pytest.org)

Jenis Test Minimal

Jenis Test	Tujuan
Unit test sederhana	memastikan fungsi kecil bekerja
Test manual	mencoba program dari CLI
Test endpoint API	menguji API dengan curl
Test input salah	melihat apakah program tahan error
Test data kosong	memastikan program tidak crash
Test data besar	melihat performa dasar

Contoh test endpoint:

```

curl http://127.0.0.1:8000/todos
curl -X POST http://127.0.0.1:8000/todos \
  -H "Content-Type: application/json" \
  -d '{"title":"Belajar testing"}'

```

Golden rule testing:

Jangan hanya test kondisi benar. Test juga kondisi salah, kosong, dan ekstrem.

9.5 Anti-Halusinasi Kode

Hallucination dalam AI coding terjadi ketika AI memberi jawaban yang terlihat meyakinkan tetapi salah. Bentuknya bisa bermacam-macam:

- library yang sebenarnya tidak ada,
- fungsi yang namanya salah,
- parameter yang tidak didukung,
- kode yang jalan tetapi logikanya keliru,
- contoh API yang sudah tidak berlaku,
- saran keamanan yang lemah,
- solusi terlalu rumit untuk masalah sederhana.

Jangan percaya kode AI sebelum:

- **dijalankan,**
- **dibaca,**
- **diuji,**
- **dibandingkan dengan dokumentasi resmi,**
- **dipahami,**
- **di-commit dengan catatan yang jelas.**

Untuk keamanan, gunakan prinsip dari OWASP. OWASP Secure Coding Practices adalah panduan praktik coding aman yang bersifat teknologi-agnostik dan dapat diterjemahkan menjadi kebutuhan coding, meski developer belum mendalami semua detail eksploitasi keamanan. (owasp.org)

Checklist anti-halusinasi:

Pertanyaan	Ya/Tidak
Apakah library yang disebut AI benar-benar ada?	☐
Apakah fungsi yang dipakai ada di dokumentasi resmi?	☐
Apakah kode sudah dijalankan?	☐
Apakah hasilnya sesuai kebutuhan?	☐
Apakah input salah sudah diuji?	☐
Apakah ada risiko data sensitif bocor?	☐
Apakah kode terlalu rumit untuk pemula?	☐
Apakah sudah di-commit?	☐

Prompt anti-halusinasi:

Periksa kode ini dari sisi halusinasi AI.

Tolong cek:

1. apakah ada library atau fungsi yang mencurigakan,
2. apakah ada asumsi yang tidak dijelaskan,
3. apakah ada risiko keamanan dasar,
4. apakah ada bagian yang perlu dicek ke dokumentasi resmi,
5. berikan perbaikan minimal.

9.6 Kesalahan Umum Pemula

Kesalahan pemula bukan hal memalukan. Yang berbahaya adalah mengulanginya tanpa sadar.

Kesalahan	Risiko	Solusi
Copy-paste buta	Tidak paham kode	Minta AI jelaskan baris demi baris
Tidak pakai Git	Sulit <i>rollback</i>	Commit setiap fitur kecil
Tidak membaca error	Debug lambat	Baca pesan error dari bawah dan cari baris penyebab
Terlalu banyak tool	Bingung	Fokus Python → Web → Android
Tidak testing	Kode rapuh	Buat test minimal
Tidak dokumentasi	Sulit lanjut	Buat README sejak awal
Langsung membuat fitur besar	Proyek tidak selesai	Mulai dari fitur kecil
Tidak cek dokumentasi resmi	Terkena halusinasi AI	Verifikasi library dan API

Contoh commit kecil:

```
git add .
git commit -m "tambah validasi input kosong"
```

Lihat riwayat commit:

```
git log --oneline
```

Dokumentasi Git menjelaskan bahwa `git log` menampilkan daftar commit dalam urutan kronologis terbalik, sehingga commit terbaru muncul lebih dulu. ([Git SCM](#))

9.7 Roadmap Setelah Buku Ini

Setelah menyelesaikan buku ini, pembaca sebaiknya tidak langsung mengejar teknologi paling rumit. Lanjutkan secara bertahap.

Tahap 1 — Perkuat Dasar

- Python lanjut.
- Struktur data.
- Fungsi dan modul.
- Error handling.
- File, CSV, JSON.
- Unit test.

Tahap 2 — Web dan Database

- FastAPI lanjut.
- Validasi input lebih rapi.
- Database PostgreSQL/MariaDB.
- Autentikasi sederhana.
- Logging.
- Testing endpoint API.

Tahap 3 — Lingkungan Produksi

- Docker Compose.
- Deployment ke VPS Linux.
- Reverse proxy.
- Backup database.
- Monitoring sederhana.
- Keamanan aplikasi.

Tahap 4 — Android Lanjut

- Jetpack Compose lanjut.
- Navigasi antar layar.
- Konsumsi API.
- Penyimpanan lokal.
- Error handling jaringan.

Tahap 5 — AI Lokal dan RAG

- Ollama untuk eksperimen lokal.
- Prompt engineering lebih disiplin.
- RAG sederhana untuk dokumen proyek.
- Dokumentasi internal.
- Evaluasi jawaban AI.

Tahap 6 — Keamanan dan Kualitas

- Secure coding.
- Validasi input.
- Dependency review.
- Test otomatis.
- Code review.
- Dokumentasi teknis.

Roadmap ini menjaga pembaca tetap fokus. Jangan melompat terlalu jauh sebelum dasar kuat. **Kecepatan terbaik adalah kecepatan yang bisa dipertahankan.**

Hands-on Penutup — Audit Proyek Todo

Gunakan proyek Todo dari Bab 8. Dari folder proyek:

```
git status
git log --oneline
```

Jalankan backend:

```
cd backend
source .venv/bin/activate
uvicorn main:app --reload
```

Test API:

```
curl http://127.0.0.1:8000/todos

curl -X POST http://127.0.0.1:8000/todos \
  -H "Content-Type: application/json" \
  -d '{"title":"Audit proyek akhir"}'
```

Buat file checklist:

```
cd ..
nano CHECKLIST_FINAL.md
```

Isi:

```
# Checklist Final Proyek Todo

- [ ] Backend berjalan
- [ ] Database tersimpan
- [ ] Frontend bisa tambah data
- [ ] Endpoint sudah dites dengan curl
- [ ] Input kosong sudah diuji
- [ ] README tersedia
- [ ] Screenshot tersedia
- [ ] Error utama sudah dicatat
- [ ] Semua perubahan sudah di-commit
```

Commit:

```
git add .
git commit -m "tambah checklist final proyek"
```

Prompt AI untuk audit akhir:

Saya sudah membuat proyek Todo dengan FastAPI, SQLite, frontend web, dan Android client sederhana.

Tolong audit proyek ini dari sisi:

1. bug dasar,
2. test yang belum dilakukan,
3. dokumentasi yang kurang,
4. risiko keamanan pemula,
5. perbaikan prioritas.

Jangan sarankan fitur besar dulu.

Ringkasan Bab

Bab ini menegaskan bahwa AI coding bukan sekadar membuat kode cepat. Programmer harus punya workflow: **rancang, generate, jalankan, test, pahami, refactor, commit**. Kode AI harus diuji dengan input benar, salah, kosong, dan ekstrem. Error harus dibaca. Dokumentasi harus dibuat. Git harus dipakai sejak awal.

Setelah buku ini selesai, pembaca sudah memiliki fondasi untuk lanjut ke Python, FastAPI, database, Android, deployment, AI lokal, RAG, testing, dan keamanan aplikasi. Yang paling penting: pembaca tidak lagi menjadi pengguna AI pasif, tetapi mulai menjadi **programmer yang memimpin AI**.

Golden rule Bab 9:

AI mempercepat coding, tetapi disiplin testing, debugging, dokumentasi, keamanan, dan tanggung jawab tetap milik manusia.

Lampiran A. Template Prompt AI untuk Membuat Kode

Lampiran ini berisi template prompt praktis untuk membantu pembaca memakai AI sebagai **asisten coding**, bukan pengganti programmer. Gunakan template ini saat membuat kode baru, menambah fitur, belajar konsep, debugging, refactoring, dan dokumentasi.

Prinsip utama:

Prompt yang baik harus jelas: tujuan, bahasa, batasan, input, output, dan cara penjelasan.

A.1 Template Prompt Dasar

Kebutuhan	Template Prompt
Membuat kode baru	"Buatkan program [tujuan] dengan [bahasa] . Gunakan kode sederhana untuk pemula. Jelaskan langkahnya."
Membuat fitur	"Tambahkan fitur [fitur] pada kode berikut. Jangan mengubah struktur besar."
Belajar konsep	"Jelaskan konsep [konsep] dengan contoh kode kecil."

A.2 Template Prompt Lengkap

Gunakan format ini jika ingin hasil yang lebih rapi.

Tujuan:

[Buat apa?]

Konteks:

[Saya pemula / mahasiswa / programmer awal. Jelaskan kebutuhan program.]

Bahasa:

[Python / JavaScript / Kotlin / lainnya]

Batasan:

[Gunakan kode sederhana. Jangan gunakan library tambahan. Jangan ubah struktur besar.]

Input:

[Data yang masuk ke program]

Output:

[Hasil yang diharapkan]

Style kode:

[Mudah dibaca, komentar singkat, cocok untuk pemula]

Cara penjelasan:

[Jelaskan langkah demi langkah / jelaskan per bagian / jelaskan baris penting]

A.3 Contoh Prompt Membuat Kode Baru

Buatkan program Python untuk menghitung luas persegi panjang.

Syarat:

- gunakan input dari keyboard
- gunakan kode sederhana untuk pemula
- jangan gunakan library tambahan
- tampilkan contoh input dan output
- jelaskan langkah program secara singkat

Contoh lain:

Buatkan program Python untuk membaca file CSV berisi kolom nama,nilai.

Tugas:

- baca file nilai.csv
- tampilkan nama dan nilai
- jika nilai ≥ 75 , tampilkan status Lulus
- jika nilai < 75 , tampilkan status Perlu Belajar
- gunakan modul csv bawaan Python
- jelaskan kode per bagian

A.4 Contoh Prompt Menambahkan Fitur

Tambahkan fitur validasi input kosong pada kode berikut.

Syarat:

- jangan mengubah struktur besar
- berikan perubahan minimal
- jelaskan bagian yang diubah
- tambahkan contoh input dan output

Kode:

[paste kode di sini]

Contoh untuk API:

Tambahkan endpoint GET /todos/{id} pada aplikasi FastAPI berikut.

Syarat:

- ambil data dari SQLite
- jika data tidak ditemukan, tampilkan pesan sederhana
- jangan mengubah struktur besar
- jelaskan perubahan singkat

Kode:

[paste kode di sini]

A.5 Contoh Prompt Belajar Konsep

Jelaskan konsep list dan dict di Python untuk pemula.

Syarat:

- gunakan contoh kode kecil
- berikan perbandingan sederhana
- jelaskan kapan memakai list dan kapan memakai dict
- berikan 3 latihan singkat

Contoh untuk web:

Jelaskan konsep request dan response pada API.

Syarat:

- gunakan bahasa sederhana
- berikan contoh dengan FastAPI
- tunjukkan contoh response JSON
- jelaskan bagaimana frontend membaca response tersebut

A.6 Template Prompt Review Kode

Review kode berikut.

Tolong cek:

1. apakah ada bug dasar,
2. apakah ada input yang belum divalidasi,
3. apakah ada bagian yang terlalu rumit untuk pemula,
4. apakah ada risiko error,
5. berikan perbaikan minimal.

Jangan ubah seluruh struktur kode.

Kode:

[paste kode di sini]

A.7 Template Prompt Anti-Halusinasi

Periksa jawaban dan kode berikut dari kemungkinan halusinasi AI.

Tolong cek:

1. apakah library yang dipakai benar-benar umum/dikenal,
2. apakah fungsi yang dipakai masuk akal,
3. apakah ada asumsi yang belum dijelaskan,
4. apakah ada bagian yang perlu dicek ke dokumentasi resmi,
5. apakah kode bisa dijalankan oleh pemula.

Berikan catatan kritis dan perbaikan minimal.

A.8 Template Prompt untuk Debugging

Saya mendapat error berikut:

[paste error]

Ini kode saya:

[paste kode]

Tolong:

1. jelaskan arti error dengan bahasa pemula,
2. tunjukkan kemungkinan baris penyebab,
3. berikan 3 kemungkinan solusi,
4. berikan perbaikan minimal,
5. jangan ubah seluruh kode.

A.9 Template Prompt Refactoring

Rapikan kode berikut agar lebih mudah dibaca.

Syarat:

- jangan mengubah fungsi utama
- jangan menambah fitur baru
- pisahkan ke fungsi kecil jika perlu
- gunakan nama variabel yang jelas
- tambahkan komentar hanya jika membantu
- jelaskan perubahan singkat

Kode:

[paste kode di sini]

A.10 Template Prompt Dokumentasi

Buatkan README.md untuk proyek berikut.

Isi README:

- tujuan proyek
- fitur utama
- struktur folder
- cara install
- cara menjalankan
- contoh penggunaan
- cara test
- masalah umum dan solusi

Konteks proyek:

[paste deskripsi proyek]

A.11 Checklist Prompt yang Baik

Pertanyaan	Ya/Tidak
Apakah tujuan program jelas?	☐
Apakah bahasa pemrograman disebutkan?	☐
Apakah input dijelaskan?	☐
Apakah output dijelaskan?	☐
Apakah batasan ditulis?	☐
Apakah diminta kode sederhana untuk pemula?	☐
Apakah diminta penjelasan?	☐
Apakah diminta perubahan minimal jika memperbaiki kode?	☐

Golden Rule Lampiran A

Prompt yang baik tidak hanya meminta kode, tetapi juga meminta batasan, penjelasan, contoh input-output, dan cara menguji hasilnya.

Lampiran B. Template Prompt Debugging

Lampiran ini berisi template prompt untuk membantu pembaca memakai AI saat menghadapi **error**, *bug*, atau perilaku program yang tidak sesuai harapan. Tujuannya bukan agar AI langsung menulis ulang seluruh kode, tetapi agar pembaca **paham penyebab error** dan bisa memperbaikinya secara bertahap.

Prinsip utama:

Debugging yang baik memperbaiki akar masalah, bukan mengganti semua kode.

B.1 Template Prompt Debugging Dasar

Saya mendapat error berikut:

[paste error]

Ini kode saya:

[paste kode]

Tolong:

1. jelaskan penyebab error,
2. tunjukkan baris yang bermasalah,
3. berikan solusi minimal,
4. jangan ubah seluruh kode,
5. jelaskan agar pemula paham.

B.2 Template Prompt Debugging yang Lebih Lengkap

Gunakan template ini jika error cukup panjang atau program memiliki beberapa file.

Saya sedang belajar coding dan mendapat error berikut:

[paste error lengkap]

Konteks:

- Bahasa pemrograman: [Python / JavaScript / Kotlin / lainnya]
- Sistem operasi: [Linux/Ubuntu/dll]
- Cara menjalankan program: [contoh: python3 main.py]
- Tujuan program: [jelaskan singkat]
- File yang error: [nama file]

Ini kode saya:

[paste kode]

Tolong bantu saya:

1. jelaskan arti error dalam bahasa sederhana,
2. tunjukkan kemungkinan baris penyebab error,
3. jelaskan mengapa error itu terjadi,
4. berikan solusi minimal,
5. jangan ubah seluruh struktur kode,
6. berikan contoh cara menjalankan ulang,
7. berikan test sederhana setelah diperbaiki.

B.3 Template Debugging Python

Saya mendapat error Python berikut:

[paste traceback lengkap]

Ini kode saya:

[paste kode Python]

Tolong:

1. baca traceback dari bawah ke atas,
2. jelaskan tipe error,
3. tunjukkan baris yang menyebabkan error,
4. jelaskan penyebabnya untuk pemula,
5. berikan perbaikan minimal,
6. tambahkan contoh input-output setelah diperbaiki.

Contoh kasus:

Saya mendapat error Python berikut:

ValueError: invalid literal for int() with base 10: 'abc'

Ini kode saya:

```
angka = int(input("Masukkan angka: "))  
print(angka * 2)
```

Tolong:

1. jelaskan penyebab error,
2. tunjukkan baris yang bermasalah,
3. berikan solusi minimal,
4. jangan ubah seluruh kode,
5. jelaskan agar pemula paham.

B.4 Template Debugging FastAPI

Saya mendapat error saat menjalankan FastAPI:

[paste error / traceback]

Konteks:

- Backend: FastAPI
- Database: [SQLite / lainnya]
- Cara menjalankan: `uvicorn main:app --reload`
- Endpoint yang diuji: [GET/POST/PUT/DELETE + URL]
- Cara test: [curl atau browser]

Ini kode endpoint saya:

[paste kode]

Tolong:

1. jelaskan penyebab error,
2. tunjukkan bagian kode yang perlu diperiksa,
3. berikan solusi minimal,
4. jangan ubah struktur aplikasi besar,
5. berikan perintah curl untuk test ulang.

B.5 Template Debugging Frontend fetch()

Saya mendapat error `fetch()` di frontend:

[paste error dari console browser]

Konteks:

- Frontend: HTML, CSS, JavaScript murni
- Backend: FastAPI
- URL API: [contoh: `http://127.0.0.1:8000/todos`]
- Cara menjalankan frontend: `python3 -m http.server 8080`

Ini kode JavaScript saya:

[paste kode]

Tolong:

1. jelaskan penyebab paling mungkin,
2. cek apakah URL API sudah benar,
3. cek kemungkinan CORS,
4. cek apakah backend sudah berjalan,
5. berikan solusi minimal,
6. jangan ubah seluruh kode.

B.6 Template Debugging Android / Kotlin

Saya mendapat error build Android/Kotlin berikut:

[paste error build]

Konteks:

- Bahasa: Kotlin
- UI: Jetpack Compose
- File yang diubah: MainActivity.kt
- Tujuan kode: [jelaskan singkat]

Ini kode saya:

[paste kode]

Tolong:

1. jelaskan arti error,
2. tunjukkan kemungkinan baris penyebab,
3. cek apakah ada import yang kurang,
4. cek apakah ada salah penulisan fungsi/komponen,
5. berikan perbaikan minimal,
6. jangan ubah seluruh struktur aplikasi.

B.7 Template Debugging Logic Error

Kadang program tidak error, tetapi hasilnya salah. Ini disebut **logic error**.

Program saya bisa berjalan, tetapi hasilnya salah.

Tujuan program:

[jelaskan tujuan program]

Hasil yang saya harapkan:

[jelaskan output yang benar]

Hasil yang muncul:

[jelaskan output yang salah]

Ini kode saya:

[paste kode]

Tolong:

1. cari kemungkinan kesalahan logika,
2. jelaskan alur program saat ini,
3. tunjukkan bagian yang perlu diperiksa,
4. berikan perbaikan minimal,
5. tambahkan contoh test agar kesalahan ini tidak terulang.

B.8 Template Debugging Input Kosong / Salah

Program saya error ketika input kosong atau input salah.

Contoh input bermasalah:

[paste input]

Error / hasil yang muncul:

[paste error atau output]

Ini kode saya:

[paste kode]

Tolong:

1. jelaskan mengapa input ini membuat program bermasalah,
2. tambahkan validasi input sederhana,
3. jangan ubah struktur besar,
4. berikan contoh input valid dan input tidak valid,
5. jelaskan agar pemula paham.

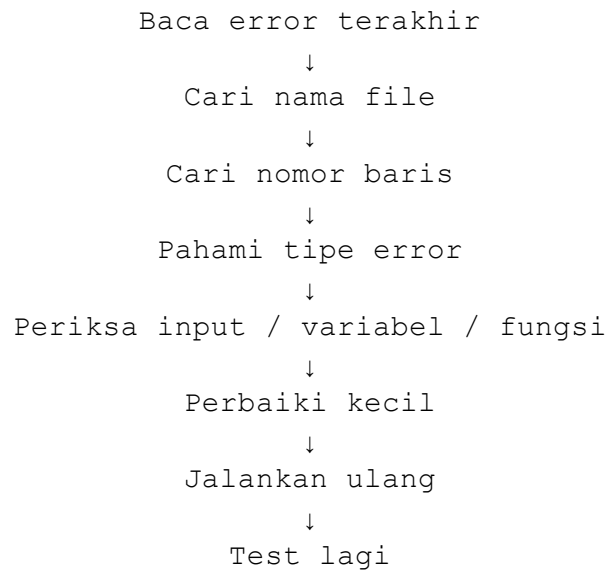
B.9 Checklist Sebelum Bertanya ke AI

Sebelum menempelkan error ke AI, cek dulu:

Pertanyaan	Ya/Tidak
Apakah error sudah disalin lengkap?	☐
Apakah kode yang relevan sudah disertakan?	☐
Apakah bahasa pemrograman disebutkan?	☐
Apakah cara menjalankan program dijelaskan?	☐
Apakah input yang menyebabkan error disertakan?	☐
Apakah output yang diharapkan dijelaskan?	☐
Apakah diminta solusi minimal?	☐
Apakah diminta agar struktur kode tidak diubah total?	☐

B.10 Pola Membaca Error

Biasakan membaca error dengan urutan berikut:



Contoh:

```
ValueError: invalid literal for int() with base 10: 'abc'
```

Maknanya:

- `ValueError` = nilai tidak sesuai.
- `int()` mencoba mengubah teks menjadi angka.
- `'abc'` bukan angka.
- Solusi minimal: gunakan `try-except` atau validasi input.

B.11 Contoh Prompt Siap Pakai

Saya mendapat error Python berikut:

```
ValueError: invalid literal for int() with base 10: 'abc'
```

Ini kode saya:

```
angka = int(input("Masukkan angka: "))  
print(angka * 2)
```

Tolong:

1. jelaskan penyebab error,
2. tunjukkan baris yang bermasalah,
3. berikan solusi minimal,
4. jangan ubah seluruh kode,
5. jelaskan agar pemula paham.

Kemungkinan perbaikan minimal:

```
try:  
    angka = int(input("Masukkan angka: "))  
    print(angka * 2)  
except ValueError:  
    print("Error: input harus berupa angka.")
```

Golden Rule Lampiran B

Saat debugging dengan AI, jangan hanya minta jawaban. Minta penjelasan penyebab, baris bermasalah, solusi minimal, dan cara mengetes ulang.

Lampiran C. Template Prompt Refactoring

Lampiran ini berisi template prompt untuk membantu pembaca memakai AI dalam proses **refactoring**, yaitu merapikan kode agar lebih mudah dibaca, lebih mudah diuji, dan lebih mudah dikembangkan tanpa mengubah fungsi utamanya.

Prinsip utama:

Refactoring bukan menambah fitur. Refactoring adalah merapikan kode agar perilakunya tetap sama, tetapi strukturnya lebih sehat.

C.1 Template Prompt Refactoring Dasar

Rapikan kode berikut agar lebih mudah dibaca.
Jangan mengubah fungsi utama.
Pisahkan ke fungsi kecil jika perlu.
Tambahkan komentar hanya jika membantu pemula.

Kode:
[paste kode di sini]

C.2 Template Prompt Refactoring Lengkap

Gunakan template ini jika kode sudah agak panjang.

Saya ingin melakukan refactoring kode berikut.

Tujuan:

- membuat kode lebih mudah dibaca
- mengurangi pengulangan
- memisahkan bagian yang terlalu panjang ke fungsi kecil
- menjaga fungsi program tetap sama

Batasan:

- jangan menambah fitur baru
- jangan mengubah output program
- jangan mengubah nama file
- jangan memakai library tambahan
- komentar hanya jika membantu pemula

Tolong:

1. jelaskan masalah utama pada kode,
2. berikan versi kode yang lebih rapi,
3. jelaskan perubahan yang dilakukan,
4. sebutkan bagian yang tidak diubah,
5. berikan test manual untuk memastikan hasil tetap sama.

Kode:
[paste kode di sini]

C.3 Template Refactoring Python Dasar

Refactor kode Python berikut agar lebih rapi.

Syarat:

- jangan ubah fungsi utama program
- pisahkan logika ke fungsi kecil
- gunakan nama variabel yang lebih jelas
- jangan gunakan library tambahan
- tambahkan komentar hanya jika perlu
- jelaskan perubahan untuk pemula

Kode:

[paste kode Python di sini]

Contoh kebutuhan:

Refactor kode kalkulator berikut.

Pisahkan operasi tambah, kurang, kali, dan bagi ke fungsi masing-masing.

Jangan ubah cara input dan output program.

C.4 Template Refactoring FastAPI

Refactor kode FastAPI berikut agar lebih mudah dibaca.

Syarat:

- jangan mengubah endpoint yang sudah ada
- jangan mengubah format response JSON
- pisahkan fungsi database jika perlu
- jangan menambah fitur baru
- jangan mengubah nama route
- jelaskan perubahan singkat

Kode:

[paste kode FastAPI di sini]

Contoh permintaan:

Rapikan kode FastAPI Todo berikut.

Pisahkan koneksi database, inisialisasi tabel, dan query Todo ke fungsi kecil.

Endpoint tetap:

- GET /todos
- POST /todos
- PUT /todos/{id}
- DELETE /todos/{id}

C.5 Template Refactoring Frontend

Refactor kode HTML, CSS, dan JavaScript berikut agar lebih mudah dibaca.

Syarat:

- jangan mengubah tampilan utama
- jangan mengubah URL API
- jangan mengubah fungsi utama aplikasi
- pisahkan fungsi JavaScript jika perlu
- rapikan nama variabel
- komentar hanya jika membantu pemula

Kode:

[paste kode frontend di sini]

Contoh permintaan:

Rapikan JavaScript berikut.

Pisahkan fungsi untuk:

1. mengambil data Todo,
2. menampilkan data ke tabel,
3. menambah Todo,
4. menampilkan pesan error.

Jangan ubah struktur HTML terlalu banyak.

C.6 Template Refactoring Android / Kotlin

Refactor kode Kotlin Jetpack Compose berikut agar lebih rapi.

Syarat:

- jangan menambah fitur baru
- jangan mengubah fungsi utama aplikasi
- pisahkan UI ke beberapa fungsi @Composable jika perlu
- gunakan nama fungsi yang mudah dipahami
- komentar hanya jika membantu pemula
- jelaskan perubahan singkat

Kode:

[paste kode Kotlin di sini]

Contoh permintaan:

Rapikan MainActivity.kt berikut.

Pisahkan menjadi:

- TodoApp()
- TodoInput()
- TodoList()

Jangan ubah perilaku aplikasi.

Tetap gunakan state sederhana untuk pemula.

C.7 Contoh Refactoring Sederhana

Kode awal:

```
print("Kalkulator")
a = float(input("Angka pertama: "))
b = float(input("Angka kedua: "))
op = input("Operasi (+/-): ")

if op == "+":
    print(a + b)
elif op == "-":
    print(a - b)
else:
    print("Operasi tidak dikenal")
```


Prompt:

Rapikan kode berikut agar lebih mudah dibaca.
Jangan mengubah fungsi utama.
Pisahkan ke fungsi kecil jika perlu.
Tambahkan komentar hanya jika membantu pemula.

Kode:

[paste kode]

Kemungkinan hasil refactoring:

```
def tambah(a, b):  
    return a + b  
  
def kurang(a, b):  
    return a - b  
  
def hitung(a, b, operasi):  
    if operasi == "+":  
        return tambah(a, b)  
    elif operasi == "-":  
        return kurang(a, b)  
    else:  
        return "Operasi tidak dikenal"  
  
print("Kalkulator")  
  
angka_pertama = float(input("Angka pertama: "))  
angka_kedua = float(input("Angka kedua: "))  
operasi = input("Operasi (+/-): ")  
  
hasil = hitung(angka_pertama, angka_kedua, operasi)  
print(hasil)
```

Perubahan utama:

- nama variabel dibuat lebih jelas,
- operasi dipisahkan ke fungsi,
- logika hitung dipusatkan di fungsi hitung,
- perilaku program tetap sama.

C.8 Checklist Refactoring

Pertanyaan	Ya/Tidak
Apakah fungsi utama program tetap sama?	☐
Apakah output masih sama untuk input yang sama?	☐
Apakah kode lebih mudah dibaca?	☐
Apakah nama variabel lebih jelas?	☐
Apakah fungsi terlalu panjang sudah dipisah?	☐
Apakah komentar tidak berlebihan?	☐
Apakah tidak ada fitur baru yang tidak diminta?	☐
Apakah kode sudah dites ulang?	☐
Apakah perubahan sudah di-commit?	☐

C.9 Prompt untuk Mengecek Hasil Refactoring

Bandingkan kode sebelum dan sesudah refactoring berikut.

Tolong cek:

1. apakah fungsi utama masih sama,
2. apakah ada output yang berubah,
3. apakah ada bug baru,
4. apakah kode sesudah refactoring lebih mudah dibaca,
5. test apa yang harus dilakukan untuk memastikan aman.

Kode sebelum:

[paste kode lama]

Kode sesudah:

[paste kode baru]

C.10 Prompt Refactoring Aman

Gunakan jika kode sudah berjalan dan tidak ingin rusak.

Kode berikut sudah berjalan.
Saya hanya ingin merapikan struktur.

Tolong refactor secara aman:

- jangan tambah fitur baru
- jangan ubah output
- jangan ubah format input
- jangan ganti library
- buat perubahan sekecil mungkin
- jelaskan setiap perubahan
- berikan test manual setelah refactoring

Kode:
[paste kode]

C.11 Kesalahan Umum Saat Refactoring

Kesalahan	Risiko	Solusi
Menambah fitur saat refactoring	Sulit tahu sumber bug baru	Pisahkan refactoring dan fitur baru
Mengubah terlalu banyak sekaligus	Sulit debugging	Refactor kecil-kecil
Tidak testing ulang	Bug tidak terlihat	Jalankan test setelah refactor
Komentar terlalu banyak	Kode jadi ramai	Komentar hanya untuk bagian penting
Nama fungsi tidak jelas	Kode tetap sulit dibaca	Gunakan nama sesuai tugas fungsi
Tidak commit sebelum refactor	Sulit kembali ke versi lama	Commit dulu sebelum perubahan besar

C.12 Pola Kerja Refactoring dengan Git

Sebelum refactoring:

```
git status
git add .
git commit -m "simpan versi sebelum refactoring"
```

Setelah refactoring dan test berhasil:

```
git add .
git commit -m "refactor kode agar lebih mudah dibaca"
```

Jika hasil refactoring merusak program, pembaca masih punya versi lama yang aman.

Golden Rule Lampiran C

Refactoring yang baik membuat kode lebih mudah dipahami tanpa mengubah perilaku program. Jika perilaku berubah, itu bukan sekadar refactoring—itu sudah menjadi perubahan fitur.

Lampiran D. Template Prompt Dokumentasi

Lampiran ini berisi template prompt untuk membantu pembaca membuat dokumentasi proyek, terutama file **README.md**. Dokumentasi penting karena kode yang bagus tetapi tidak bisa dijalankan orang lain akan sulit dipakai, sulit diuji, dan sulit dikembangkan.

Prinsip utama:

Dokumentasi yang baik menjawab tiga pertanyaan: proyek ini untuk apa, cara menjalankannya bagaimana, dan apa yang harus dilakukan jika terjadi error.

D.1 Template Prompt Dokumentasi Dasar

Buatkan README.md untuk proyek berikut.

Isi:

- tujuan proyek,
- cara install,
- cara menjalankan,
- struktur folder,
- contoh penggunaan,
- masalah umum dan solusi.

Konteks proyek:

[paste deskripsi proyek di sini]

D.2 Template Prompt Dokumentasi Lengkap

Gunakan template ini jika proyek sudah memiliki beberapa komponen, misalnya backend, frontend, database, dan Android client.

Buatkan README.md untuk proyek berikut.

Konteks:

[nama proyek]

[deskripsi singkat proyek]

Komponen:

- Backend: [contoh: FastAPI]
- Database: [contoh: SQLite]
- Frontend: [contoh: HTML, CSS, JavaScript]
- Android client: [jika ada]

Tolong buat README.md dengan isi:

1. judul proyek,
2. deskripsi singkat,
3. tujuan proyek,
4. fitur utama,
5. struktur folder,
6. kebutuhan sistem,
7. cara install,
8. cara menjalankan backend,
9. cara menjalankan frontend,
10. cara menjalankan test,
11. contoh penggunaan,
12. endpoint API jika ada,
13. masalah umum dan solusi,
14. catatan pengembangan berikutnya.

Syarat:

- gunakan bahasa Indonesia,
- mudah dipahami pemula,
- gunakan format Markdown,
- jangan terlalu panjang,
- sertakan contoh perintah CLI.

D.3 Template README.md untuk Proyek Python

Buatkan README.md untuk proyek Python berikut.

Nama proyek:
[nama proyek]

Tujuan:
[jelaskan tujuan proyek]

Fitur:
- [fitur 1]
- [fitur 2]
- [fitur 3]

Cara menjalankan:
- menggunakan Python 3
- dijalankan dari terminal

Tolong isi README dengan:
- deskripsi proyek,
- struktur folder,
- cara membuat virtual environment,
- cara install dependency,
- cara menjalankan program,
- contoh input dan output,
- masalah umum dan solusi.

Gunakan bahasa Indonesia dan format Markdown.

Contoh perintah yang sebaiknya masuk README:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 main.py
```

D.4 Template README.md untuk Proyek FastAPI

Buatkan README.md untuk proyek FastAPI berikut.

Konteks:

Aplikasi backend Todo sederhana dengan FastAPI dan SQLite.

Fitur:

- tambah Todo,
- lihat daftar Todo,
- update status Todo,
- hapus Todo,
- simpan data ke SQLite.

Tolong buat README.md berisi:

1. deskripsi proyek,
2. teknologi yang digunakan,
3. struktur folder,
4. cara install,
5. cara menjalankan server,
6. cara test endpoint dengan curl,
7. daftar endpoint,
8. contoh request dan response JSON,
9. masalah umum dan solusi.

Gunakan bahasa Indonesia, format Markdown, dan contoh CLI.

Contoh bagian endpoint:

Endpoint API

Method	Endpoint	Fungsi
GET	/todos	Melihat semua Todo
POST	/todos	Menambah Todo
PUT	/todos/{id}	Menandai Todo selesai
DELETE	/todos/{id}	Menghapus Todo

D.5 Template README.md untuk Proyek Frontend

Buatkan README.md untuk proyek frontend sederhana berikut.

Konteks:

Frontend Todo menggunakan HTML, CSS, dan JavaScript murni.
Frontend mengambil data dari backend FastAPI memakai `fetch()`.

Tolong buat README.md berisi:

- tujuan proyek,
- struktur folder,
- cara menjalankan server lokal,
- URL backend yang digunakan,
- cara menambah Todo,
- cara menampilkan data,
- masalah umum `fetch()` dan solusinya.

Syarat:

- gunakan bahasa Indonesia,
- mudah dipahami pemula,
- sertakan contoh perintah CLI,
- jangan gunakan framework tambahan.

Contoh perintah:

```
python3 -m http.server 8080
```

D.6 Template README.md untuk Proyek Android

Buatkan README.md untuk proyek Android sederhana berikut.

Konteks:

Aplikasi Todo lokal menggunakan Kotlin dan Jetpack Compose.

Tolong buat README.md berisi:

1. tujuan proyek,
2. fitur utama,
3. struktur project,
4. cara build dari CLI,
5. lokasi file APK debug,
6. cara menjalankan di emulator/perangkat,
7. masalah umum build dan solusi,
8. catatan untuk pengembangan berikutnya.

Gunakan bahasa Indonesia, format Markdown, dan contoh perintah CLI.

Contoh perintah:

```
./gradlew assembleDebug  
ls app/build/outputs/apk/debug/
```

D.7 Template README.md untuk Proyek Akhir Todo AI-Ready

Buatkan README.md untuk proyek akhir Todo AI-Ready.

Komponen:

- Backend: FastAPI
- Database: SQLite
- Frontend: HTML, CSS, JavaScript
- Android client: Kotlin + Jetpack Compose sederhana

Fitur:

- tambah Todo,
- lihat daftar Todo,
- update status Todo,
- hapus Todo,
- akses dari web,
- akses sederhana dari Android.

Tolong buat README.md berisi:

1. judul proyek,
2. deskripsi singkat,
3. tujuan proyek,
4. arsitektur sederhana,
5. struktur folder,
6. cara menjalankan backend,
7. cara menjalankan frontend,
8. cara test API dengan curl,
9. endpoint API,
10. cara build Android client,
11. screenshot yang perlu disiapkan,
12. masalah umum dan solusi,
13. roadmap pengembangan berikutnya.

Gunakan bahasa Indonesia, format Markdown, narasi pendek, dan contoh CLI.

D.8 Contoh Kerangka README.md

```
# Nama Proyek

Deskripsi singkat proyek.

## Tujuan

Proyek ini dibuat untuk:
- tujuan 1,
- tujuan 2,
- tujuan 3.

## Fitur

- Fitur 1
- Fitur 2
- Fitur 3

## Struktur Folder

```text
nama-proyek/
├── backend/
├── frontend/
├── README.md
└── screenshots/
```

### Kebutuhan Sistem

- Python 3
- Git
- Browser
- Terminal Linux/Ubuntu

### Cara Install

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

## Cara Menjalankan

```
python3 main.py
```

## Contoh Penggunaan

```
python3 main.py
```

Output:

```
Program berjalan dengan baik.
```

## Masalah Umum dan Solusi

Masalah	Penyebab	Solusi
Module tidak ditemukan	Dependency belum diinstall	Jalankan <code>pip install -r requirements.txt</code>
File tidak ditemukan	Path salah	Cek nama dan lokasi file
Program tidak jalan	Virtual environment belum aktif	Jalankan <code>source .venv/bin/activate</code>

## Catatan

Proyek ini dibuat untuk latihan dan pembelajaran.

```

D.9 Checklist README yang Baik

| **Pertanyaan** | **Ya/Tidak** |
|---|---|
| Apakah tujuan proyek jelas? | ☐ |
| Apakah fitur utama ditulis? | ☐ |
| Apakah struktur folder dijelaskan? | ☐ |
| Apakah cara install tersedia? | ☐ |
| Apakah cara menjalankan tersedia? | ☐ |
| Apakah contoh penggunaan tersedia? | ☐ |
| Apakah contoh CLI tersedia? | ☐ |
| Apakah masalah umum dan solusi ditulis? | ☐ |
| Apakah README cukup singkat dan mudah dibaca? | ☐ |
| Apakah README sudah diuji oleh orang lain atau dibaca ulang? | ☐ |

D.10 Prompt untuk Memperbaiki README
```

```text

Review README.md berikut.

Tolong cek:

1. apakah instruksi install jelas,
2. apakah cara menjalankan sudah lengkap,
3. apakah ada perintah CLI yang kurang,
4. apakah struktur folder mudah dipahami,
5. apakah masalah umum dan solusi sudah cukup,
6. apakah ada bagian yang terlalu panjang.

Berikan versi perbaikan yang lebih ringkas dan mudah dipahami pemula.

README:

[paste README di sini]

D.11 Prompt untuk Membuat Dokumentasi dari Kode

Buatkan dokumentasi singkat dari kode berikut.

Tolong jelaskan:

1. tujuan kode,
2. fungsi utama,
3. input yang dibutuhkan,
4. output yang dihasilkan,
5. cara menjalankan dari CLI,
6. contoh penggunaan,
7. error umum yang mungkin terjadi.

Kode:

[paste kode di sini]

D.12 Kesalahan Umum dalam Dokumentasi

| Kesalahan | Risiko | Solusi |
|----------------------------|--------------------------------|---------------------------------|
| Tidak ada cara install | Orang lain tidak bisa mencoba | Tulis langkah install dari awal |
| Tidak ada cara menjalankan | Proyek sulit dipakai | Sertakan perintah CLI |
| Terlalu banyak teori | README sulit dibaca | Fokus pada langkah praktis |
| Tidak ada contoh output | Pembaca tidak tahu hasil benar | Tambahkan contoh penggunaan |
| Tidak ada masalah umum | Debugging lambat | Buat tabel masalah dan solusi |
| README tidak diperbarui | Dokumentasi menyesatkan | Update setelah perubahan besar |

Golden Rule Lampiran D

README yang baik bukan yang panjang, tetapi yang membuat orang lain bisa menginstall, menjalankan, menguji, dan memahami proyek tanpa bertanya terlalu banyak.

Lampiran E. Checklist Setup Laptop Coding

Lampiran ini membantu pembaca memastikan laptop sudah siap untuk belajar **AI untuk Programming dan Coding Cepat**. Checklist ini sebaiknya diselesaikan sebelum masuk latihan Python, Web API, frontend, Android, dan proyek akhir.

Prinsip utama:

Laptop coding yang baik tidak harus mahal, tetapi harus rapi, bisa menjalankan terminal, Python, Git, AI lokal, dan tool pengembangan dasar.

E.1 Checklist Utama

| Komponen | Status |
|------------------------------------|--------------------------|
| Ubuntu/Linux siap | ☐ |
| Terminal bisa dipakai | ☐ |
| Python terinstall | ☐ |
| Git terinstall | ☐ |
| uv terinstall | ☐ |
| Editor kode siap | ☐ |
| Ollama terinstall | ☐ |
| Model AI lokal bisa jalan | ☐ |
| Docker terinstall | ☐ |
| Android Studio / tool Android siap | ☐ |

E.2 Cek Sistem Operasi

Pastikan laptop memakai Linux/Ubuntu atau distro Linux lain yang nyaman digunakan.

Cek versi sistem:

```
lsb_release -a
```

Alternatif:

```
cat /etc/os-release
```

Checklist:

| Pemeriksaan | Status |
|---------------------------------|--------------------------|
| Sistem operasi bisa boot normal | ☐ |
| Bisa login ke desktop | ☐ |
| Koneksi internet berjalan | ☐ |
| Update package bisa dilakukan | ☐ |

Update awal:

```
sudo apt update  
sudo apt upgrade -y
```

E.3 Cek Terminal

Terminal adalah pusat kerja utama untuk coding.

Cek perintah dasar:

```
pwd
ls
mkdir test-terminal
cd test-terminal
nano catatan.txt
```

Isi **catatan.txt**:

```
Terminal siap untuk belajar coding.
```

Cek isi folder:

```
ls
cat catatan.txt
```

Checklist:

| Kemampuan Terminal | Status |
|---|--------------------------|
| Bisa membuka terminal | ☐ |
| Bisa menjalankan pwd | ☐ |
| Bisa menjalankan ls | ☐ |
| Bisa membuat folder dengan mkdir | ☐ |
| Bisa pindah folder dengan cd | ☐ |
| Bisa membuat file dengan nano | ☐ |
| Bisa membaca file dengan cat | ☐ |

E.4 Cek Python

Install Python dan tool dasar:

```
sudo apt install python3 python3-pip python3-venv -y
```

Cek versi:

```
python3 --version  
pip --version
```

Buat program uji:

```
nano hello.py
```

Isi:

```
print("Python siap dipakai.")
```

Jalankan:

```
python3 hello.py
```

Checklist:

| Pemeriksaan Python | Status |
|---|--------------------------|
| <code>python3 --version</code> berhasil | ☐ |
| <code>pip --version</code> berhasil | ☐ |
| <code>python3 -m venv</code> tersedia | ☐ |
| File <code>.py</code> bisa dijalankan | ☐ |
| Output Python tampil di terminal | ☐ |

E.5 Cek Git

Install Git:

```
sudo apt install git -y
```

Cek versi:

```
git --version
```

Konfigurasi awal:

```
git config --global user.name "Nama Anda"  
git config --global user.email "email@contoh.local"
```

Buat repo uji:

```
mkdir test-git  
cd test-git  
git init  
echo "Latihan Git" > README.md  
git add README.md  
git commit -m "commit pertama"  
git log --oneline
```

Checklist:

| Pemeriksaan Git | Status |
|--------------------------------------|--------------------------|
| Git terinstall | ☐ |
| git init berhasil | ☐ |
| git status bisa digunakan | ☐ |
| git add bisa digunakan | ☐ |
| git commit berhasil | ☐ |
| git log --oneline menampilkan commit | ☐ |

E.6 Cek uv

Install **uv**:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Aktifkan ulang shell:

```
source ~/.bashrc
```

Cek versi:

```
uv --version
```

Buat proyek uji:

```
mkdir test-uv  
cd test-uv  
uv init  
uv run python --version
```

Checklist:

| Pemeriksaan uv | Status |
|---|--------------------------|
| uv --version berhasil | ☐ |
| uv init berhasil | ☐ |
| uv run python --version berhasil | ☐ |
| Proyek Python modern bisa dibuat | ☐ |

E.7 Cek Editor Kode

Gunakan editor kode yang nyaman dan sesuai kebutuhan. Yang penting editor bisa membuka folder proyek, membaca file Python, Markdown, HTML, JavaScript, dan Kotlin.

Checklist:

| Kemampuan Editor | Status |
|--|--------------------------|
| Editor bisa membuka folder proyek | ☐ |
| Editor bisa membuka file <code>.py</code> | ☐ |
| Editor bisa membuka file <code>.md</code> | ☐ |
| Editor bisa membuka file <code>.html</code> | ☐ |
| Editor bisa membuka file <code>.js</code> | ☐ |
| Editor bisa membuka terminal atau bekerja berdampingan dengan terminal | ☐ |

Test sederhana:

```
mkdir test-editor
cd test-editor
nano main.py
```

Isi:

```
print("Editor dan terminal siap.")
```

Jalankan:

```
python3 main.py
```

E.8 Cek Ollama

Install Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Cek service:

```
ollama --version  
sudo systemctl status ollama
```

Jalankan model lokal kecil:

```
ollama run qwen2.5-coder:7b
```

Prompt uji:

```
Buatkan program Python hello world dan jelaskan untuk pemula.
```

Cek daftar model:

```
ollama list
```

Checklist:

| Pemeriksaan Ollama | Status |
|--|--------------------------|
| Ollama terinstall | ☐ |
| Service Ollama berjalan | ☐ |
| Model lokal berhasil diunduh | ☐ |
| Model bisa menjawab prompt | ☐ |
| <code>ollama list</code> menampilkan model | ☐ |

E.9 Cek Docker

Install Docker Engine sesuai dokumentasi resmi untuk distro yang digunakan.

Cek versi:

```
docker --version
docker compose version
```

Test Docker:

```
docker run hello-world
```

Jika perlu menambahkan user ke grup Docker:

```
sudo usermod -aG docker $USER
```

Lalu logout dan login kembali.

Checklist:

| Pemeriksaan Docker | Status |
|---|--------------------------|
| Docker terinstall | ☐ |
| Docker service berjalan | ☐ |
| <code>docker --version</code> berhasil | ☐ |
| <code>docker compose version</code> berhasil | ☐ |
| <code>docker run hello-world</code> berhasil | ☐ |
| Docker bisa dijalankan tanpa error permission | ☐ |

E.10 Cek Tool Android

Untuk Android, pastikan tool pengembangan sudah bisa membuat project, melakukan build, dan menghasilkan APK debug.

Cek dari folder project Android:

```
./gradlew tasks
```

Build debug:

```
./gradlew assembleDebug
```

Cek APK:

```
ls app/build/outputs/apk/debug/
```

Checklist:

| Pemeriksaan Android | Status |
|---|--------------------------|
| Tool Android siap | ☐ |
| Project Android bisa dibuat | ☐ |
| File gradlew tersedia | ☐ |
| ./gradlew tasks berhasil | ☐ |
| ./gradlew assembleDebug berhasil | ☐ |
| APK debug berhasil dibuat | ☐ |
| Emulator atau perangkat uji tersedia | ☐ |

E.11 Checklist Final Setup

| Komponen | Perintah Cek | Status |
|----------------|--------------------------------------|--------------------------|
| Ubuntu/Linux | <code>cat /etc/os-release</code> | ☐ |
| Terminal | <code>pwd && ls</code> | ☐ |
| Python | <code>python3 --version</code> | ☐ |
| pip | <code>pip --version</code> | ☐ |
| Git | <code>git --version</code> | ☐ |
| uv | <code>uv --version</code> | ☐ |
| Ollama | <code>ollama --version</code> | ☐ |
| Model lokal | <code>ollama list</code> | ☐ |
| Docker | <code>docker --version</code> | ☐ |
| Docker Compose | <code>docker compose version</code> | ☐ |
| Android build | <code>./gradlew assembleDebug</code> | ☐ |

E.12 Masalah Umum dan Solusi

| Masalah | Kemungkinan Penyebab | Solusi |
|--------------------------------------|--|---|
| <code>python3</code> tidak ditemukan | Python belum terinstall | <code>sudo apt install python3 -y</code> |
| <code>pip</code> tidak ditemukan | pip belum terinstall | <code>sudo apt install python3-pip -y</code> |
| <code>git commit</code> gagal | Nama/email Git belum diset | Jalankan <code>git config --global user.name</code> dan <code>user.email</code> |
| <code>uv</code> tidak ditemukan | PATH belum aktif | Jalankan <code>source ~/.bashrc</code> atau buka terminal baru |
| Ollama tidak menjawab | Service belum berjalan | <code>sudo systemctl start ollama</code> |
| Model lokal berat | RAM/VRAM kurang | Gunakan model lebih kecil |
| Docker permission denied | User belum masuk grup docker | <code>sudo usermod -aG docker \$USER</code> , lalu login ulang |
| Android build gagal | Dependency atau konfigurasi belum siap | Baca error terakhir, jalankan ulang <code>./gradlew assembleDebug</code> |

E.13 Prompt AI untuk Mengecek Setup

Saya sedang menyiapkan laptop untuk belajar AI coding.

Kondisi saat ini:

- OS: [isi OS]
- Python: [hasil python3 --version]
- Git: [hasil git --version]
- uv: [hasil uv --version]
- Ollama: [hasil ollama --version]
- Docker: [hasil docker --version]
- Android build: [hasil ./gradlew assembleDebug]

Tolong:

1. cek komponen mana yang sudah siap,
2. jelaskan komponen yang belum siap,
3. berikan langkah perbaikan minimal,
4. jangan menyarankan tool tambahan yang tidak perlu,
5. jelaskan dengan bahasa pemula.

Golden Rule Lampiran E

Sebelum belajar coding cepat dengan AI, pastikan lingkungan kerja bersih, terminal berjalan, Python siap, Git aktif, AI lokal bisa dicoba, dan semua hasil bisa diulang dari CLI.

Lampiran F. Checklist Review Kode dari AI

Lampiran ini membantu pembaca mengecek kode yang dibuat atau diperbaiki oleh AI. Tujuannya sederhana: **jangan langsung percaya kode AI sebelum dibaca, dijalankan, diuji, dan dipahami.**

Prinsip utama:

Kode AI adalah draf awal. Programmer tetap bertanggung jawab atas hasil akhir.

F.1 Checklist Utama Review Kode AI

| Pertanyaan | Ya/Tidak |
|--|--------------------------|
| Apakah saya paham fungsi kode ini? | ☐ |
| Apakah kode sudah dijalankan? | ☐ |
| Apakah ada error? | ☐ |
| Apakah sudah diuji dengan input salah? | ☐ |
| Apakah ada risiko keamanan? | ☐ |
| Apakah library yang dipakai benar-benar ada? | ☐ |
| Apakah kode terlalu kompleks untuk pemula? | ☐ |
| Apakah sudah masuk Git? | ☐ |

F.2 Cara Memakai Checklist

Gunakan checklist ini setiap kali AI memberikan kode baru, memperbaiki error, menambahkan fitur, atau melakukan *refactoring*. Jangan lanjut ke fitur berikutnya sebelum kode saat ini benar-benar diperiksa.

Pola kerja yang disarankan:

Baca kode → Jalankan → Uji input benar → Uji input salah →
Cek keamanan → Commit

Jika ada satu jawaban yang masih “Tidak”, berarti kode belum siap dianggap selesai.

F.3 Pertanyaan Review yang Lebih Detail

| Area Review | Pertanyaan Tambahan | Status |
|---------------------|---|--------------------------|
| Pemahaman | Apakah saya bisa menjelaskan kode ini dengan kata-kata sendiri? | ☐ |
| Input | Apakah semua input penting sudah divalidasi? | ☐ |
| Output | Apakah output sesuai dengan kebutuhan awal? | ☐ |
| Error | Apakah pesan error mudah dipahami pengguna? | ☐ |
| Library | Apakah library yang digunakan memang tersedia dan terdokumentasi? | ☐ |
| Keamanan | Apakah ada data sensitif, token, password, atau API key di kode? | ☐ |
| Kompleksitas | Apakah kode bisa disederhanakan tanpa mengubah fungsi? | ☐ |
| Testing | Apakah sudah diuji dengan input normal, kosong, salah, dan ekstrem? | ☐ |
| Git | Apakah perubahan sudah di- <i>commit</i> dengan pesan jelas? | ☐ |

F.4 Checklist Review untuk Kode Python

| Pertanyaan | Ya/Tidak |
|--|--------------------------|
| Apakah file <code>.py</code> bisa dijalankan dari CLI? | ☐ |
| Apakah semua variabel memiliki nama yang jelas? | ☐ |
| Apakah fungsi terlalu panjang sudah dipisah? | ☐ |
| Apakah input pengguna sudah divalidasi? | ☐ |
| Apakah <code>try-except</code> digunakan pada bagian yang rawan error? | ☐ |
| Apakah file yang dibaca benar-benar ada? | ☐ |
| Apakah output sudah dicek manual? | ☐ |
| Apakah ada test sederhana? | ☐ |

Contoh perintah cek:

```
python3 main.py
```

F.5 Checklist Review untuk API FastAPI

| Pertanyaan | Ya/Tidak |
|--|--------------------------|
| Apakah server bisa jalan dengan <code>uvicorn</code> ? | ☐ |
| Apakah endpoint bisa dites dengan <code>curl</code> ? | ☐ |
| Apakah response berbentuk JSON yang benar? | ☐ |
| Apakah input kosong ditangani? | ☐ |
| Apakah ID yang tidak ditemukan ditangani? | ☐ |
| Apakah database tersimpan dengan benar? | ☐ |
| Apakah error tidak membocorkan informasi sensitif? | ☐ |
| Apakah semua endpoint penting sudah dites? | ☐ |

Contoh perintah cek:

```
uvicorn main:app --reload  
curl http://127.0.0.1:8000/todos
```

F.6 Checklist Review untuk Frontend

| Pertanyaan | Ya/Tidak |
|--|--------------------------|
| Apakah halaman bisa dibuka dari server lokal? | ☐ |
| Apakah tombol berfungsi? | ☐ |
| Apakah <code>fetch()</code> menuju URL API yang benar? | ☐ |
| Apakah data JSON berhasil ditampilkan? | ☐ |
| Apakah pesan error muncul jika backend mati? | ☐ |
| Apakah input kosong dicegah? | ☐ |
| Apakah kode JavaScript mudah dibaca? | ☐ |
| Apakah tampilan cukup jelas untuk pemula? | ☐ |

Contoh menjalankan frontend:

```
python3 -m http.server 8080
```

F.7 Checklist Review untuk Android / Kotlin

| Pertanyaan | Ya/Tidak |
|---|--------------------------|
| Apakah project bisa di- <i>build</i> dari CLI? | ☐ |
| Apakah <code>MainActivity.kt</code> mudah dibaca? | ☐ |
| Apakah <i>state</i> digunakan dengan benar? | ☐ |
| Apakah tombol menjalankan aksi yang benar? | ☐ |
| Apakah input kosong ditangani? | ☐ |
| Apakah UI tidak terlalu kompleks? | ☐ |
| Apakah error <i>build</i> sudah dipahami? | ☐ |
| Apakah APK debug berhasil dibuat? | ☐ |

Contoh build:

```
./gradlew assembleDebug
```


F.8 Checklist Risiko Keamanan Dasar

| Pertanyaan Keamanan | Ya/Tidak |
|---|--------------------------|
| Apakah tidak ada password di dalam kode? | ☐ |
| Apakah tidak ada API key di dalam kode? | ☐ |
| Apakah input pengguna divalidasi? | ☐ |
| Apakah error tidak menampilkan detail internal berlebihan? | ☐ |
| Apakah file penting tidak ikut <i>ter-commit</i> ? | ☐ |
| Apakah dependency berasal dari sumber yang jelas? | ☐ |
| Apakah kode tidak menjalankan perintah berbahaya tanpa kontrol? | ☐ |

Catatan penting: untuk pemula, keamanan dimulai dari hal sederhana: **jangan simpan rahasia di kode, validasi input, dan jangan menjalankan kode yang tidak dipahami.**

F.9 Prompt AI untuk Review Kode

Gunakan prompt ini setiap kali menerima kode dari AI.

Review kode berikut sebelum saya pakai.

Tolong cek:

1. apakah fungsi kode sesuai tujuan,
2. apakah ada bug dasar,
3. apakah input sudah divalidasi,
4. apakah ada risiko keamanan,
5. apakah library yang dipakai benar-benar ada,
6. apakah kode terlalu kompleks untuk pemula,
7. test apa yang harus saya lakukan,
8. perbaiki minimal apa yang disarankan.

Jangan menulis ulang seluruh kode kecuali benar-benar perlu.

Kode:

[paste kode di sini]

F.10 Prompt AI untuk Mengecek Library

Cek library yang dipakai dalam kode berikut.

Tolong jelaskan:

1. library apa saja yang digunakan,
2. apakah library tersebut umum dan terdokumentasi,
3. apakah ada alternatif bawaan Python yang lebih sederhana,
4. apakah library ini perlu diinstall,
5. perintah install yang benar,
6. risiko jika library tidak tersedia.

Kode:

[paste kode di sini]

F.11 Prompt AI untuk Menyederhanakan Kode

Kode berikut terlalu kompleks untuk pemula.

Tolong sederhanakan dengan syarat:

- fungsi utama tetap sama,
- jangan tambah fitur baru,
- jangan gunakan library tambahan,
- gunakan nama variabel yang jelas,
- jelaskan perubahan singkat,
- berikan test manual setelah disederhanakan.

Kode:

[paste kode di sini]

F.12 Perintah Git Setelah Kode Lolos Review

Jika kode sudah dibaca, dijalankan, diuji, dan dipahami, simpan ke Git:

```
git status
git add .
git commit -m "review dan simpan kode hasil bantuan AI"
```

Lihat riwayat:

```
git log --oneline
```

Golden Rule Lampiran F

Jangan pernah menganggap kode AI selesai sebelum paham fungsinya, menjalankannya, mengujinya, mengecek risikonya, dan menyimpannya dengan Git.

Glossary

| Istilah | Arti Singkat |
|------------------------------------|--|
| AI | <i>Artificial Intelligence</i> , teknologi yang dapat membantu manusia membuat, menjelaskan, memperbaiki, dan mengevaluasi kode. |
| AI coding assistant | AI yang membantu menulis, menjelaskan, memperbaiki, dan merapikan kode. |
| AI lokal | AI yang berjalan di komputer sendiri, misalnya memakai Ollama, sehingga lebih mandiri dan lebih privat. |
| Aider | Tool <i>AI pair programming</i> berbasis terminal yang dapat membantu mengedit kode dalam repo Git lokal. |
| Android app | Aplikasi yang berjalan pada perangkat Android. |
| Android client | Aplikasi Android yang berperan sebagai pengguna API, misalnya membaca data Todo dari backend. |
| API | Antarmuka agar aplikasi bisa saling berkomunikasi. |
| APK | File paket aplikasi Android yang dapat dipasang atau diuji pada perangkat Android. |
| Backend | Bagian aplikasi yang berjalan di server dan menangani logika, API, database, serta response. |
| Backup | Salinan data atau folder untuk mencegah kehilangan data penting. |
| Boolean / <code>bool</code> | Tipe data benar/salah, misalnya <code>True</code> atau <code>False</code> . |
| Build | Proses mengubah kode menjadi aplikasi yang bisa dijalankan, misalnya menghasilkan APK pada Android. |
| CLI | <i>Command Line Interface</i> , cara bekerja dengan komputer melalui perintah di terminal. |
| Code review | Proses membaca dan memeriksa kode untuk menemukan bug, risiko, atau bagian yang perlu dirapikan. |
| Commit | Titik simpan perubahan kode di Git. |
| Container | Lingkungan terisolasi untuk menjalankan aplikasi agar lebih konsisten dan tidak mengotori sistem utama. |
| CORS | Mekanisme keamanan browser yang mengatur apakah frontend boleh memanggil backend dari alamat berbeda. |

| | |
|---|--|
| CSV | Format file tabel sederhana, sering dipakai untuk data seperti nilai, transaksi, inventaris, atau log. |
| Curl | Perintah CLI untuk mengirim request HTTP, sering dipakai untuk menguji API. |
| Database | Tempat penyimpanan data aplikasi. Dalam buku ini digunakan SQLite untuk latihan. |
| DataFrame | Struktur data tabel pada pandas untuk membaca, mengolah, dan meringkas data. |
| Debugging | Proses mencari penyebab error dan memperbaiki masalah pada program. |
| Dependency | Package atau library tambahan yang dibutuhkan program agar bisa berjalan. |
| Dict / dictionary | Struktur data Python berbentuk pasangan <i>key-value</i> , sangat penting untuk memahami JSON dan API. |
| Docker | Tool untuk menjalankan aplikasi dalam container agar lingkungan lebih konsisten. |
| Docker Compose | Tool untuk menjalankan beberapa service sekaligus, misalnya backend, database, dan service pendukung. |
| Dokumentasi | Penjelasan tentang tujuan proyek, cara install, cara menjalankan, contoh penggunaan, dan masalah umum. |
| Editor kode | Aplikasi untuk menulis dan membaca kode, misalnya VSCodium, Geany, Kate, atau Neovim. |
| Emulator | Perangkat virtual untuk menjalankan dan menguji aplikasi Android tanpa perangkat fisik. |
| Endpoint | Alamat API tertentu, misalnya <code>/todos</code> , yang menerima request dan mengirim response. |
| Error | Pesan atau kondisi ketika program tidak berjalan sesuai harapan. Dalam buku ini error dipakai sebagai bahan belajar. |
| Error handling | Teknik menangani error agar program tidak langsung berhenti, misalnya memakai <code>try-except</code> . |
| FastAPI | Framework Python untuk membuat backend API modern dan cepat dipelajari. |
| Fetch API / <code>fetch()</code> | Fungsi JavaScript untuk mengirim request dari frontend ke backend dan membaca response. |

| | |
|---------------------------------|--|
| File .py | File kode Python yang dapat dijalankan dengan <code>python3 nama_file.py</code> . |
| Float | Tipe data angka desimal, misalnya <code>3.14</code> . |
| Frontend | Bagian aplikasi yang dilihat dan digunakan langsung oleh pengguna. |
| Fungsi / function | Blok kode yang diberi nama agar bisa dipakai ulang. Dalam Python dibuat dengan <code>def</code> . |
| Git | Sistem <i>version control</i> untuk mencatat perubahan kode dari waktu ke waktu. |
| Git init | Perintah untuk membuat repo Git baru pada folder proyek. |
| Git add | Perintah untuk menyiapkan file sebelum disimpan dalam commit. |
| Git status | Perintah untuk melihat kondisi perubahan file dalam repo Git. |
| Gradle Wrapper / gradlew | Tool bawaan project Android untuk menjalankan proses build dari CLI. |
| Hallucination | Jawaban AI yang tampak meyakinkan tetapi salah. |
| HTML | Bahasa markup untuk membuat struktur halaman web. |
| HTTP request | Permintaan dari client ke server, misalnya <code>GET</code> , <code>POST</code> , <code>PUT</code> , atau <code>DELETE</code> . |
| HTTP response | Jawaban server terhadap request, sering kali berbentuk JSON. |
| IDE / editor terpadu | Lingkungan kerja untuk menulis, menjalankan, dan membaca kode. Dalam buku ini cukup dipahami sebagai alat bantu, bukan pusat pembelajaran. |
| Input | Data yang dimasukkan pengguna atau dikirim ke program. |
| Integer / int | Tipe data bilangan bulat, misalnya <code>10</code> atau <code>25</code> . |
| JavaScript | Bahasa pemrograman yang dipakai di frontend untuk interaksi, tombol, form, dan pemanggilan API. |
| Jetpack Compose | Toolkit modern untuk membuat UI Android dengan Kotlin. |
| JSON | Format data ringan untuk komunikasi antar aplikasi. |
| Kotlin | Bahasa pemrograman yang digunakan untuk membuat aplikasi Android modern. |

| | |
|---------------------------|---|
| Lambda | Fungsi pendek, sering dipakai pada aksi seperti klik tombol di Android/Compose. |
| LazyColumn | Komponen Jetpack Compose untuk menampilkan daftar data secara efisien. |
| Library | Kumpulan kode siap pakai yang bisa digunakan oleh program. |
| Linux/Ubuntu | Sistem operasi yang digunakan sebagai contoh utama untuk belajar coding, terminal, Python, Git, Docker, dan AI lokal. |
| List | Struktur data Python untuk menyimpan banyak nilai berurutan. |
| Logika bisnis | Aturan utama aplikasi, misalnya kapan Todo dianggap selesai atau bagaimana data dihitung. |
| Markdown | Format teks ringan untuk menulis dokumentasi seperti README.md . |
| Model AI lokal | Model AI yang dijalankan di komputer sendiri melalui tool seperti Ollama. |
| Ollama | Tool untuk menjalankan model AI lokal di komputer sendiri. |
| Open-source | Perangkat lunak dengan kode sumber terbuka, dapat dipelajari, digunakan, dan dikembangkan komunitas. |
| Output | Hasil yang dikeluarkan program. |
| Package | Paket kode yang dapat diinstall untuk menambah kemampuan program. |
| Pair programmer | Teman coding yang membantu berpikir, memberi masukan, membaca error, atau membuat draf kode. Dalam buku ini AI diposisikan sebagai <i>pair programmer</i> , bukan pengganti programmer. |
| pandas | Library Python untuk mengolah data tabel, misalnya CSV menjadi DataFrame. |
| Path | Lokasi file atau folder dalam sistem. |
| pip | Tool untuk menginstall package Python. |
| Prompt | Instruksi yang diberikan ke AI. |
| Prompt engineering | Teknik menyusun prompt agar AI memberi jawaban yang lebih jelas, relevan, dan mudah diuji. |
| Pytest | Framework testing Python yang dipakai untuk menulis dan menjalankan test sederhana. |
| Python | Bahasa pemrograman utama dalam buku ini karena mudah dibaca, cocok untuk pemula, otomasi, AI, data, dan web API. |

| | |
|----------------------------------|---|
| README.md | File dokumentasi utama proyek yang menjelaskan tujuan, cara install, cara menjalankan, struktur folder, dan masalah umum. |
| Refactoring | Merapikan kode tanpa mengubah fungsi utamanya. |
| Repo / repository | Folder proyek yang dikelola oleh Git. |
| Request body | Data yang dikirim ke backend, misalnya JSON saat menambah Todo. |
| REST Client | Tool untuk menguji API dengan request HTTP. |
| Route | Jalur URL dalam aplikasi web/API, misalnya <code>/todos</code> . |
| SQLite | Database ringan yang cocok untuk latihan dan proyek kecil karena tidak memerlukan server terpisah. |
| State | Data yang berubah dan memengaruhi tampilan aplikasi, penting dalam frontend dan Android Compose. |
| String / <code>str</code> | Tipe data teks, misalnya <code>"Belajar Python"</code> . |
| Test manual | Pengujian dengan mencoba program langsung dari CLI, browser, atau API. |
| Test input kosong | Pengujian untuk melihat apakah program aman saat pengguna tidak mengisi data. |
| Test input salah | Pengujian untuk melihat apakah program tahan terhadap input yang tidak sesuai. |
| Testing | Proses memastikan program bekerja sesuai harapan. Bab 9 menekankan bahwa kode AI harus diuji sebelum dipercaya. |
| Todo | Contoh aplikasi daftar tugas yang dipakai dalam buku untuk latihan backend, frontend, Android, dan proyek akhir. |
| Traceback | Jejak error Python yang menunjukkan file, baris, dan tipe error. |
| Try-except | Struktur Python untuk menangani error agar program tidak langsung berhenti. |
| Tuple | Struktur data Python mirip list, tetapi biasanya digunakan untuk data yang tidak berubah. |
| Unit test | Test kecil untuk memeriksa fungsi tertentu. |
| uv | Tool modern untuk manajemen package dan proyek Python. |
| Uvicorn | Server ASGI yang biasa dipakai untuk menjalankan aplikasi FastAPI. |
| Validasi input | Proses mengecek apakah input pengguna benar, kosong, salah format, atau berbahaya. |

| | |
|----------------------------|---|
| venv | <i>Virtual environment</i> Python untuk memisahkan package proyek dari sistem utama. |
| Version control | Sistem untuk mencatat riwayat perubahan kode, seperti Git. |
| Vibe coding | Cara membuat software dengan memberi instruksi natural kepada AI lalu membiarkan AI menghasilkan kode awal; cepat, tetapi harus diuji dan direview. |
| Virtual environment | Lingkungan Python terpisah agar dependency proyek tidak bercampur dengan sistem. |
| Web API | API yang diakses melalui protokol web/HTTP. |
| Web app | Aplikasi yang berjalan melalui browser dan terhubung ke backend. |
| Website | Halaman web yang dapat dibuka melalui browser. |

Golden rule Glossary:

Istilah teknis tidak perlu dihafal sekaligus. Pahami istilah saat dipakai dalam praktik: jalankan kode, lihat error, baca JSON, test API, lalu simpan hasil dengan Git.

Referensi

Aider. (n.d.). *Aider documentation: AI pair programming in your terminal.*

<https://aider.chat/docs/>

Alam, A. (2023). *Modern artificial intelligence with zero coding: Build 3 practical projects & harness the power of AI to solve practical, real-world business problems with zero coding!*

Android Developers. (n.d.). *Get started with Jetpack Compose.*

<https://developer.android.com/develop/ui/compose/documentation>

Android Developers. (n.d.). *Kotlin and Android.* <https://developer.android.com/kotlin>

Android Developers. (n.d.). *Learn the Kotlin programming language.*

<https://developer.android.com/kotlin/learn>

Android Developers. (n.d.). *State and Jetpack Compose.*

<https://developer.android.com/develop/ui/compose/state>

Astral. (n.d.). *uv: An extremely fast Python package and project manager.*

<https://docs.astral.sh/uv/>

Continue. (n.d.). *Continue documentation.* <https://docs.continue.dev/>

Continue. (n.d.). *Ollama guide: Local AI development.*

<https://docs.continue.dev/guides/ollama-guide>

Crocker, N. B. (2023). *The AI-powered developer* (MEAP version 2). Manning Publications.

<https://www.manning.com/>

Davies, C. (2023). *Culture coding: Harness technology, artificial intelligence and behavioural science to empower your business culture and performance.* Rethink Press.

<https://www.rethinkpress.com/>

Docker. (n.d.). *Docker Compose documentation.* <https://docs.docker.com/compose/>

Docker. (n.d.). *Docker documentation.* <https://docs.docker.com/>

Docker. (n.d.). *Install Docker Engine on Ubuntu.*

<https://docs.docker.com/engine/install/ubuntu/>

FastAPI. (n.d.). *FastAPI documentation.* <https://fastapi.tiangolo.com/>

FastAPI. (n.d.). *CORS middleware.* <https://fastapi.tiangolo.com/tutorial/cors/>

FastAPI. (n.d.). *First steps.* <https://fastapi.tiangolo.com/tutorial/first-steps/>

Git. (n.d.). *Git: Distributed version control system.* <https://git-scm.com/>

Git. (n.d.). *Git reference documentation.* <https://git-scm.com/docs>

Git. (n.d.). *Pro Git book*. <https://git-scm.com/book/en/v2>

Goel, V. (2024). *Software programming basics: AI-generated step-by-step guide to help you start your software programming journey with ease*. Notion Press.

Gradle. (n.d.). *Gradle wrapper*.

https://docs.gradle.org/current/userguide/gradle_wrapper.html

Henstock, P. V., & Herszfang, H. P. (2025). *Supercharged coding with GenAI: From vibe coding to best practices using GitHub Copilot, ChatGPT, and OpenAI*. Packt Publishing.

HTMX. (n.d.). *HTMX documentation*. <https://htmx.org/docs/>

Kofler, M., Öggl, B., & Springer, S. (2025). *AI-assisted coding: The practical guide for software development*. Rheinwerk Publishing.

Kotlin. (n.d.). *Kotlin documentation*. <https://kotlinlang.org/docs/home.html>

Kotlin. (n.d.). *Kotlin for Android*. <https://kotlinlang.org/docs/android-overview.html>

MDN Web Docs. (n.d.). *CSS: Cascading Style Sheets*.

<https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs. (n.d.). *Fetch API*.

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

MDN Web Docs. (n.d.). *HTML: HyperText Markup Language*.

<https://developer.mozilla.org/en-US/docs/Web/HTML>

MDN Web Docs. (n.d.). *JavaScript*. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

Mertz, D. Q. (2023). *Regular expression puzzles and AI coding assistants: 24 puzzles solved by the author, with and without assistance from Copilot, ChatGPT and more*. Manning Publications.

Minnick, C. (2024). *Coding with AI for dummies*. John Wiley & Sons. <https://www.wiley.com/>

Ollama. (n.d.). *Ollama API documentation*. <https://docs.ollama.com/api/introduction>

Ollama. (n.d.). *Ollama FAQ*. <https://docs.ollama.com/faq>

Ollama. (n.d.). *Ollama Linux documentation*. <https://docs.ollama.com/linux>

Ollama. (n.d.). *Ollama quickstart*. <https://docs.ollama.com/quickstart>

Osmani, A. (2025). *Beyond vibe coding: From coder to AI-era developer*. O'Reilly Media.

OWASP Foundation. (n.d.). *Secure coding practices quick reference guide*.

<https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

pandas. (n.d.). *pandas.DataFrame.groupby*.
<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html>

pandas. (n.d.). *pandas.read_csv*.
https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

pytest. (n.d.). *pytest documentation: Get started*.
<https://docs.pytest.org/en/stable/getting-started.html>

Python Software Foundation. (n.d.). *csv — CSV file reading and writing*.
<https://docs.python.org/3/library/csv.html>

Python Software Foundation. (n.d.). *Errors and exceptions*.
<https://docs.python.org/3/tutorial/errors.html>

Python Software Foundation. (n.d.). *The Python tutorial*.
<https://docs.python.org/3/tutorial/index.html>

Python Software Foundation. (n.d.). *unittest — Unit testing framework*.
<https://docs.python.org/3/library/unittest.html>

Python Software Foundation. (n.d.). *venv — Creation of virtual environments*.
<https://docs.python.org/3/library/venv.html>

Python Software Foundation. (n.d.). *sqlite3 — DB-API 2.0 interface for SQLite databases*.
<https://docs.python.org/3/library/sqlite3.html>

Requests. (n.d.). *Requests: HTTP for humans*. <https://requests.readthedocs.io/>

Requests. (n.d.). *Requests quickstart*.
<https://requests.readthedocs.io/en/latest/user/quickstart/>

SQLite. (n.d.). *About SQLite*. <https://sqlite.org/about.html>

SQLite. (n.d.). *SQLite documentation*. <https://sqlite.org/docs.html>

SQLite. (n.d.). *SQLite home page*. <https://sqlite.org/>

Ubuntu. (n.d.). *Python setup on Ubuntu*.
<https://documentation.ubuntu.com/ubuntu-for-developers/howto/python-setup/>

Wienholt, N. (2025). *GitHub Copilot and AI coding tools in practice: Accelerate AI adoption from individual developers to enterprise*. Apress. <https://doi.org/10.1007/979-8-8688-1784-7>

Tentang Penulis



Onno W. Purbo, saat ini bertugas sebagai rektor di Institut Teknologi Tangerang Selatan (ITTS). Onno memperoleh gelar Ph.D bidang Electrical Engineering dari University of Waterloo, Canada, adalah seorang copyleft, educator dan ICT evangelist. Dia sudah mempublikasikan 50++ buku, termasuk free ICT ebook untuk sekolah tahun 2008. Beberapa buku terakhirnya adalah "Internet-TCP/IP: Konsep Dan Implementasi", 2018; "Sistem Operasi, Konsep Dan Membuat Linux OpenWRT Dan ROM Android", 2019; "IPv6 Untuk Mendukung Operasi Jaringan Dan Domain Name System", 2019; "Kubernetes untuk Pemula", 2024 dan "Membuat Operator Seluler 5G Sendiri", 2024.

Dia memimpin sambungan pertama Internet di Institut Teknologi Bandung, tahun 1993-2000, dan menggunakannya untuk membuat jaringan Internet pendidikan yang pertama di Indonesia. Dia membebaskan frekuensi WiFi, memperkenalkan RT/RW-net, antenna Wajanbolic dan jaringan selular OpenBTS dan private 5G sendiri. Dia memimpin jaringan telepon pertama di atas Internet, VoIP Merdeka, yang kemudian hari dikenal sebagai VoIP Rakyat berbasis SIP dan menggunakan kode area +62520 dan +62521. Dia saat ini aktif memperkenalkan e-Learning, dan menjalankan server e-Learning di <http://lms.onnocenter.or.id/moodle/> 66,000++ siswa / mahasiswa dan <https://opencourse.itts.ac.id> 39.000+ mahasiswa secara gratis. Email: onno@indo.net.id